

Tema 3: Creación de componentes visuales

Android 

 Kotlin

OBJETIVOS

- ✓ Crear componentes visuales mediante el uso de herramientas específicas.
- ✓ Identificar las herramientas para diseño y prueba de componentes.
- ✓ Crear componentes visuales, definir las propiedades y asignar valores por defecto.
- ✓ Determinar los eventos a los que debe responder o componente y asociarles las acciones correspondientes.
- ✓ Realizar pruebas unitarias y documentar los componentes creados.



3.1. Entorno de Desarrollo



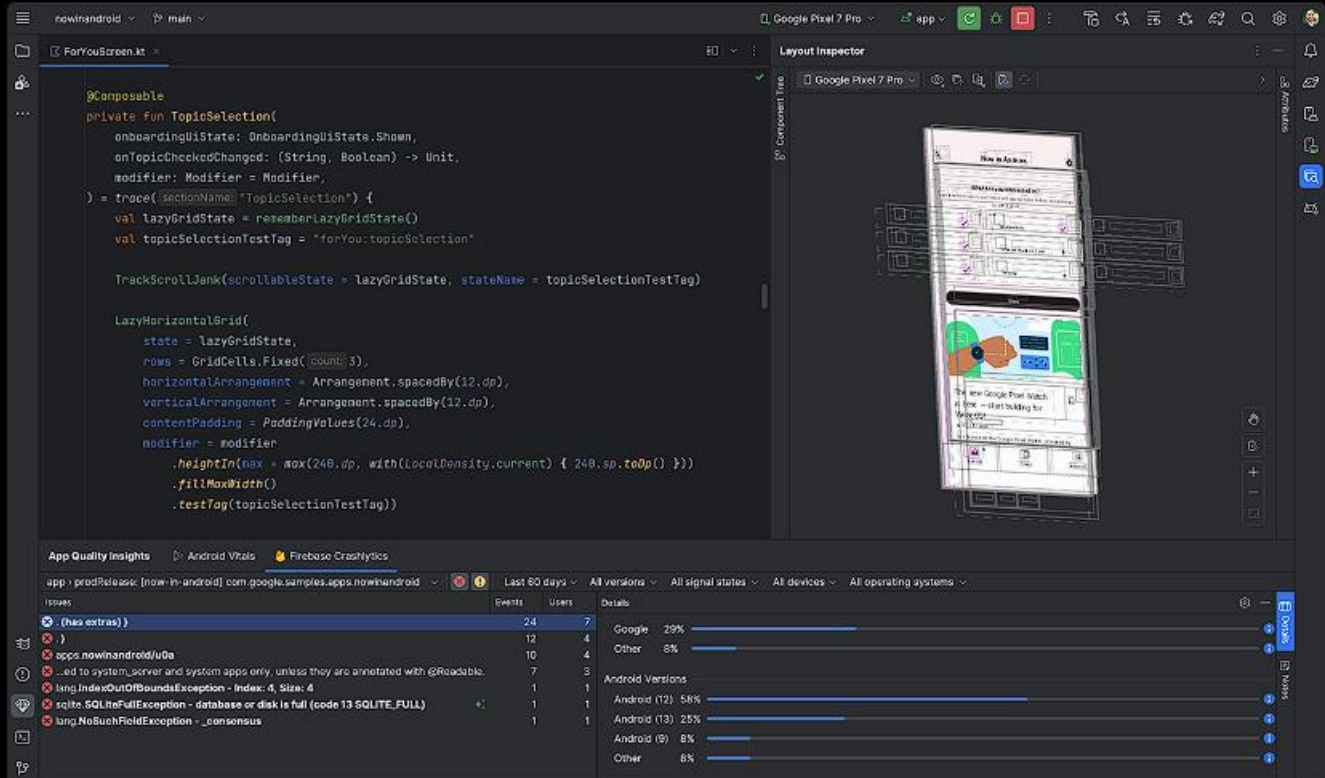
Instalación de Android Studio

Android Studio

Obtén el entorno de desarrollo integrado (IDE) oficial del desarrollo de apps para Android.

Descargar Android Studio Hedgehog 

Leer las notas de la versión 



```
@Composable
private fun TopicSelection(
    onboardingUiState: OnboardingUiState.Shown,
    onTopicCheckedChanged: (String, Boolean) -> Unit,
    modifier: Modifier = Modifier,
) = trace(sectionName = "TopicSelection") {
    val lazyGridState = rememberLazyGridState()
    val topicSelectionTestTag = "forYou:topicSelection"

    TrackScrollJank(scrollableState = lazyGridState, stateName = topicSelectionTestTag)

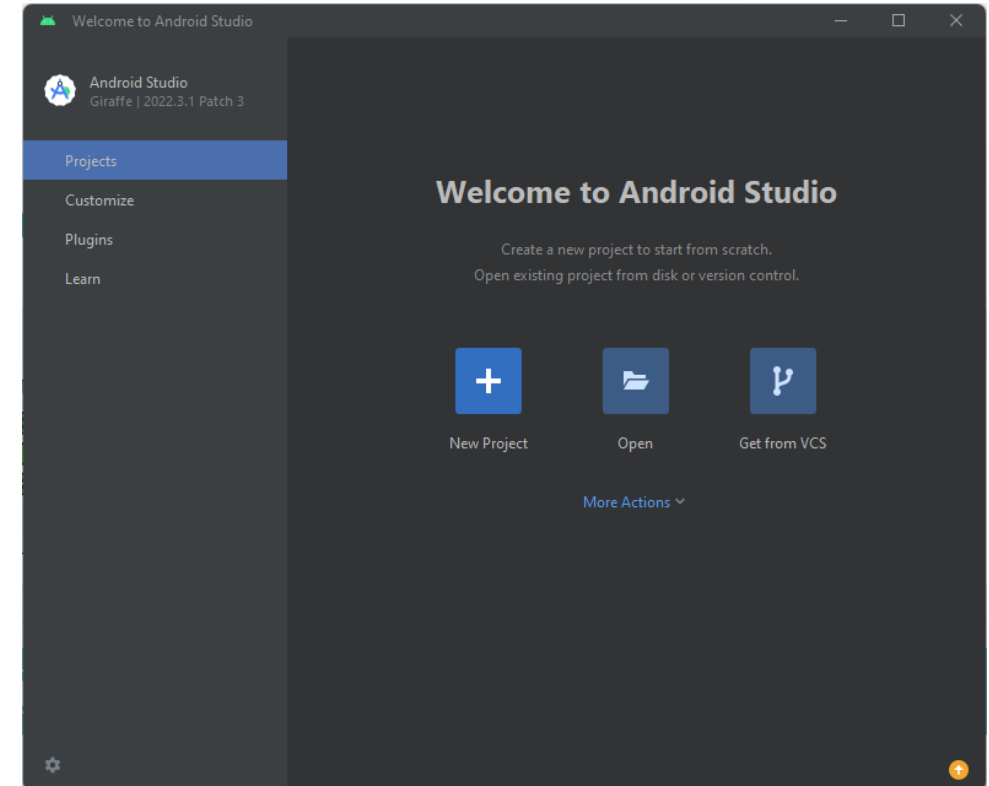
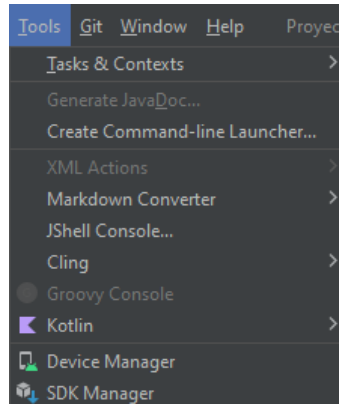
    LazyHorizontalGrid(
        state = lazyGridState,
        rows = GridCells.Fixed(3),
        horizontalArrangement = Arrangement.spacedBy(12.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp),
        contentPadding = PaddingValues(24.dp),
        modifier = modifier
    ) {
        heightIn(max = max(240.dp, with(LocalDensity.current) { 240.sp.toDp() }))
        fillMaxWidth()
        testTag(topicSelectionTestTag)
    }
}
```

[Descarga Android Studio y App Tools - Android Developers](#) | [Android Studio](#)

Instalación de Android Studio

Hay que seguir los pasos que la propia aplicación nos pide. Siguiente, siguiente. Nos informará de que estamos instalando Android Studio y después nos pedirá directamente descargar el Android Emulator, el Android SDK y varios componentes del SDK adicionales. Y finalmente llegamos a la ventana de INICIO

(Posteriormente, deberemos ir descargando los paquetes adicionales de SDK que vayamos necesitando a través del SDK Manager).



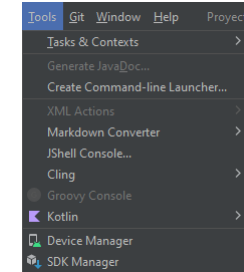
Descripción de Android Studio

Android Studio es el entorno de desarrollo integrado (IDE) oficial que se usa en el desarrollo de apps para Android. Basado en el potente editor de código y las herramientas para desarrolladores de [IntelliJ IDEA](#), Android Studio ofrece aún más funciones que mejoran tu productividad cuando compilas apps para Android, como las siguientes:

- ☐ Un sistema de compilación flexible basado en Gradle
- ☐ Un emulador rápido y cargado de funciones
- ☐ Un entorno unificado donde puedes desarrollar para todos los dispositivos Android
- ☐ Ediciones en vivo para actualizar elementos componibles en emuladores y dispositivos físicos, en tiempo real
- ☐ Integración con GitHub y plantillas de código para ayudarte a compilar funciones de apps comunes y también importar código de muestra
- ☐ Variedad de marcos de trabajo y herramientas de prueba

Instalación de Android Studio

SDK MANAGER



Primero, un **software development kit (SDK)** o kit de desarrollo de software es un conjunto de herramientas de software y programas utilizados por los desarrolladores para crear aplicaciones para plataformas específicas.

Las herramientas de un SDK pueden incluir una variedad de cosas, por ejemplo, bibliotecas, documentaciones, ejemplos de código, procesos y guías que los desarrolladores pueden usar e integrar en sus propias aplicaciones.

Los SDK están diseñados para usarse en plataformas o lenguajes específicas de programación. En consecuencia, se necesitará un kit de herramientas del **SDK de Android** para crear una aplicación de Android. Se necesita un **SDK de iOS** para crear una aplicación de iOS. Por lo tanto, tendrá que identificar el SDK adecuado a la plataforma o lenguaje de programación.

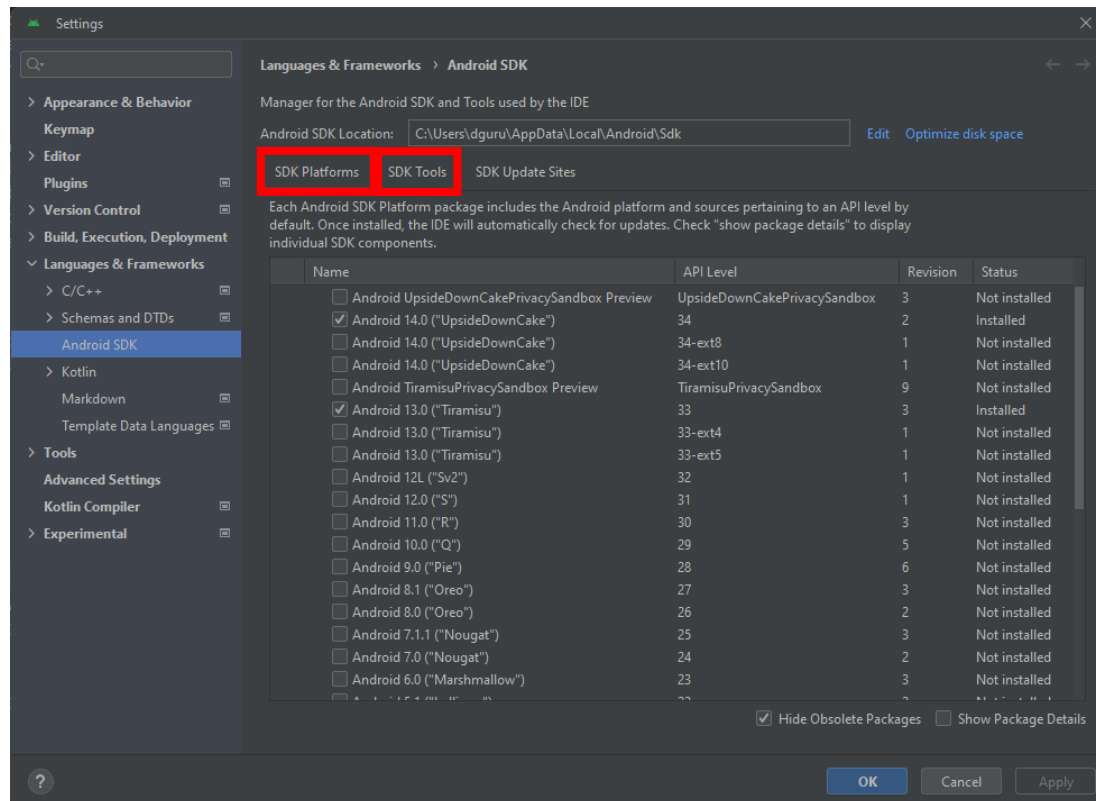
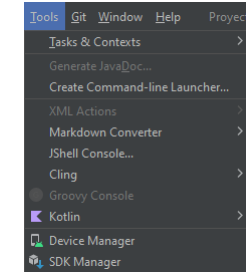
SDK vs. API: ¿cuál es la diferencia?

Una interfaz de programación de aplicaciones (API, por sus siglas en inglés), básicamente, es un código que permite una comunicación claramente definida entre dos aplicaciones separadas. Un ejemplo sencillo: es la API que permite la comunicación entre Google Maps y Uber.es. SDK puede contener una o más API y más utilidades esenciales. La API es solo una parte de un SDK. El SDK es como un “contenedor” más grande para una gama completa de herramientas SDK.

Instalación de Android Studio

SDK MANAGER (Tools → SDK Manager)

Una vez aquí deberemos seleccionar el **Android SDK Platform** y las APIs de la versión de Android para la que queramos programar.



Cuando instalamos el SDK de Android también deberíamos seleccionar y descargar herramientas como el **Android SDK Tools**, el **Android SDK Platform-tools** y la última versión del **Android SDK Build-tools**, todos ellos imprescindibles cuando queramos instalar ROMs e imágenes de fábrica como las de las preview de Android.

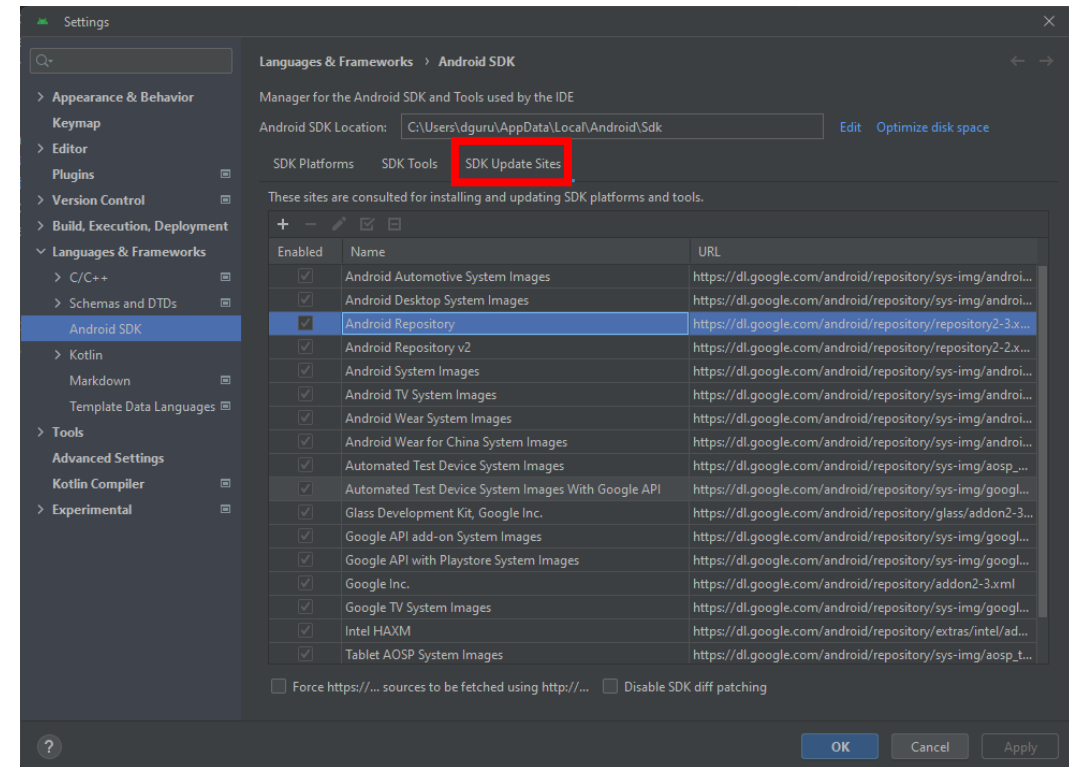
Además de esto, tenemos varios directorios extra relacionados con **las API de Google** enfocadas a los sistemas operativos Android para accesorios. Nos referimos a Wear OS, Google Cast o Android TV. También cuentan con sus propias API y son muy recomendadas.

Instalación de Android Studio

SDK MANAGER (Tools → SDK Manager)

Además, en la pestaña de **SDK Update Sites**, tienen que estar habilitadas las **Android Repository** y **Android Support** son dos librerías muy útiles que nos dan una gran cantidad de APIs compatibles con la mayoría de versiones de Android y si queremos desarrollar aprovechando los **Google Play Services** deberemos descargar el **Google Repository**.

Tener el SDK de Android instalado es el primer paso para poder añadir ROMs, configurar OTAs e instalar imágenes de fábrica a nuestro Android. El SDK es la puerta de entrada al mundo de las ROMs, las versiones actualizadas, las preview de Android O y en definitiva, todo lo que conlleva modificar el software predefinido de un móvil y cambiarlo por otro.



Instalación de Android Studio

Las versiones de Android

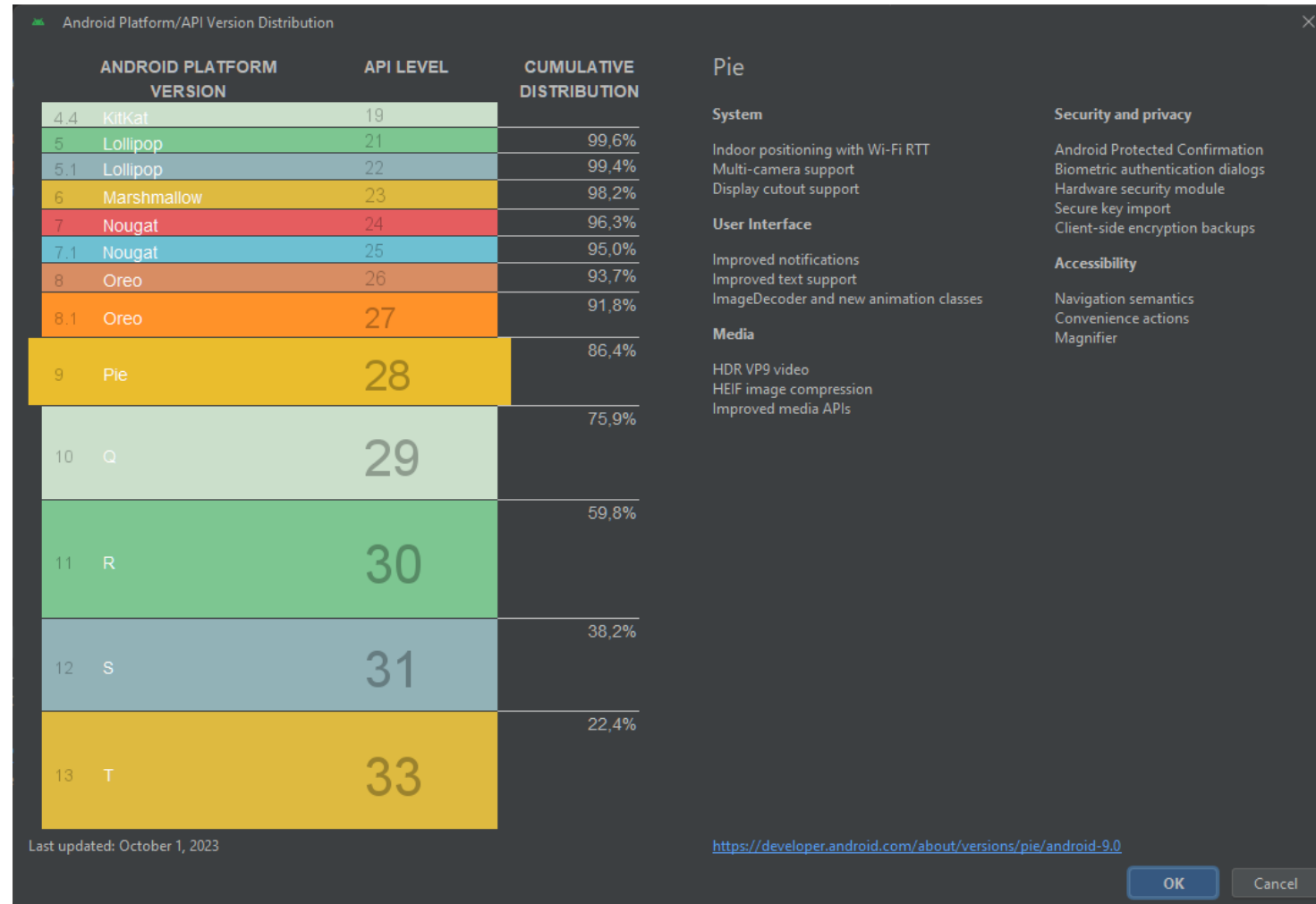
Antes de empezar a programar en Android hay que elegir la **versión del sistema** para la que deseamos realizar la aplicación. Es muy importante observar que hay clases y métodos que están disponibles a partir de una versión; si las vamos a usar, hemos de conocer la versión mínima necesaria.

Cuando se ha lanzado una nueva plataforma, siempre ha sido compatible con las versiones anteriores. Es decir, solo se añaden nuevas funcionalidades, y en el caso de modificar alguna funcionalidad, no se elimina, sino que se etiqueta como obsoleta, pero se puede continuar utilizando.

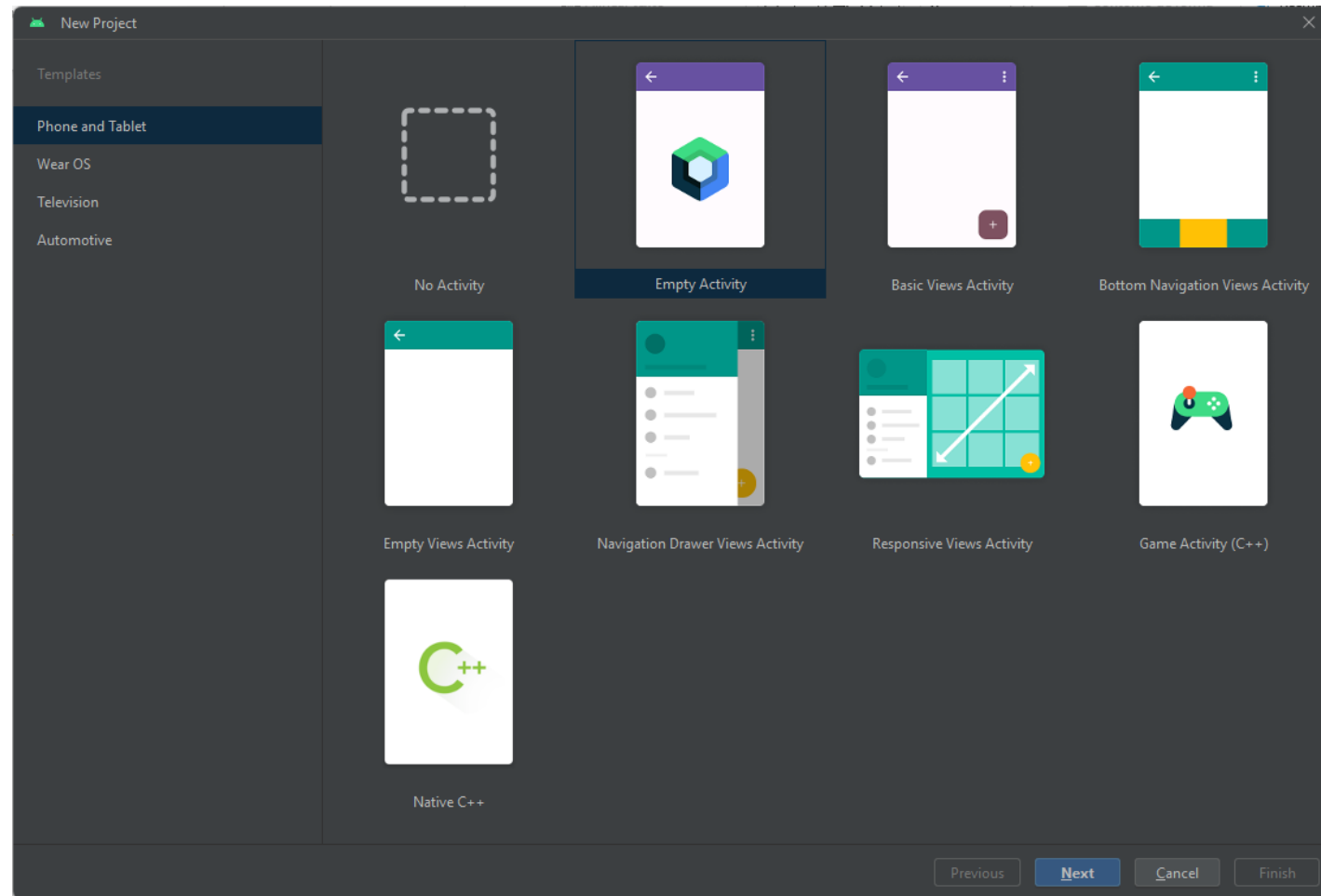
Las plataformas lanzadas hasta la fecha **se identifican de tres formas alternativas** (ha ido cambiando en su evolución): nombre comercial (código de versión), versión de la plataforma y nivel de API. El nivel de API corresponde a números enteros, comenzando desde 1. Para los nombres comerciales se han elegido postres en orden alfabético: Apple Pie (Android 1, v1, API 1, 2008) ,Cupcake (Android Cupcake, v1.5, API 3, 2009), ..., Tiramisú (**Android 13**, v13, API 33, 2022), Upside Down Cake (**Android 14**, v14, API 34, 2023).



Instalación de Android Studio

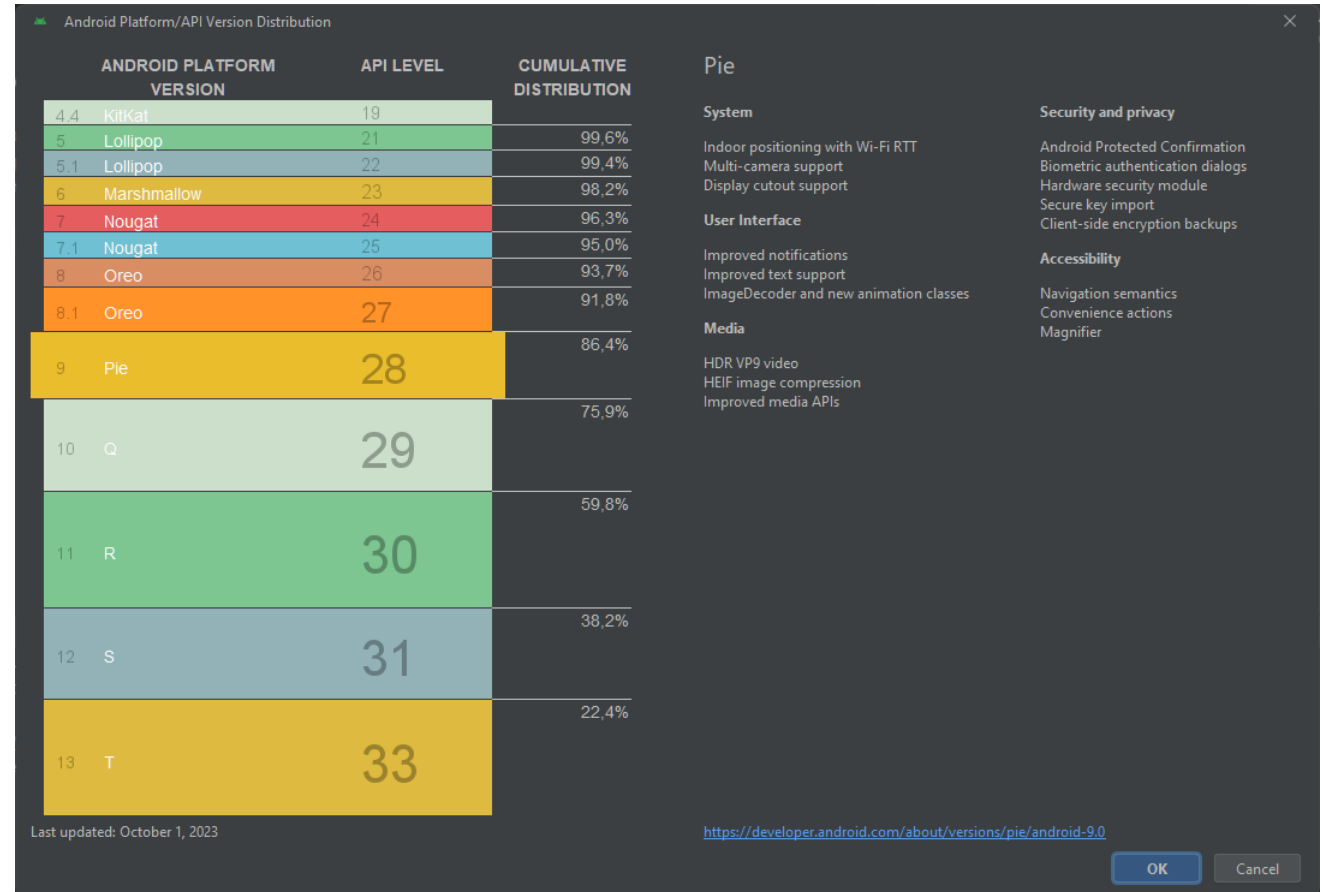


Nuevo proyecto



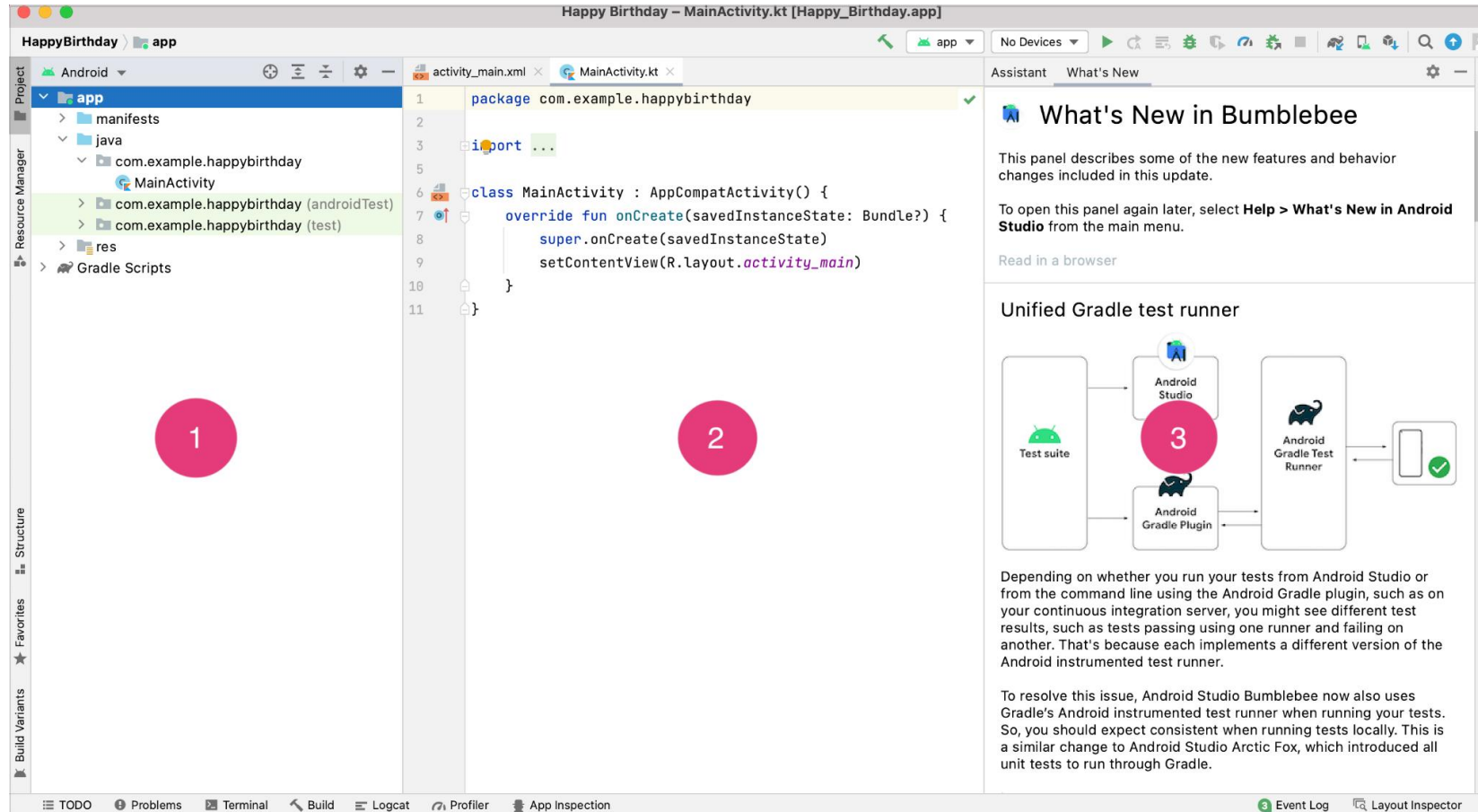
Nuevo proyecto

Consulta la nota debajo de Minimum SDK, que informa la cantidad de dispositivos en los que se puede ejecutar tu app con el nivel de API elegido. Si te interesa, haz clic en el vínculo Help me choose para ver una lista de diferentes versiones de Android, como se muestra a continuación. Luego, vuelve a la ventana New Project.



Nuevo proyecto

Android Studio abrirá el proyecto y todos sus archivos.



Cuando abras Android Studio por primera vez, verás tres ventanas:

(1) En la ventana Project, se muestran los archivos y las carpetas de tu proyecto.

(2) En la ventana Editing, se edita el código.

(3) En la ventana What's New, encontrarás novedades y sugerencias útiles. Aquí también verás lo relacionado con los dispositivos virtuales

IU de Android Studio

Desarrollo de aplicaciones multiplataforma

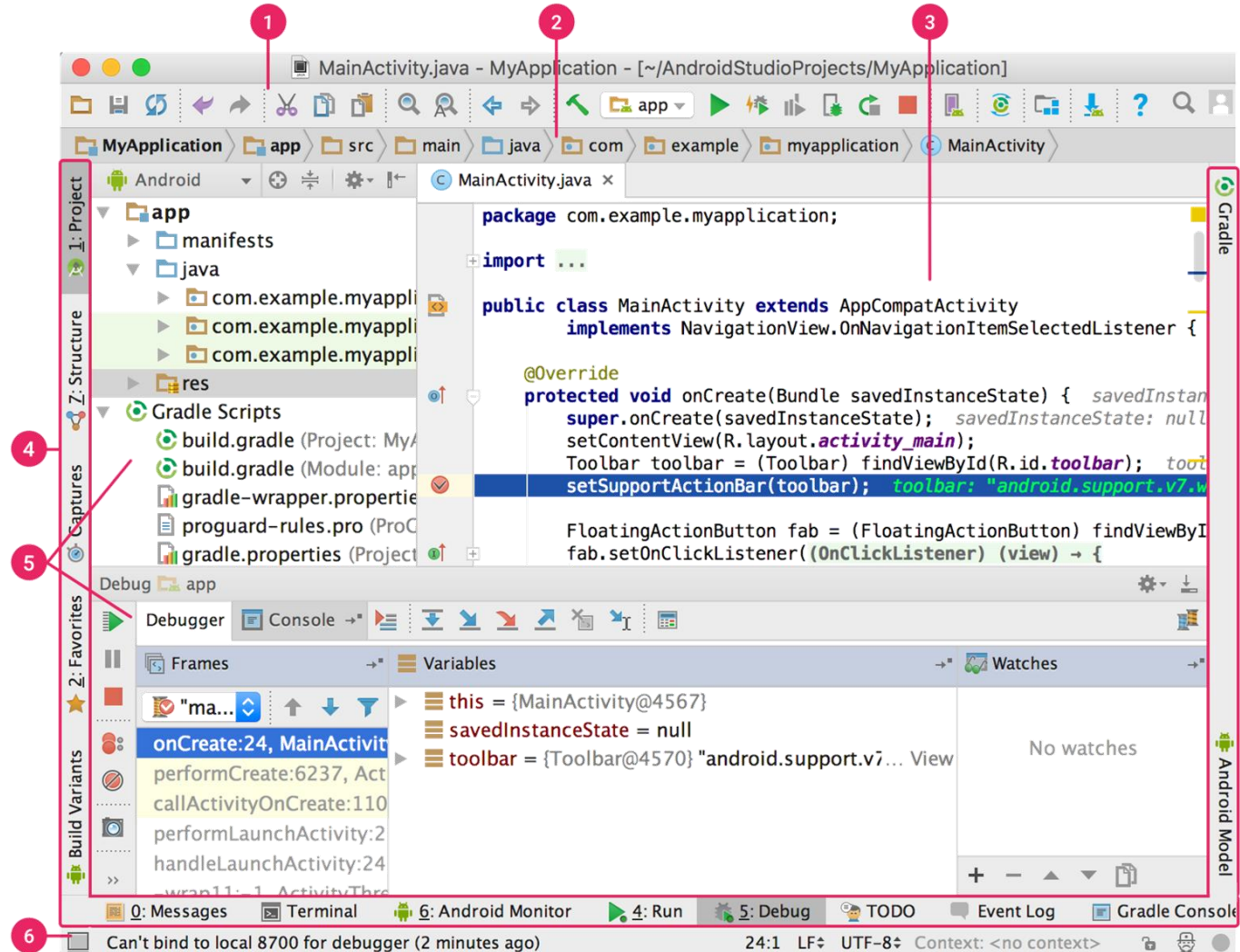
Desarrollo de Interfaces

Tema 3: Creación de componentes visuales

IU de Android Studio

La ventana principal de Android Studio consta de varias áreas lógicas que se muestran en la figura anterior

- 1 **Barra de herramientas:** Te permite realizar una gran variedad de acciones, como ejecutar tu app, además de iniciar las herramientas de Android.
- 2 **Barra de navegación:** Te ayuda a explorar tu proyecto y abrir archivos para editar. Proporciona una vista más compacta de la estructura visible en la ventana **Project**.
- 3 **Ventana del editor:** Permite crear y modificar el código. Según el tipo de actividad actual, el editor puede cambiar. Por ejemplo, cuando ves un archivo de diseño, el editor muestra el Editor de diseño.
- 4 **Barra de ventanas de herramientas:** Usa los botones en el exterior de la ventana del IDE para expandir o contraer ventanas de herramientas individuales.
- 5 **Ventanas de herramientas:** Brinda acceso a tareas específicas, como la administración de proyectos, la búsqueda, el control de versiones, entre otras. Puedes expandirlas y contraerlas.
- 6 **Barra de estado:** Muestra el estado de tu proyecto y el IDE, además de advertencias o mensajes.



IU de Android Studio

La ventana principal de Android Studio consta de varias áreas lógicas que se muestran en la figura anterior

- 1 **Barra de herramientas:** Te permite realizar una gran variedad de acciones, como ejecutar tu app, además de iniciar las herramientas de Android.
- 2 **Barra de navegación:** Te ayuda a explorar tu proyecto y abrir archivos para editar. Proporciona una vista más compacta de la estructura visible en la ventana **Project**.
- 3 **Ventana del editor:** Permite crear y modificar el código. Según el tipo de actividad actual, el editor puede cambiar. Por ejemplo, cuando ves un archivo de diseño, el editor muestra el Editor de diseño.
- 4 **Barra de ventanas de herramientas:** Usa los botones en el exterior de la ventana del IDE para expandir o contraer ventanas de herramientas individuales.
- 5 **Ventanas de herramientas:** Brinda acceso a tareas específicas, como la administración de proyectos, la búsqueda, el control de versiones, entre otras. Puedes expandirlas y contraerlas.
- 6 **Barra de estado:** Muestra el estado de tu proyecto y el IDE, además de advertencias o mensajes.

IU de Android Studio

Para **organizar la ventana principal** de modo que haya más espacio en pantalla, oculta o mueve las barras y las ventanas de herramientas. También puedes usar combinaciones de teclas para acceder a la mayoría de las funciones del IDE.

Para **buscar** en el código fuente, las bases de datos, las acciones, los elementos de la interfaz de usuario y mucho más, realiza una de las siguientes acciones:

- Presiona dos veces la tecla Mayúsculas.
- Haz clic en la lupa que se encuentra en la esquina superior derecha de la ventana de Android Studio.

Esto puede ser muy útil, por ejemplo, si intentas localizar una acción específica del IDE que olvidaste cómo activar.

Combinaciones de teclas

Ventana de herramientas	Windows y Linux	macOS
Proyecto	Alt + 1	Comando + 1
Control de versiones	Alt + 9	Comando + 9
Ejecutar	Mayús + F10	Control + R
Cómo depurar	Mayús + F9	Control + D
Logcat	Alt + 6	Comando + 6
Volver al editor	Esc	Esc
Ocultar todas las ventanas de herramientas	Control + Mayús + F12	Comando + Mayús + F12

Cómo completar el código

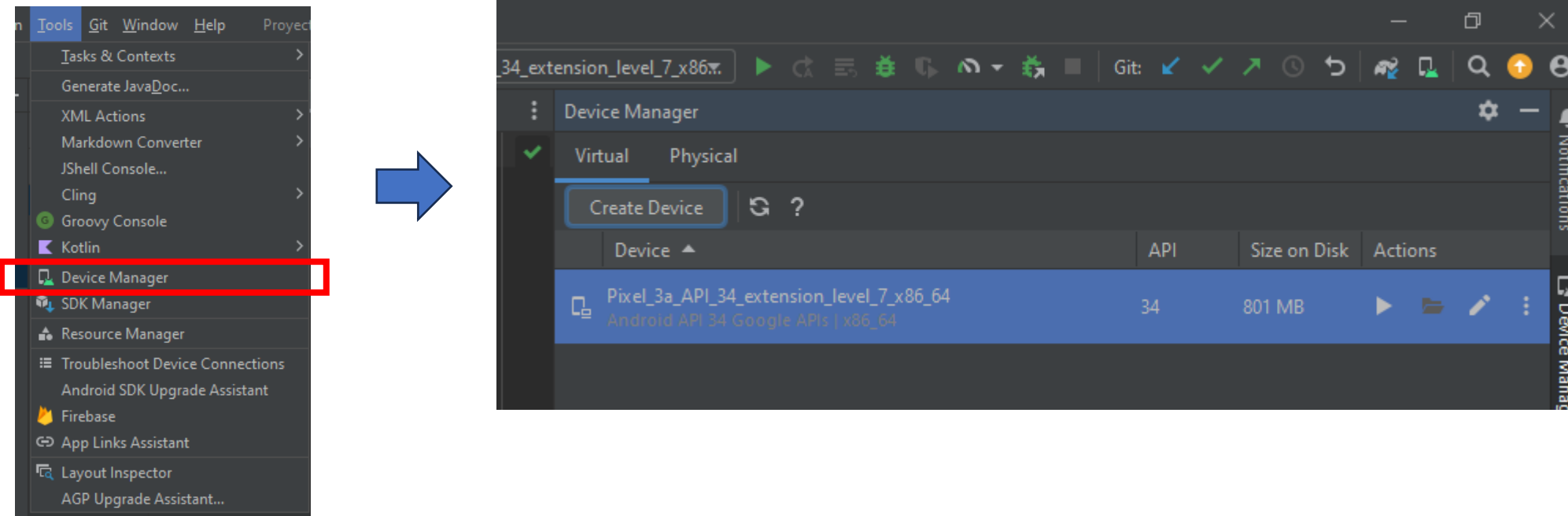
Android Studio ofrece tres opciones para completar el código, a las que puedes acceder con combinaciones de teclas, como se muestra en la siguiente tabla.

Tipo	Descripción	Windows y Linux	macOS
Completar de manera básica	Muestra sugerencias básicas para variables, tipos, métodos y expresiones, entre otras. Si completas de manera básica dos veces seguidas, verás más resultados. Entre otros, miembros privados y miembros estáticos sin importar.	Control + Barra espaciadora	Control + Barra espaciadora
Completar de manera inteligente	Muestra opciones relevantes en función del contexto. La función de completar de manera inteligente tiene en cuenta el tipo y los flujos de datos esperados. Si completas de manera inteligente dos veces seguidas, verás más resultados. Por ejemplo, cadenas.	Control + Mayús + Barra espaciadora	Control + Mayús + Barra espaciadora
Completar sentencias	Completa la sentencia actual agregando paréntesis, corchetes, llaves y formatos faltantes, entre otros elementos.	Control + Mayús + Intro	Comando + Mayús + Intro

Para realizar correcciones rápidas y mostrar [acciones de intent](#) , presiona Alt + Intro.

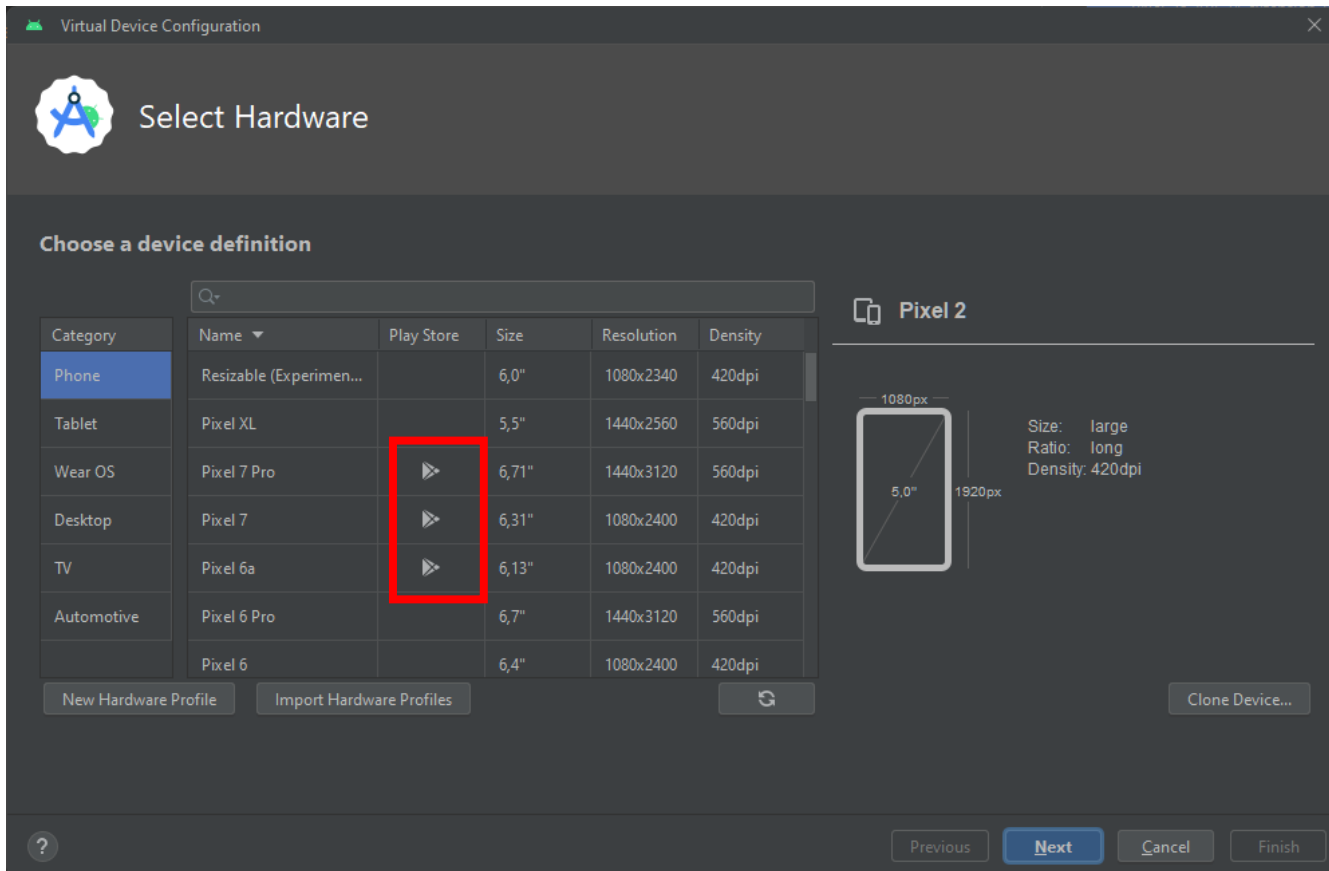
Cómo crear y administrar dispositivos virtuales

Un dispositivo virtual de Android (AVD) es una configuración que define las características de un teléfono o una tablet Android, o de un dispositivo Wear OS, Android TV o Automotive OS, que desees simular en Android Emulator. El Administrador de dispositivos es una herramienta que puedes iniciar desde Android Studio, y te permite **CREAR** y **administrar** AVD



Cómo crear y administrar dispositivos virtuales

1º Creación del Dispositivo “Hardware”



Una definición del dispositivo etiquetada con el **logotipo de Google** Play en la columna Play Store incluye la **app de Google Play Store** y acceso a los Servicios de Google Play, incluida la pestaña Google Play del diálogo Extended controls, que proporciona un botón práctico para **actualizar** los Servicios de Google Play en el dispositivo.

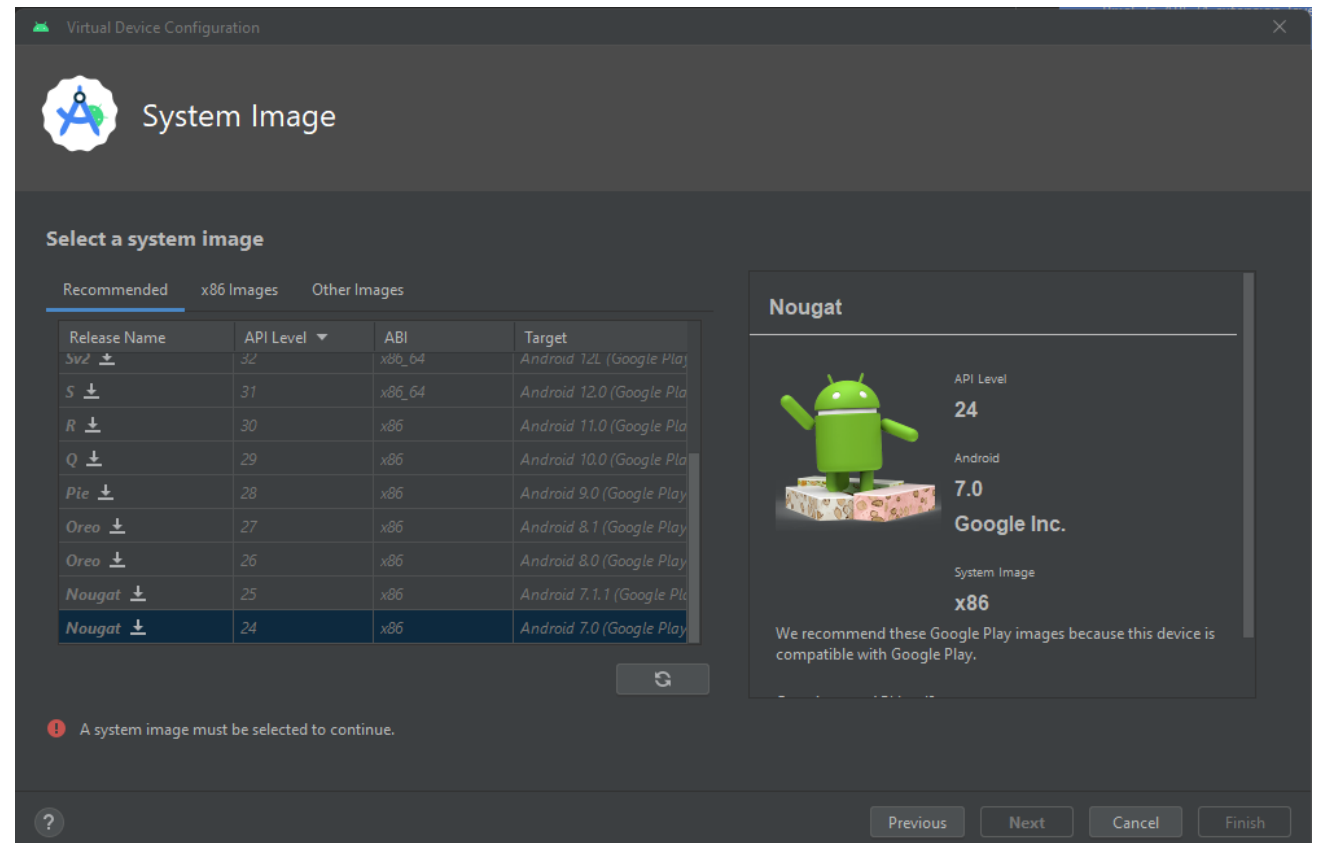
Cómo crear y administrar dispositivos virtuales

2º “Instalación” del S.O del dispositivo “Software”

Selecciona la imagen del sistema para un nivel de API determinado y haz clic en **Next**.

En la pestaña **Recommended**, aparece una lista de imágenes del sistema recomendadas. En las otras pestañas, se incluye una lista más completa. En el panel derecho, se describe la imagen del sistema seleccionada.

Si ves un ícono de **descarga** junto a una imagen del sistema, significa que aún no está instalada en tu máquina de desarrollo. Haz clic en el ícono para descargar la imagen del sistema. Para descargar imágenes del sistema, debes tener conexión a Internet.



Cómo ejecutar apps en Android Emulator

Android Emulator simula dispositivos Android en una computadora para que puedas probar tu app en diferentes dispositivos y niveles de API de Android sin necesidad de contar con los dispositivos físicos. El emulador ofrece las siguientes ventajas:

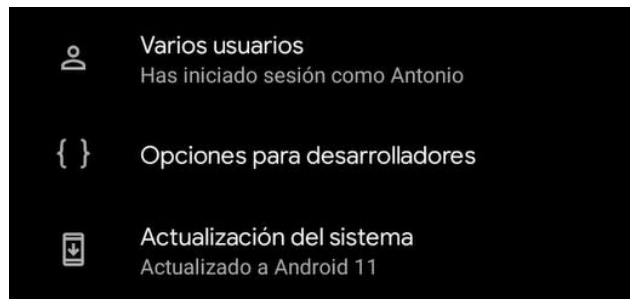
- ☐ **Flexibilidad:** Además de poder simular una variedad de dispositivos y niveles de API de Android, el emulador incluye configuraciones predefinidas para varios teléfonos y tablets Android, dispositivos Wear OS y Android TV.
- ☐ **Alta fidelidad:** El emulador proporciona casi todas las funciones de un dispositivo Android real. Puedes simular llamadas y mensajes de texto entrantes, especificar la ubicación del dispositivo, utilizar diferentes velocidades de red, probar sensores de rotación y otros sensores de hardware, acceder a Google Play Store y mucho más.
- ☐ **Velocidad:** En algunos casos, probar tu app en el emulador es más rápido y fácil que hacerlo en un dispositivo físico. Por ejemplo, puedes transferir datos con mayor velocidad al emulador que a un dispositivo conectado mediante USB.

Ejecutar apps en un dispositivo Android REAL

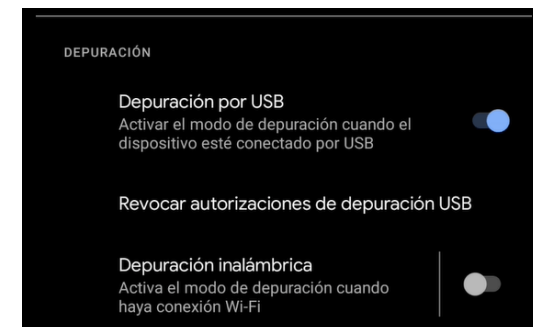
Muchas veces te va a interesar probarlo en un **dispositivo real**, y para ello necesitas saber cómo activar la depuración USB o WIFI en tu dispositivo Android.

El modo **depuración USB o WIFI** en Android es una función que permite conectar el móvil al ordenador y acceder a ciertas partes del sistema operativo que normalmente están ocultas o restringidas. Es una herramienta pensada principalmente para los **desarrolladores de aplicaciones**, que lo utilizan para crear, probar y depurar sus programas. Sin embargo, también tiene otras utilidades para los usuarios avanzados que quieren modificar o personalizar su dispositivo.

USB → El proceso para activar esta opción cambia un poco en función del dispositivo y la versión de Android. Primero hay que activar las “**Opciones para desarrolladores**” y posteriormente la “**Depuración por USB**”



Menú información del teléfono



Menú Ajustes/Sistema/Avanzado

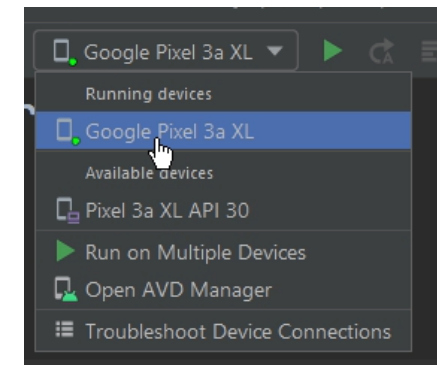
Ejecutar apps en un dispositivo Android REAL

Después tienes que conectar el teléfono al ordenador mediante un cable USB, y al hacerlo, te saltará este diálogo:



Una vez que has visto cómo activar la depuración USB, ya tienes todo lo que necesitas para probar la App en tu teléfono Android.

Tan solo tienes que seleccionarlo en el desplegable de Android Studio y darle al botón “Run”:



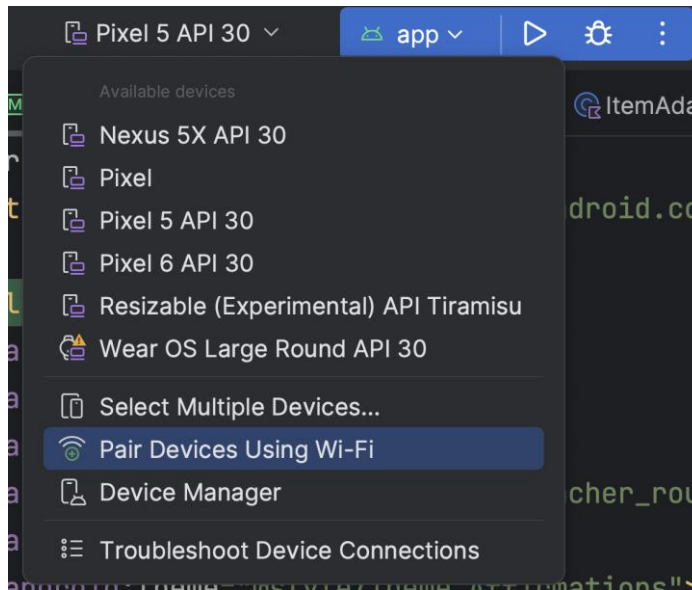
Más información:

<https://developer.android.com/studio/debug/dev-options?hl=es-419>

Ejecutar apps en un dispositivo Android REAL

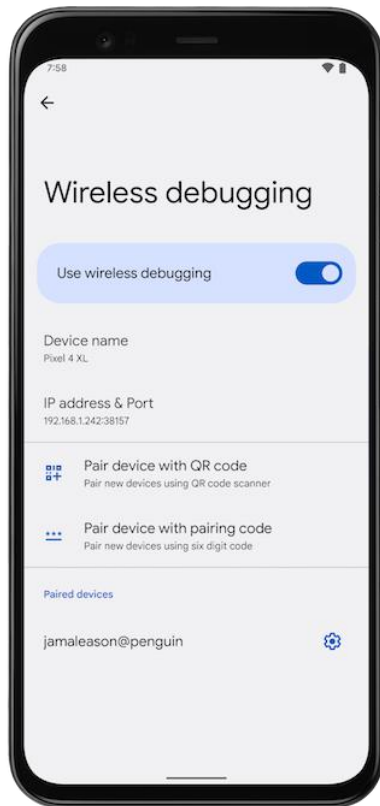
Android 11 (nivel de API 30) y las versiones posteriores admiten la implementación y la depuración de tu app de forma **inalámbrica/WIFI**.

Para usar la depuración inalámbrica, debes sincronizar tu dispositivo con tu estación de trabajo mediante un código QR o un código de vinculación. Ambos elementos deben estar conectados a la misma red inalámbrica. Para conectarte a tu dispositivo, sigue estos pasos:

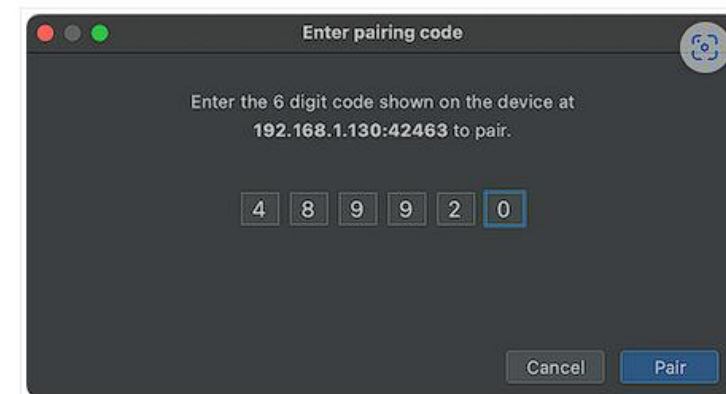


Ejecutar apps en un dispositivo Android REAL

En tu dispositivo, presiona **Depuración inalámbrica (Wireless debugging)** y vincúlalo:

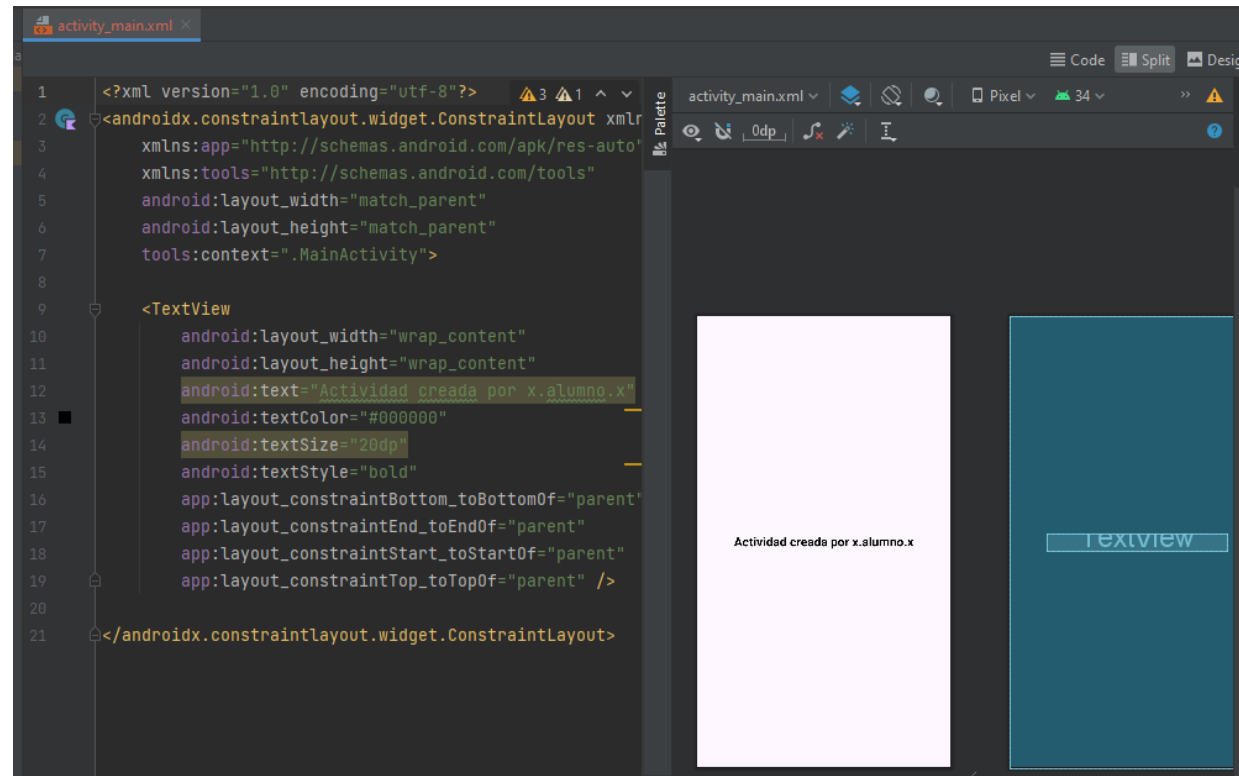


- Para vincular tu dispositivo con un código QR, selecciona Pair device with QR code y escanea el que muestre la ventana emergente Pair devices over Wi-Fi
- Para vincular tu dispositivo con un código de vinculación, selecciona Pair device with pairing code de la ventana emergente Pair devices over Wi-Fi. En el dispositivo, selecciona Pair using pairing code y toma nota del código de seis dígitos que aparece. Una vez que el dispositivo aparezca en la ventana Pair devices over Wi-Fi, puedes seleccionar Pair y, luego, ingresar el código de seis dígitos que se muestra en el dispositivo.



Edición visual de las vistas xml

Al abrir un archivo xml del proyecto, verás que en la parte superior derecha aparecen 3 opciones: **Code**, **Split** y **Design**. Con Code Podrás editar directamente el código XML, con Design Podrás realizar este diseño de forma visual. Con Split muestra las dos alternativas:



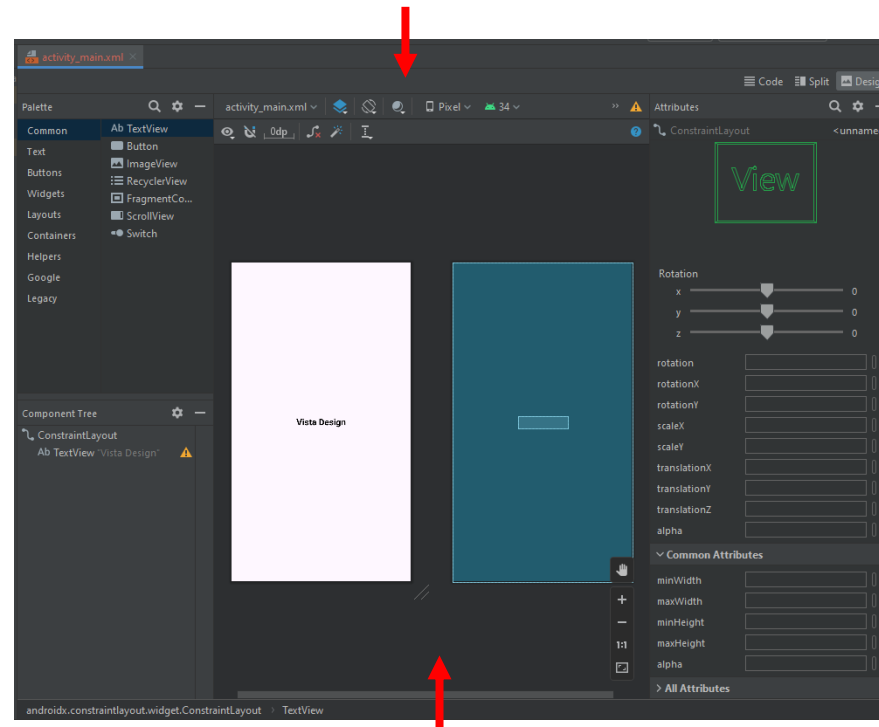
Edición visual de las vistas xml

Seleccionando la vista “Design” la pantalla se divide en

Controles para representar este layout en **diferentes configuraciones.**

Palette, te permite insertar de forma rápida nuevas vistas al layout. Puedes arrastrar cualquier elemento a la ventana de previsualización o al marco Component Tree

Component Tree con una lista con todos los elementos del layout.



Para editar un elemento, selecciónalo en el marco Component Tree o pincha directamente sobre él en la ventana de previsualización. Al seleccionarlo, puedes modificar alguna de sus propiedades (**Atributes**) en el marco situado a la derecha.

En el marco central aparece una **representación** de cómo se verá el resultado y a su derecha, con fondo azul, una representación con los nombres de cada vista y su tamaño.

Compilación de la App: APK - Android App Bundles

Un **Android App Bundle** (paquete final con todo lo necesario para que funcione nuestra aplicación) es un formato de publicación que incluye todos los **recursos y el código compilado de tu app**, pero delega la **generación del APK y la firma** a Google Play.

Google Play usa tu **paquete de aplicación** a fin de generar y entregar **APK optimizados** para la configuración de cada dispositivo, por lo que solo se descargan el código y los recursos necesarios a fin de ejecutar tu app en un dispositivo específico. Ya no es necesario compilar, firmar y administrar varios APK para optimizar la compatibilidad con diferentes dispositivos, y los usuarios obtienen descargas más pequeñas y optimizadas.

Android App Bundle fue una innovación importante ya también es que es **modular** por diseño. Esto significa que puedes **agregar módulos** a tu app que contengan funciones adicionales agregando más características y funcionalidades, sin aumentar el tamaño de la aplicación en el momento de la instalación.

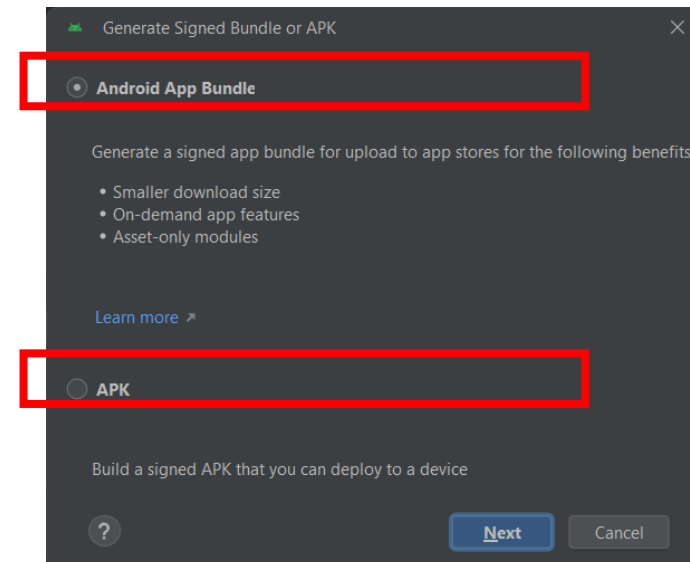
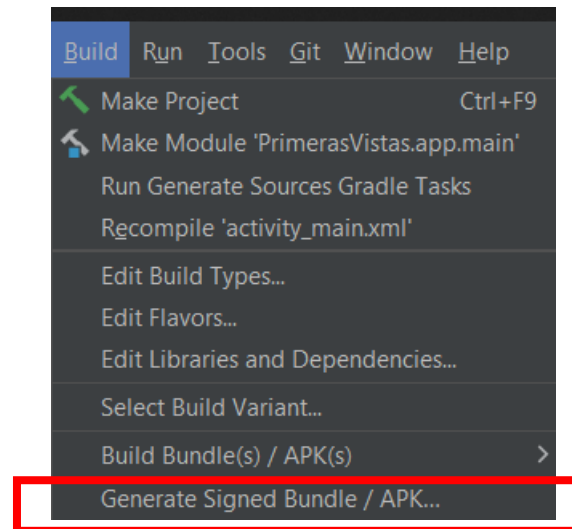
Lo mismo ocurre con los diferentes idiomas de la App, ya que la inclusión de recursos para idiomas aumentaba mucho el tamaño las aplicaciones (se estima que 37% de ahorro en tamaño).

Compilación de la App: APK - Android App Bundles

Genera una clave de carga y un almacén de claves.

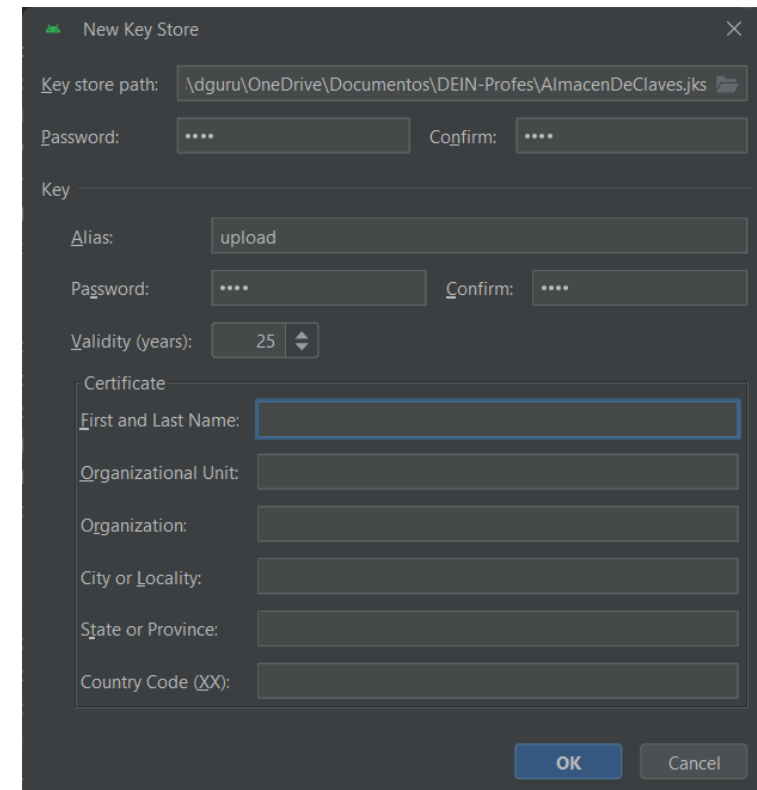
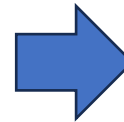
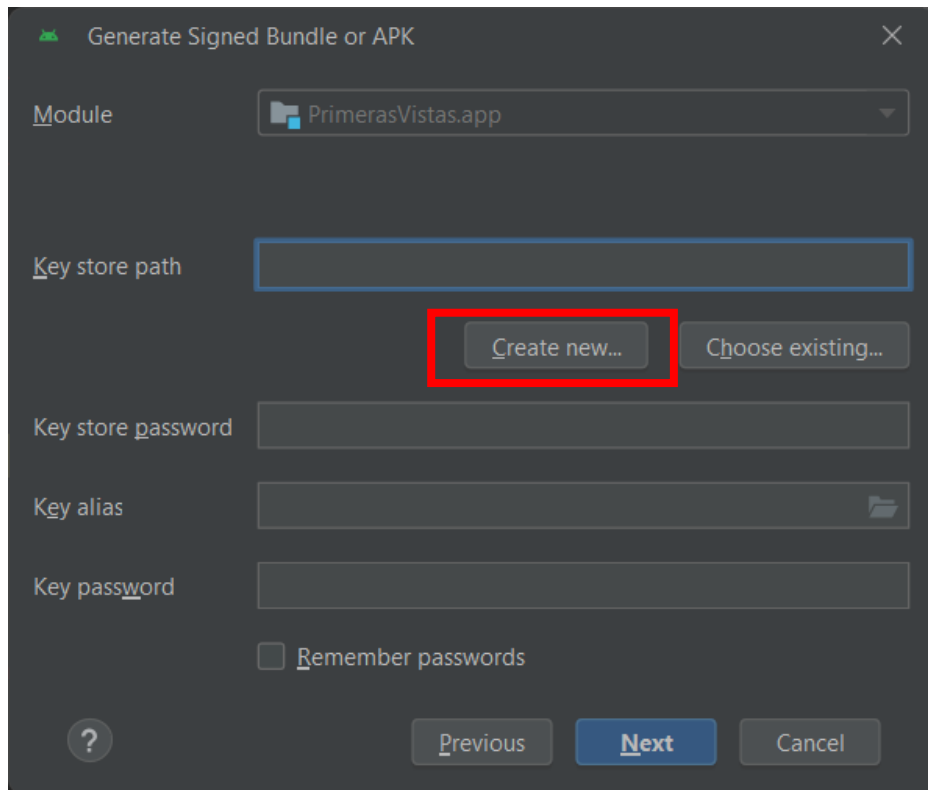
Si todavía no tienes una clave de carga, que sirve para configurar la firma de apps de Play, puedes generar una con Android Studio de la siguiente manera:

- En la barra de menú, haz clic en **Build > Generate Signed Bundle/APK**
- En el diálogo Generate Signed Bundle or APK, selecciona **Android App Bundle** (generará un archivo **aab**) o **APK** (generará un archivo **apk**), y haz clic en Next.



Compilación de la App: APK - Android App Bundles

- En el campo **Key store path**, haz clic en **Create new**.
- En la ventana **New Key Store**, proporciona la siguiente información para el almacén de claves y la clave (detalle de los pasos en página siguiente)



Compilación de la App: APK - Android App Bundles

Almacén de claves

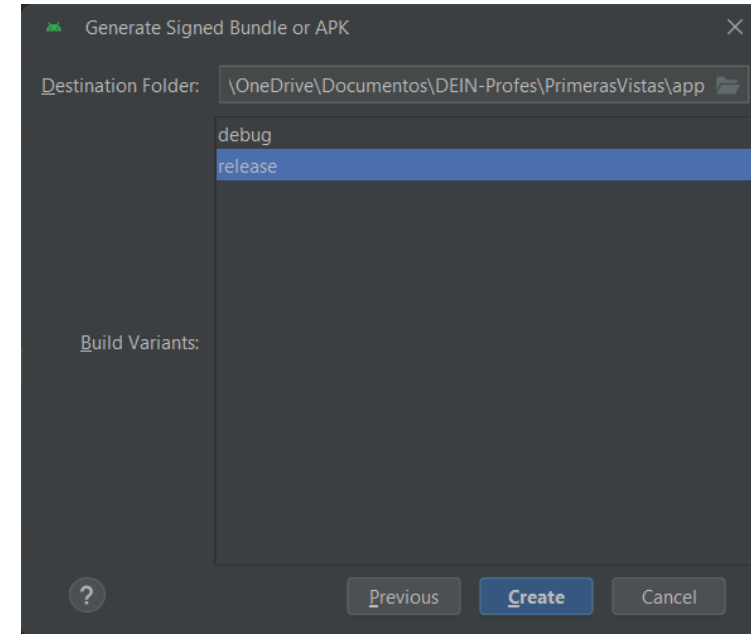
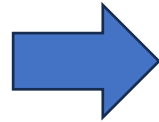
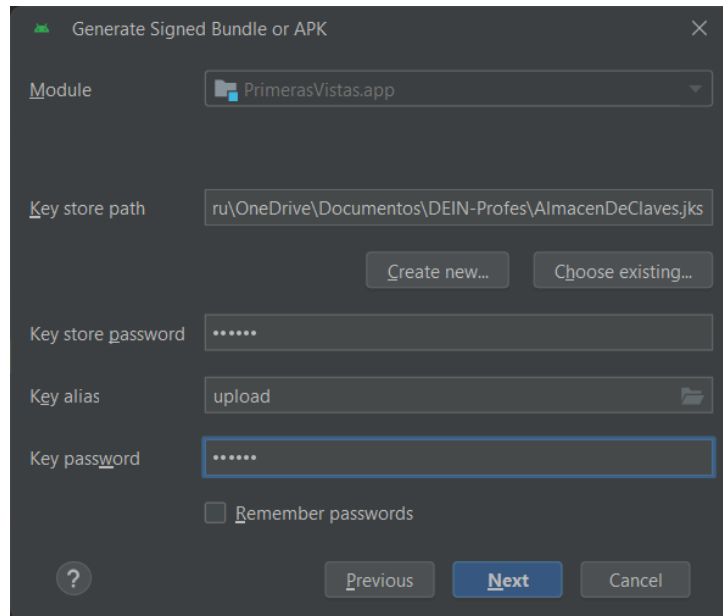
- **Key store path:** Selecciona la ubicación en la que quieras crear el almacén de claves. Además, se debe agregar un nombre de archivo al final de la ruta de acceso de ubicación con la extensión .jks.
- **Password:** Crea y confirma una contraseña segura para el almacén de claves (6).

Clave

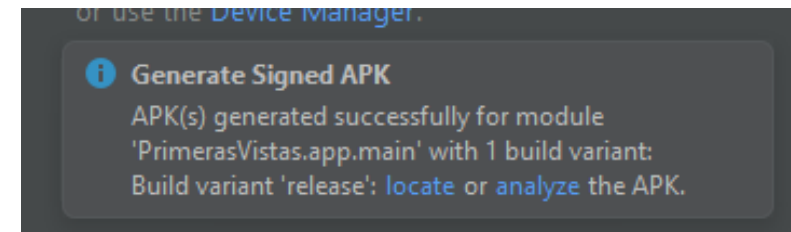
- **Alias:** Ingresa un nombre de identificación para la clave.
- **Password:** Crea y confirma una contraseña segura para tu clave. Debe ser la misma que la contraseña del almacén de claves. (Consulta el problema conocido para obtener más información).
- **Validity (years):** Establece durante cuántos años será válida la clave. Debe ser válida durante al menos 25 años para que puedas firmar actualizaciones con la misma clave durante todo el ciclo de vida de la app.
- **Certificate:** ingresa información acerca de ti para el certificado. Esta información no se muestra en la app, pero se incluye en el certificado como parte del APK.

Una vez que completes el formulario, haz clic en **OK**, y completa la ventana de firma con los datos creados.

Compilación de la App: APK - Android App Bundles



Cuando Android Studio termine de compilar la app firmada, podrás **ubicar** o **analizar** la app. Para ello, haz clic en la opción correspondiente de la notificación emergente



Compilación de la App: APK - Android App Bundles

NombreProyecto\app\release\

Nombre	Estado	Fecha de modificación	Tipo	Tamaño
 app-release		09/01/2024 13:02	Android App Package	4.597 KB

Ahora puedes habilitar tu app para la firma de apps de Play y subirla para lanzarla. Si es la primera vez que publicas una app, consulta [Descripción general del lanzamiento](#). De lo contrario, continúa con la página sobre [Cómo subir la app a Play Console](#).

Para subir tu paquete de aplicación a Play Store, crea una nueva versión en una pista de lanzamiento elegida. Puedes arrastrar y soltar el paquete en la sección "Paquetes de aplicaciones y APK" o usar la API de [Google Play Developer](#)

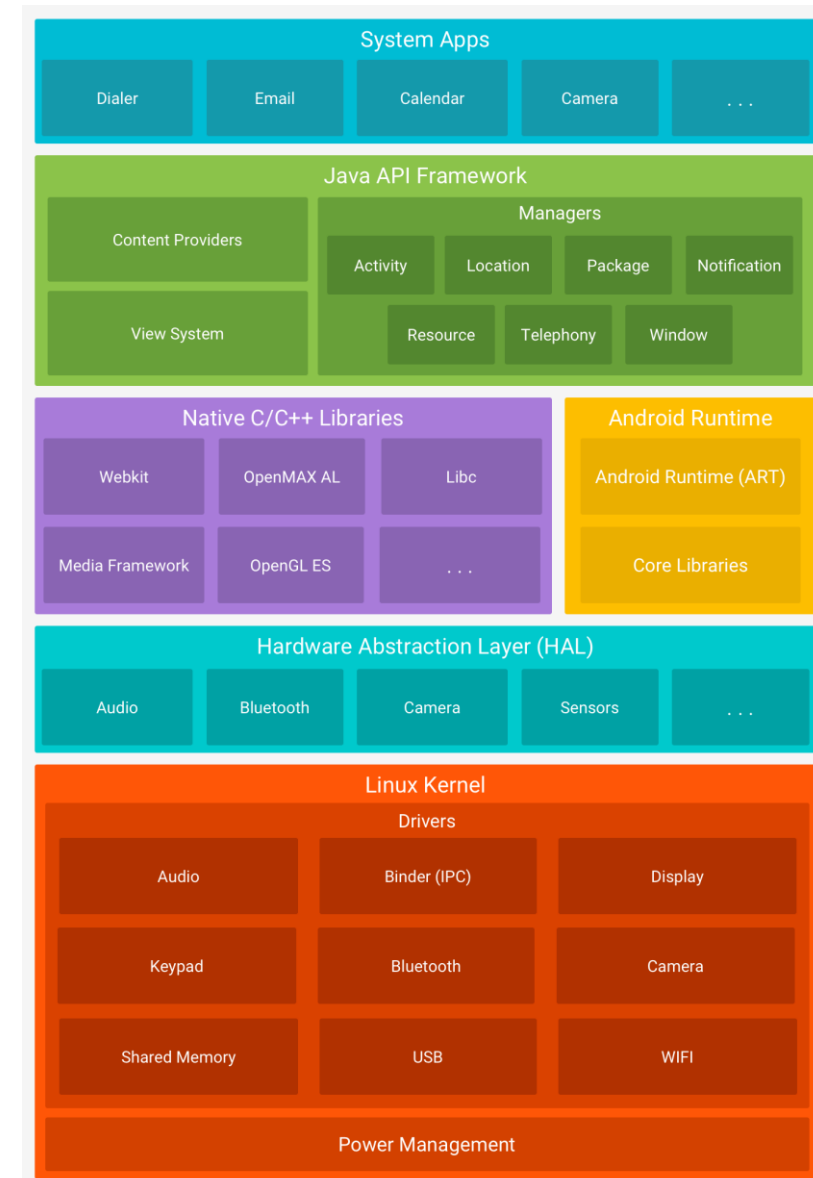
[Firma tu app](#) | [Android Studio](#) | [Android Developers](#)

3.2. Estructura de un proyecto Android (Android Studio)

Arquitectura de la plataforma

Android es una pila de software de código abierto basado en Linux creada para una variedad amplia de dispositivos y factores de forma. En el siguiente diagrama, se muestran los componentes principales de la plataforma Android.

- ❖ **Kernel de Linux**
- ❖ **Capa de abstracción de hardware (HAL)** consiste en varios módulos de biblioteca y cada uno de estos implementa una interfaz para un tipo específico de componente de hardware, como el módulo de la [cámara](#) o de [Bluetooth](#).
- ❖ **Bibliotecas C/C++ nativas**
- ❖ **Apps del sistema**



Flujo de trabajo

El flujo de trabajo de desarrollo de una app para Android es conceptualmente el mismo que el de otras plataformas de apps. Sin embargo, compilar de manera eficiente una app para Android bien diseñada requiere algunas herramientas especializadas.

En esta página, se proporciona una descripción general del proceso de compilación de una app para Android y se incluyen vínculos a más información sobre las herramientas de Android Studio para cada fase de desarrollo.

1. Cómo configurar el espacio de trabajo

Este es el primer paso del proceso de desarrollo de apps para Android. Para obtener más información, consulta la [página de instalación de Android Studio](#) y la guía para crear un proyecto.

Completa una explicación con Android Studio y aprende algunos aspectos básicos del desarrollo de Android con la guía [Cómo compilar tu primera app para Android](#).

2. Cómo escribir la app

Una vez que hayas configurado el espacio de trabajo, puedes comenzar a escribir la app. Android Studio incluye inteligencia y una variedad de herramientas que te ayudarán a trabajar más rápido, escribir código de calidad, diseñar una IU y crear recursos para diferentes tipos de dispositivos. Para obtener más información sobre las herramientas y las funciones disponibles, consulta [Cómo escribir tu app](#).

Flujo de trabajo

3. Compilación y ejecución

Durante la fase de compilación y ejecución, compilas el proyecto en un paquete de APK depurable que puedes instalar y ejecutar en el emulador o en un dispositivo con Android. Para obtener más información sobre cómo ejecutar el código, consulta [Cómo compilar y ejecutar la app](#).

En esta fase, también puedes personalizar la compilación. Por ejemplo, puedes [crear variantes de compilación](#) que generen diferentes versiones de la app a partir del mismo proyecto y [reducir el código y los recursos](#) para que la app sea más pequeña. Para ver una introducción a las configuraciones de compilación personalizadas, consulta [Cómo configurar tu compilación](#).

4. Cómo depurar, generar perfiles y realizar pruebas

En esta fase iterativa, continúas el desarrollo de la app, eliminas los errores y optimizas el rendimiento de la app. Si quieres obtener ayuda para depurar y optimizar la app, [pruébala en Android Studio](#).

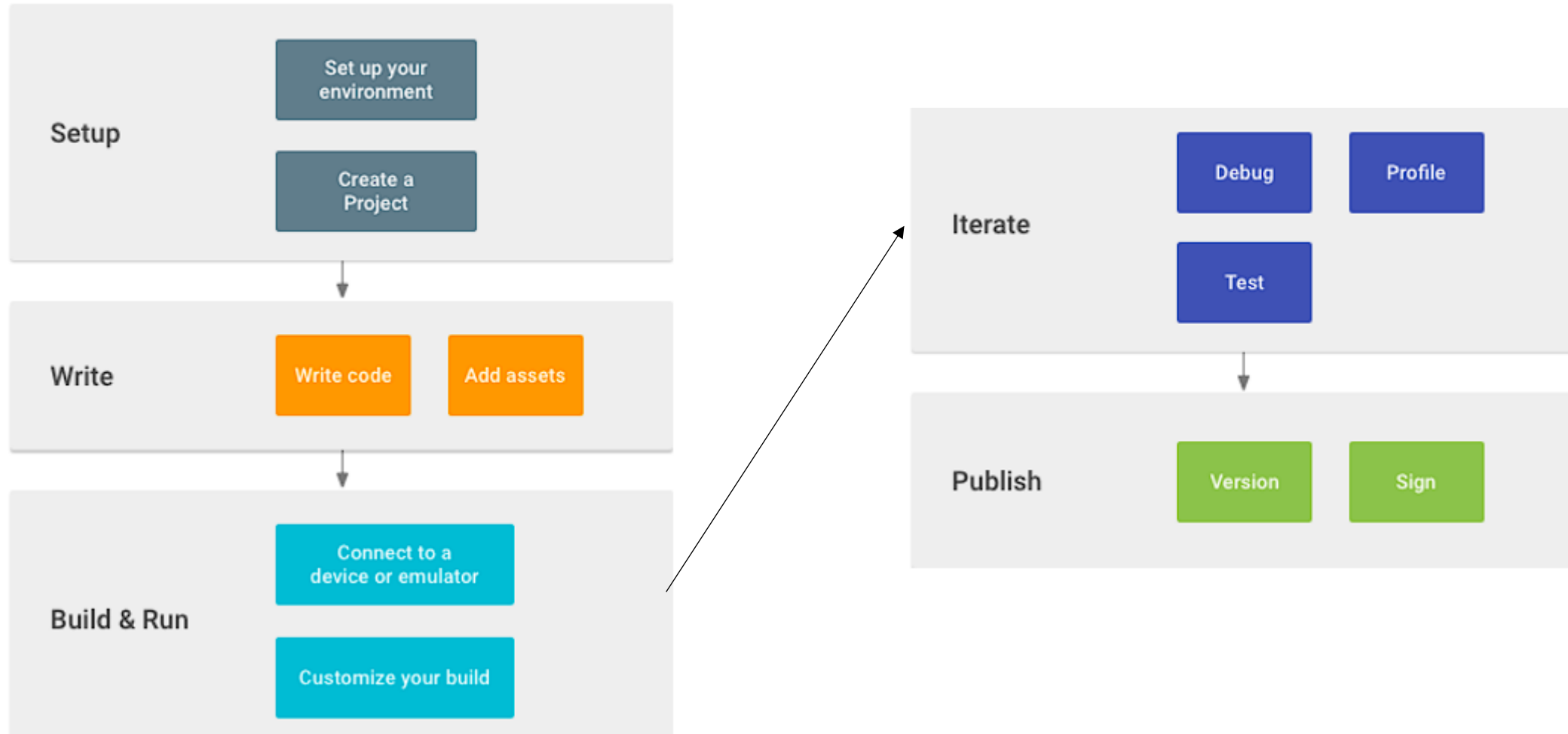
Para obtener más información sobre la depuración, consulta [Cómo depurar la app](#) y [Cómo escribir y ver registros con Logcat](#).

Para ver y analizar las diferentes métricas de rendimiento, como el uso de memoria, el tráfico de red y el impacto en la CPU, entre otras, consulta [Cómo perfilar el rendimiento de la app](#).

5. Cómo llevar a cabo la publicación

Para preparar la app para el lanzamiento a los usuarios, deberás compilar un [Android App Bundle](#), firmarlo con una llave de seguridad y prepararte para publicarlo en Google Play Store. Para obtener más información, consulta [Cómo publicar la app](#).

Flujo de trabajo

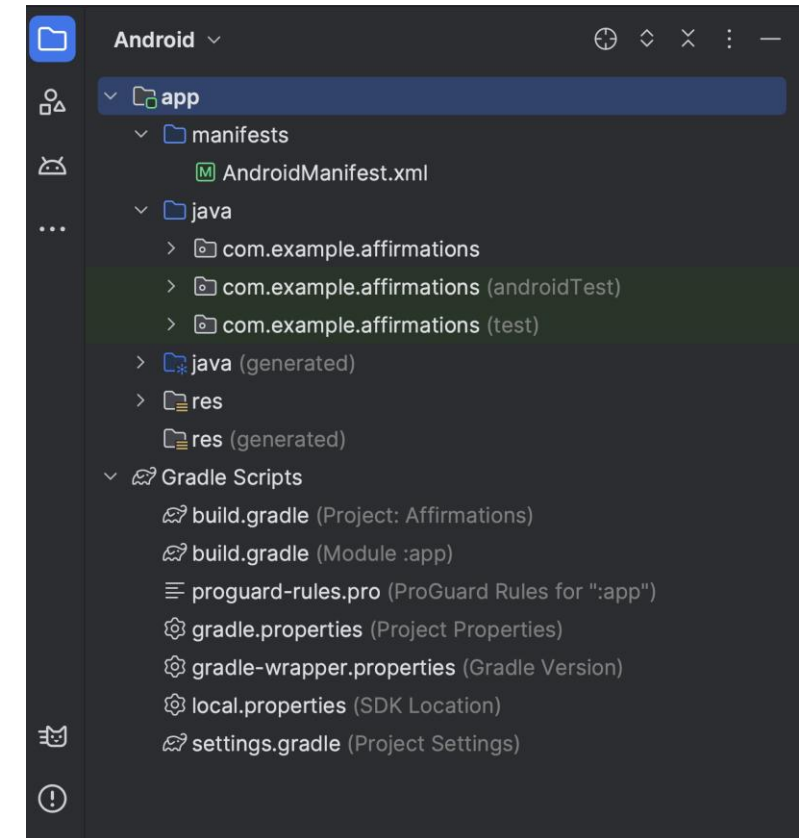


Estructura del proyecto en Android Studio

Cada proyecto de Android Studio incluye uno o más módulos con archivos de código fuente y archivos de recursos. Entre los tipos de módulos, se incluyen los siguientes:

- ☐ Módulos de apps para Android
- ☐ Módulos de biblioteca
- ☐ Módulos de Google App Engine

De manera predeterminada, Android Studio muestra los archivos de tu proyecto en la vista de proyecto de Android. Esta vista está organizada en módulos para que puedas acceder rápidamente a los archivos fuente clave de tu proyecto. Puedes ver todos los archivos de compilación en el nivel superior, en **Gradle Scripts**.

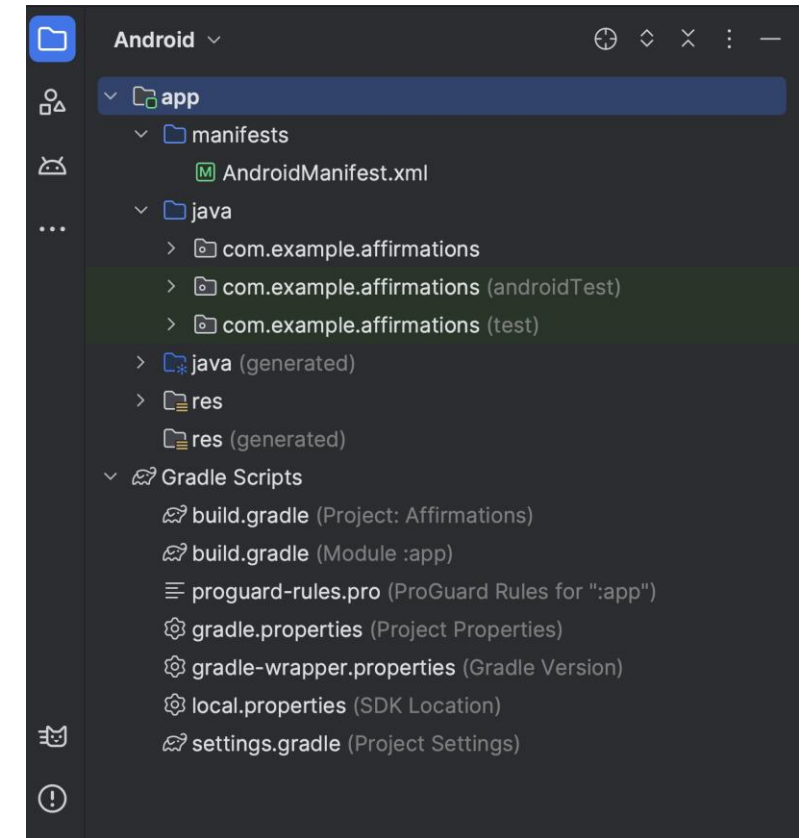


Estructura del proyecto en Android Studio

Cada módulo de app contiene las siguientes carpetas:

- ❑ **manifests:** Contiene el archivo **AndroidManifest.xml**.
- ❑ **java:** Contiene los archivos de código fuente de **Kotlin y Java**, incluido el código de prueba JUnit.
- ❑ **res:** Contiene todos los recursos sin código, como cadenas de IU y, además, imágenes de mapa de bits.

La estructura del proyecto de Android en el disco difiere de esta representación plana. Para ver la estructura real de archivos del proyecto, selecciona **Project** en lugar de **Android** en el menú Project.



Sistema de compilación de Gradle en Android Studio

Android Studio usa **Gradle** como base del sistema de compilación, y el complemento de Android para Gradle proporciona capacidades específicas de Android. Este sistema de compilación se ejecuta en una herramienta integrada desde el menú de Android Studio, y lo hace independientemente de la línea de comandos. Puedes usar las funciones del sistema de compilación para lo siguiente:

- ✓ Personalizar, configurar y extender el proceso de compilación
- ✓ Crear varios APKs para tu app; diferentes funciones usan el mismo proyecto y los mismos módulos
- ✓ Volver a utilizar códigos y recursos entre conjuntos de orígenes
- ✓ Gracias a la flexibilidad de Gradle, puedes lograrlo sin modificar los archivos fuente de tu app.

Los archivos de compilación de Android Studio se denominan **build.gradle.kts** si usas Kotlin

El sistema de compilación puede ayudarte a crear diferentes versiones de la misma app a partir de un solo proyecto. Por ejemplo, para hacer una versión gratis o una versión de pago de tu app, o para crear APKs independientes de una app para densidades de pantalla diferentes (hdpi y mdpi)

Sistema de compilación de Gradle en Android Studio

Además, cada vez que compilas tu programa, Android Studio ejecuta automáticamente inspecciones de **lint** y otras inspecciones de IDE (entorno de desarrollo integrado) para ayudarte a identificar y corregir problemas con la **calidad estructural** de tu código de manera sencilla.

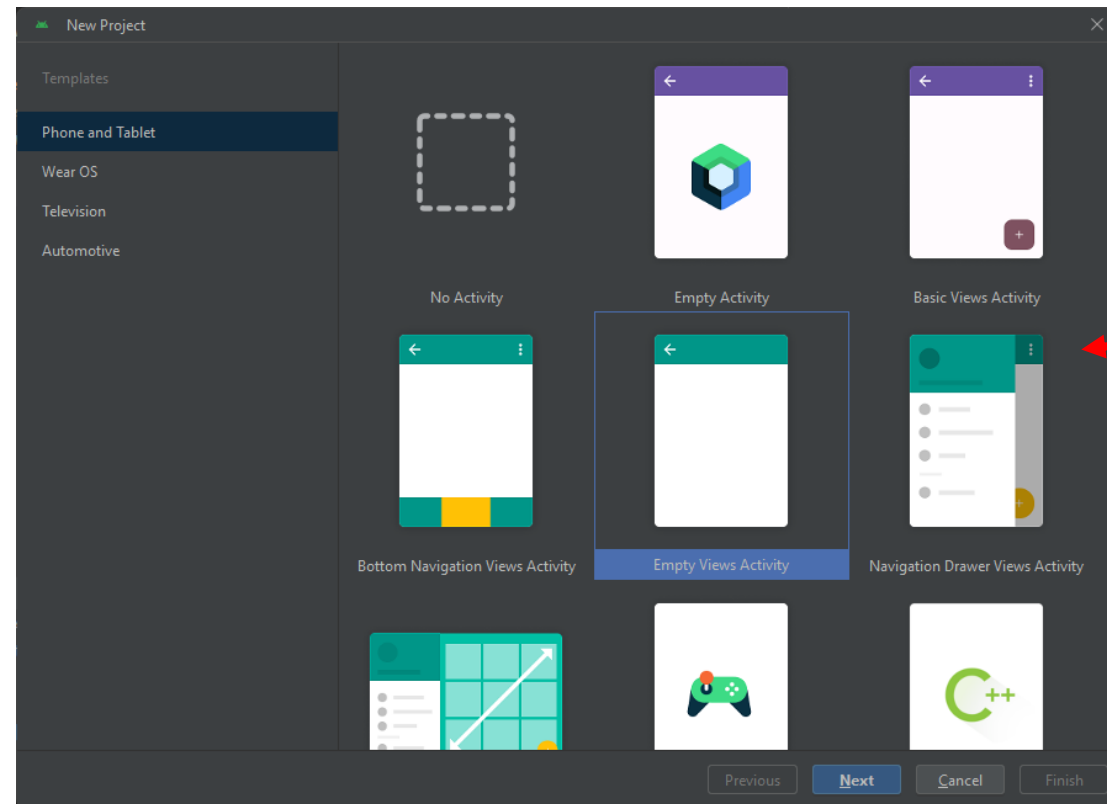
La herramienta **lint** comprueba los archivos de origen de tu proyecto de Android en busca de posibles errores y para realizar mejoras relacionadas con la **precisión, la seguridad, el rendimiento, la usabilidad, la accesibilidad y la internacionalización**.

Además, realizará una reducción de recursos que quita automáticamente los recursos sin usar de tu aplicación empaquetada y las dependencias de bibliotecas.

Cómo crear un proyecto

1. Selecciona **File > New > New Project**.

Tipos de dispositivos

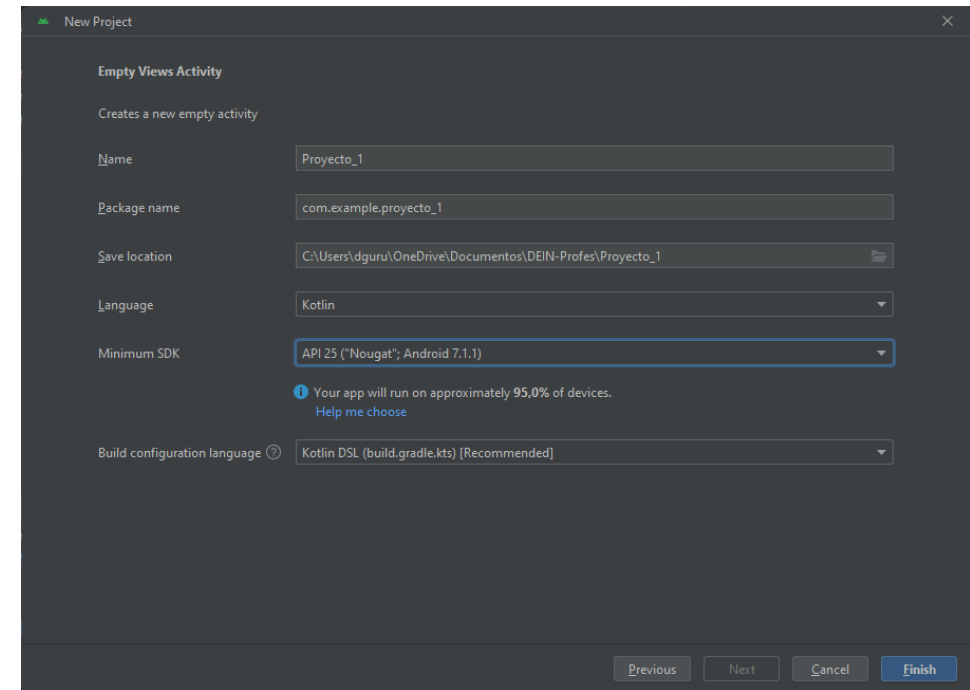


Plantillas, o proyecto vacío

Cómo crear un proyecto

2. En la ventana **Configure your project**, completa lo siguiente:

1. Escribe el “nombre del proyecto” en el campo **Name**.
2. Puedes escribir un nombre para el paquete de la App o dejar el que aparece por defecto "com.example.proyecto_1" en el campo **Package name**.
3. Si quieres colocar el proyecto en una carpeta diferente, cambia su ubicación en **Save**.
4. Selecciona **Java** o **Kotlin** en el menú desplegable **Language**.
5. Selecciona la versión más baja de Android que desees que admita tu app en el campo **Minimum SDK**.
6. Deja las otras opciones como están.



Cómo crear un proyecto

En la pestaña **Project** de la izquierda, y en la vista **Android** (se puede seleccionar la vista **Project**) podrás ver los siguientes archivos:

app > java > com.example.myfirstapp > MainActivity

Esta es la actividad principal. Es el punto de entrada de tu app. Cuando compilas y ejecutas la app, el sistema inicia una instancia del elemento Activity y carga su diseño (MainActivity.kt).

app > res > layout > activity_main.xml

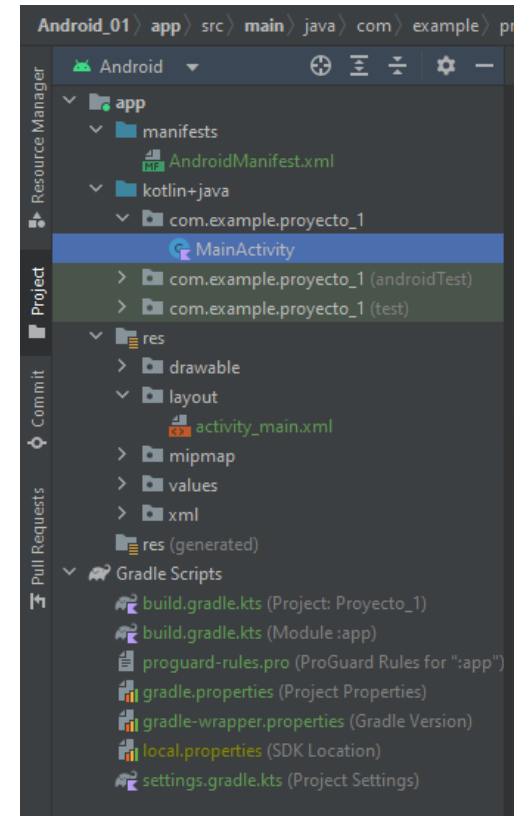
Este archivo en formato XML define el diseño de la interfaz de usuario (IU) de la actividad.

app > manifests > AndroidManifest.xml

En el archivo de “manifiesto”, se describen las características fundamentales de la app, y se define cada uno de sus componentes.

Gradle Scripts > build.gradle

Hay dos archivos con este nombre: uno para el proyecto ("Project: Nombre") y otro para el módulo de la app ("Module: My_First_App.app"). Cada módulo tiene su propio archivo build.gradle, y al comenzar sólo tiene un solo módulo. Usa el archivo build.gradle de cada módulo para controlar cómo el complemento de Gradle compila tu app.



Cómo crear un proyecto

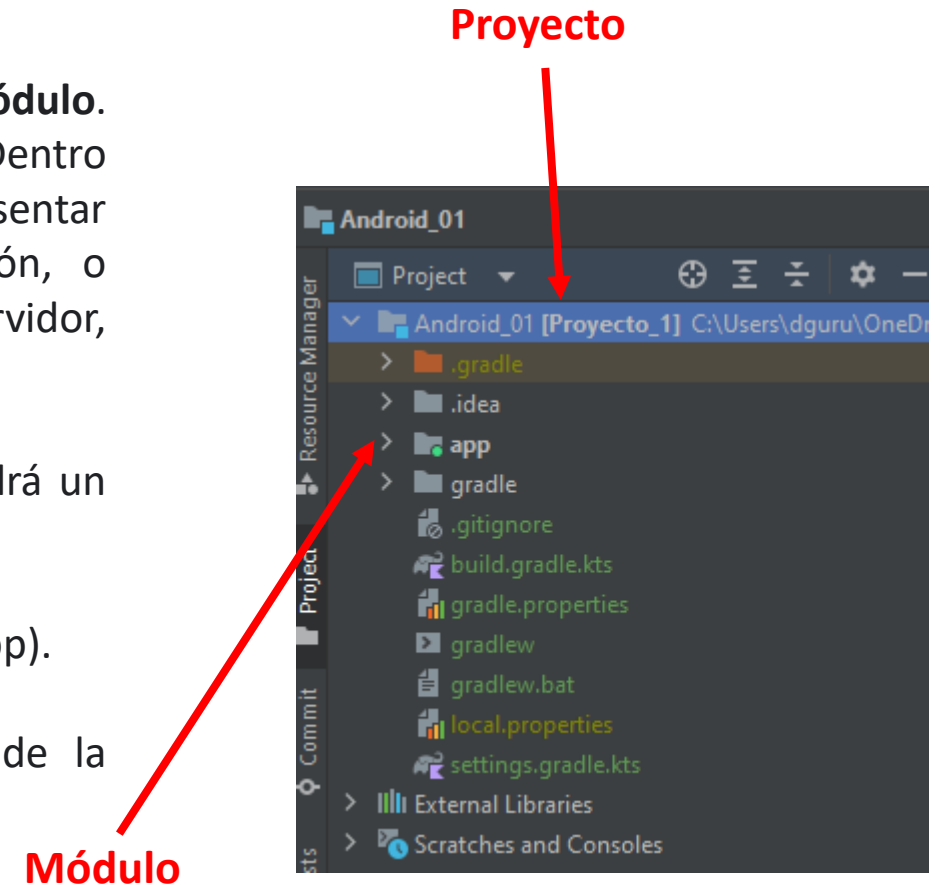
Otra opción en la pestaña **Project** es utilizar la vista **Project**, donde podrás ver mejor como es la estructura del proyecto.

Lo primero que debemos distinguir son los conceptos de **proyecto y módulo**. La entidad proyecto es única, y engloba a todos los demás elementos. Dentro de un proyecto podemos incluir **varios módulos**, que pueden representar **aplicaciones distintas, versiones diferentes** de una misma aplicación, o **distintos componentes** de un sistema (aplicación móvil, aplicación servidor, librerías, ...).

En la mayoría de los casos, trabajaremos con un proyecto que contendrá un sólo módulo correspondiente a nuestra aplicación principal.

La mayoría del trabajo lo vamos a realizar sobre la carpeta del **módulo** (app).

La carpeta de “glade” es utilizada para los temas de compilación de la aplicación.



Cómo crear un proyecto

Módulos

Un módulo es una colección de archivos de origen y parámetros de configuración de compilación que te permiten **dividir tu proyecto** en unidades de funcionalidad discretas. Tu proyecto puede tener uno o más módulos, y un módulo puede usar otro como dependencia. **Puedes compilar, probar y depurar cada módulo por separado.**

Los módulos adicionales son útiles cuando creas **bibliotecas de código** dentro de tu propio proyecto o si deseas crear **diferentes conjuntos de código y recursos** para **distintos tipos de dispositivos**, como teléfonos y wearables, y al mismo tiempo mantener todos los archivos almacenados dentro del mismo proyecto y compartir código.

Para agregar un módulo nuevo a tu proyecto, haz clic en **File > New > New Module**.

Android Studio ofrece diferentes tipos de módulos:

- Módulo de app para Android
- Módulo de funciones
- Módulo de bibliotecas

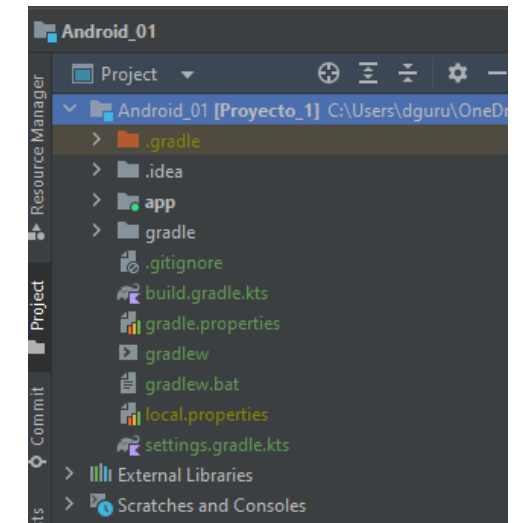
Cómo crear un proyecto

Módulo de app para Android

Proporciona un contenedor para el código fuente de tu app, los archivos de recursos y los parámetros de configuración de niveles de app, como el archivo de compilación de nivel de módulo y el archivo de manifiesto de Android. Cuando creas un proyecto nuevo, el módulo predeterminado de la app se llama "app".

Tipos de módulos de apps:

- Teléfono y tablet
- Automotive
- Wear OS
- Televisión
- Generador de perfiles de Baseline
- Comparativas



Cada uno proporciona archivos esenciales y algunas plantillas de código apropiadas para el tipo de dispositivo o app correspondiente.

Cómo crear un proyecto

Carpetas más importantes:

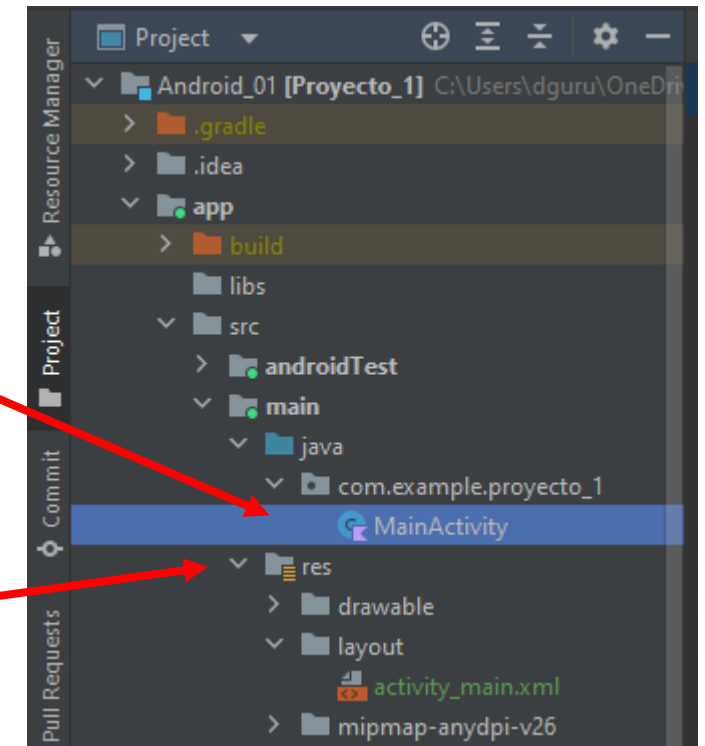
Carpeta /app/src/main/java

Esta carpeta (aunque se llame java también se utiliza para Kotlin), contendrá todo el código fuente de la aplicación, clases auxiliares, etc.

Inicialmente, Android Studio creará por nosotros el código básico de la pantalla (actividad o activity) principal de la aplicación, que por defecto se llamará **MainActivity**, y siempre bajo la estructura del paquete java definido durante la creación del proyecto.

Carpeta /app/src/main/res/

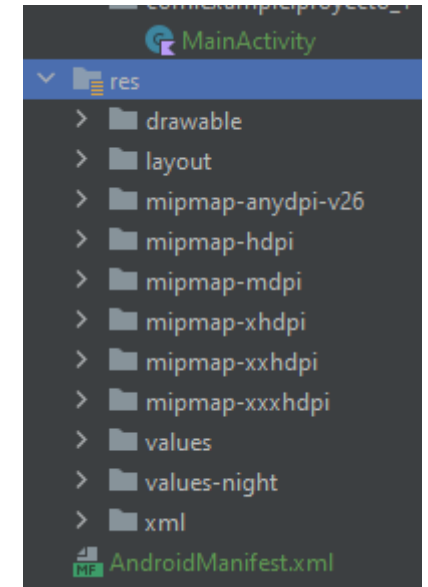
Contiene todos los ficheros de recursos necesarios para el proyecto: imágenes, layouts, cadenas de texto, etc. Los diferentes tipos de recursos se pueden distribuir entre las siguientes subcarpetas.



Cómo crear un proyecto

Sub-Carpetas de recursos “res”

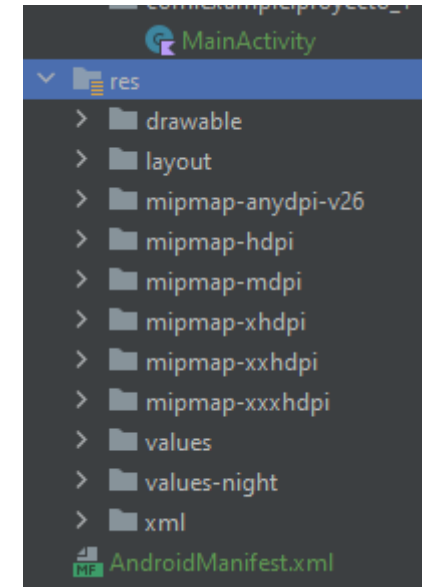
Carpeta	Descripción
/res/drawable/	Contiene las imágenes y otros elementos gráficos usados por la aplicación. Para poder definir diferentes recursos dependiendo de la resolución y densidad de la pantalla del dispositivo se suele dividir en varias subcarpetas: * /drawable (recursos independientes de la densidad) * /drawable-ldpi (densidad baja) * /drawable-mdpi (densidad media) * /drawable-hdpi (densidad alta) * /drawable-xhdpi (densidad muy alta) * /drawable-xxhdpi (densidad muy muy alta :)
	Contiene los iconos de lanzamiento de la aplicación (el icono que aparecerá en el menú de aplicaciones del dispositivo) para las distintas densidades de pantalla existentes. Al igual que en el caso de las carpetas /drawable, se dividirá en varias subcarpetas dependiendo de la densidad de pantalla:



Cómo crear un proyecto

Sub-Carpetas de recursos “res”

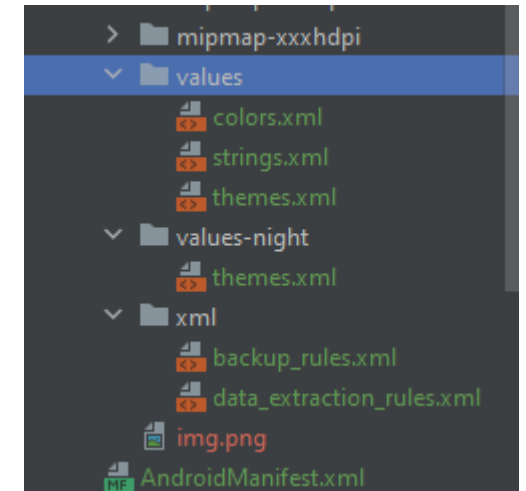
/res/mipmap/	Contiene los iconos de lanzamiento de la aplicación (el icono que aparecerá en el menú de aplicaciones del dispositivo) para las distintas densidades de pantalla existentes. Al igual que en el caso de las carpetas /drawable, se dividirá en varias subcarpetas dependiendo de la densidad de pantalla: * /mipmap-mdpi * /mipmap-hdpi * /mipmap-xhdpi * ...
/res/layout/	Contiene los ficheros de definición XML de las diferentes pantallas de la interfaz gráfica. Para definir distintos <i>layouts</i> dependiendo de la orientación del dispositivo se puede dividir también en subcarpetas: * /layout (vertical) * /layout-land (horizontal)



Cómo crear un proyecto

Sub-Carpetas de recursos “res”

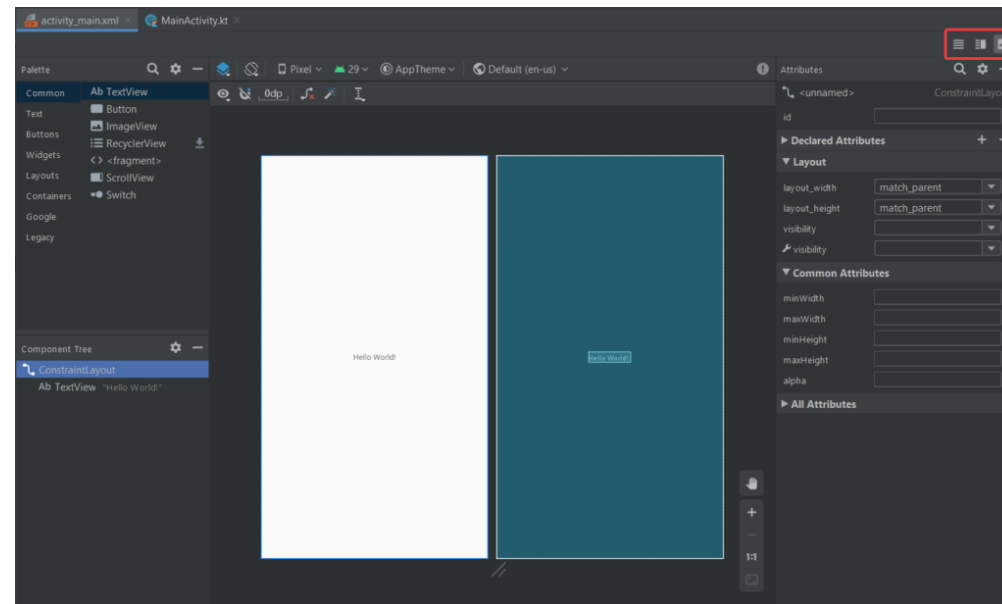
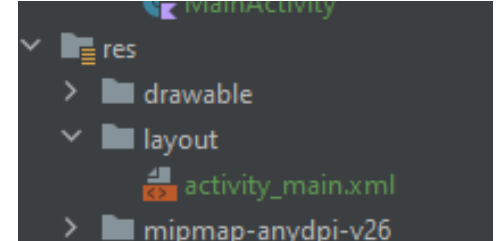
/res/anim/ /res/animator/	Contienen la definición de las animaciones utilizadas por la aplicación.
/res/color/	Contiene ficheros XML de definición de listas de colores según estado.
/res/menu/	Contiene la definición XML de los menús de la aplicación.
/res/xml/	Contiene otros ficheros XML de datos utilizados por la aplicación.
/res/raw/	Contiene recursos adicionales, normalmente en formato distinto a XML, que no se incluyan en el resto de carpetas de recursos.
/res/values/	Contiene otros ficheros XML de recursos de la aplicación, como por ejemplo cadenas de texto (<i>strings.xml</i>), estilos (<i>styles.xml</i>), colores (<i>colors.xml</i>), arrays de valores (<i>arrays.xml</i>), tamaños (<i>dimens.xml</i>), etc.



Cómo crear un proyecto

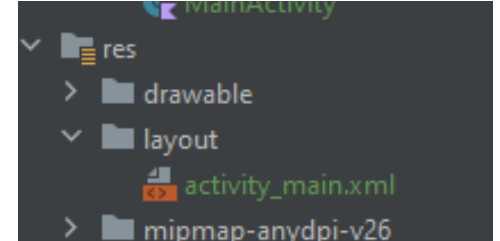
Sub-Carpeta Layouts, fichero “activity_main.xml”

Entre los recursos creados por defecto cabe destacar los layouts, en nuestro caso sólo tendremos por ahora el llamado “activity_main.xml”, que contienen la definición de la interfaz gráfica de la pantalla principal de la aplicación. Si hacemos doble clic sobre este fichero Android Studio nos mostrará esta interfaz en su editor gráfico.

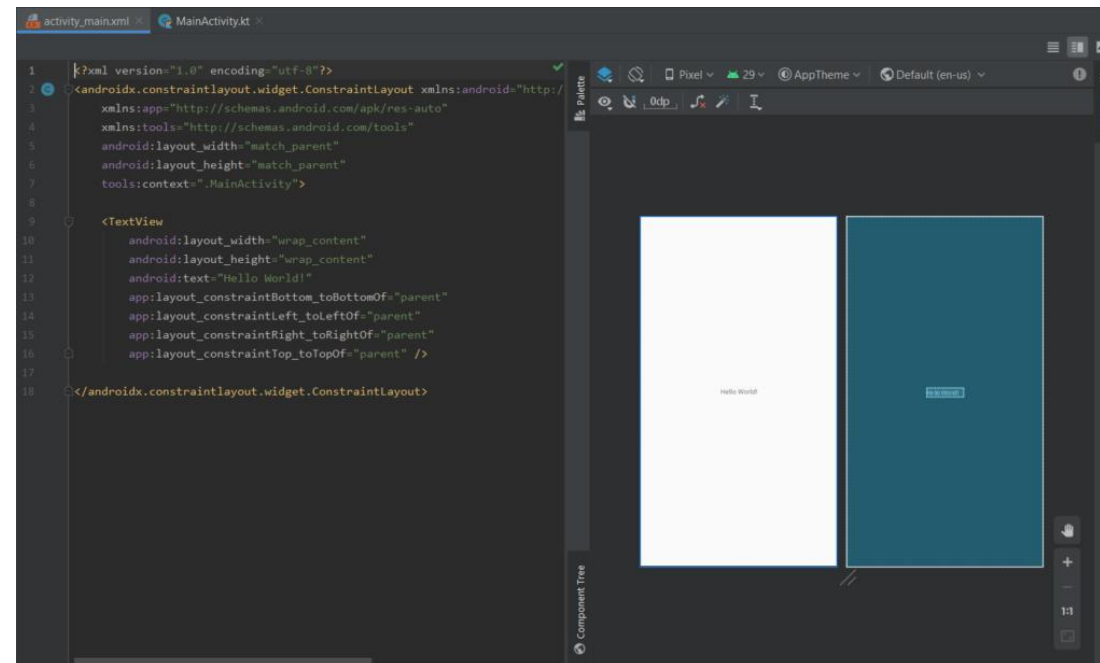


Cómo crear un proyecto

Sub-Carpeta Layouts, fichero “activity_main.xml”!



Pulsando sobre los tres botones de la esquina superior derecha (resaltados en rojo en la imagen anterior) podemos alternar entre el editor gráfico (tipo arrastrar-y-soltar), mostrado en la imagen anterior, el editor de código XML, o una vista compartida que permita visualizar ambas cosas de forma simultánea, como en la imagen siguiente:



Cómo crear un proyecto

Fichero `/app/src/main/AndroidManifest.xml`

Contiene la definición en XML de muchos de los aspectos principales de la aplicación, como por ejemplo su identificación (nombre, icono, ...), sus componentes (pantallas, servicios, ...), o los permisos necesarios para su ejecución. Veremos más adelante más detalles de este fichero.

Fichero `/app/build.gradle`

Contiene información necesaria para la compilación del proyecto, por ejemplo, la versión del SDK de Android utilizada para compilar, la mínima versión de Android que soportará la aplicación, referencias a las librerías externas utilizadas, etc.

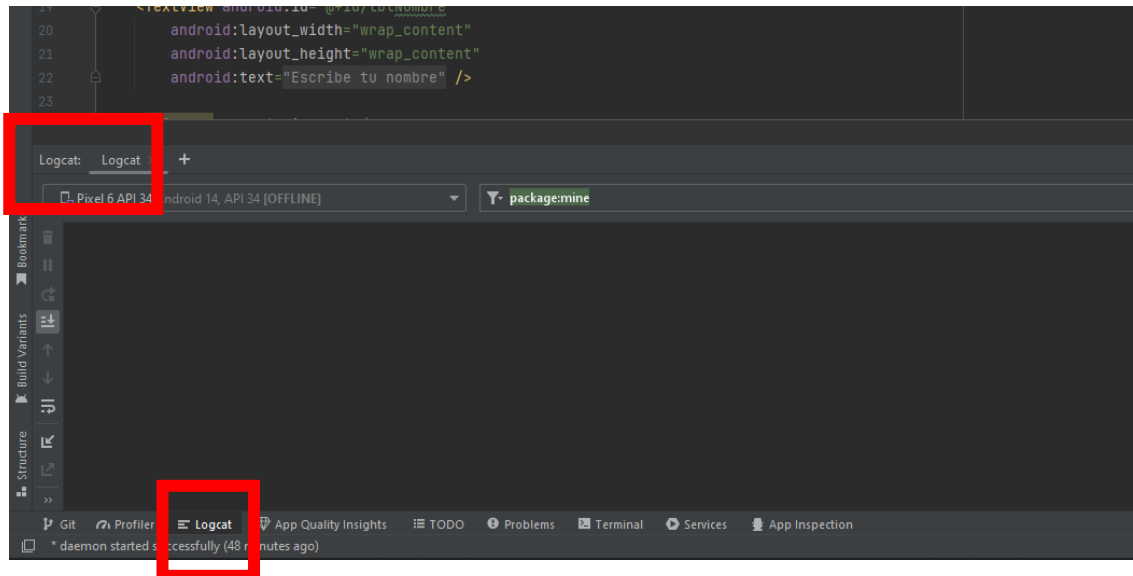
Carpeta `/app/libs`

Puede contener las librerías externas que utilice nuestra aplicación. Normalmente no incluiremos directamente aquí ninguna librería, sino que haremos referencia a ellas en el fichero `build.gradle` descrito en el punto anterior, de forma que entren en el proceso de compilación de nuestra aplicación.

Depurar con mensajes LogCat

El sistema Android utiliza el fichero LogCat para registrar todos los problemas y eventos principales que ocurren en el sistema. Ante cualquier error resulta muy interesante consultarlo para tratar de encontrar su origen.

La clase Log proporciona un mecanismo para introducir mensajes desde nuestro código en este fichero. Puede ser muy útil para depurar nuestros programas o para verificar el funcionamiento del código. Disponemos de varios métodos para generar distintos tipos de mensajes:



- **V / Verbose:** la menor prioridad, con más información de la necesaria
- **D / Debug:** mensajes útiles para la depuración de aplicaciones
- **I / Info:** mensajes de información
- **W / Warning:** mensajes con advertencias
- **E / Error:** errores de la aplicación

Depurar con mensajes LogCat

Cómo ver los registros de tu app

Para ver los mensajes del registro de una app:

1. Primero tienes que compilar y ejecuta la app en un dispositivo.
2. Después haz clic en **View > Tool Windows > Logcat** (o en Logcat en la barra de ventanas de herramientas). En la ventana Logcat se muestran los mensajes de registro de la app que se seleccionaron en el menú ubicado en la parte superior de la ventana.

Depurar con mensajes LogCat

Logcat

Huawei Nexus 6P Android 7.1.2, API 25 | com.example.android.displayingbitmaps (9224) | Show only selected application

1 [Clear] 2 [Filter] 3 [Refresh] 4 [Log Levels] 5 [Log Categories] 6 [Log Tags] 7 [Log Verbs] 8 [Log Filter] 9 [Log Help]

```
04-18 07:12:27.590 9224-9395/com.example.android.displayingbitmaps I/OpenGLRenderer: Initialized EGL, version 1.4
04-18 07:12:27.590 9224-9395/com.example.android.displayingbitmaps D/OpenGLRenderer: Swap behavior 1
04-18 07:12:27.827 9224-9229/com.example.android.displayingbitmaps I/art: Do partial code cache collection, code=29KB, data=28KB
04-18 07:12:27.827 9224-9229/com.example.android.displayingbitmaps I/art: After code cache collection, code=27KB, data=27KB
04-18 07:12:27.827 9224-9229/com.example.android.displayingbitmaps I/art: Increasing code cache capacity to 128KB
04-18 07:12:27.916 9224-9229/com.example.android.displayingbitmaps I/art: Do partial code cache collection, code=61KB, data=53KB
04-18 07:12:27.917 9224-9229/com.example.android.displayingbitmaps I/art: After code cache collection, code=61KB, data=53KB
04-18 07:12:27.917 9224-9229/com.example.android.displayingbitmaps I/art: Increasing code cache capacity to 256KB
04-18 07:12:28.098 9224-9224/com.example.android.displayingbitmaps I/art: Starting a blocking GC Explicit
04-18 07:12:28.121 9224-9224/com.example.android.displayingbitmaps I/art: Explicit concurrent mark sweep GC freed 1933(106KB)
  AllocSpace objects, 0(0B) LOS objects, 40% free, 13MB/21MB, paused 333us total 22.300ms
04-18 07:12:28.123 9224-9224/com.example.android.displayingbitmaps I/art: Starting a blocking GC Explicit
04-18 07:12:28.145 9224-9224/com.example.android.displayingbitmaps I/art: Explicit concurrent mark sweep GC freed 187(8KB)
  AllocSpace objects, 0(0B) LOS objects, 39% free, 13MB/21MB, paused 318us total 21.597ms
04-18 07:12:28.377 9224-9388/com.example.android.displayingbitmaps E/GRPC: tcp_client_posix.c:173] failed to connect to
  'ipv4:127.0.0.1:12389': socket error: connection refused
04-18 07:12:29.374 9224-9389/com.example.android.displayingbitmaps E/GRPC: tcp_client_posix.c:173] failed to connect to
  'ipv4:127.0.0.1:12389': socket error: connection refused
04-18 07:12:30.375 9224-9389/com.example.android.displayingbitmaps E/GRPC: tcp_client_posix.c:173] failed to connect to
  'ipv4:127.0.0.1:12389': socket error: connection refused
```

5: Debug | TODO | 6: Logcat | Android Profiler | Terminal | 0: Messages | Event Log | Gradle Console



Depurar con mensajes LogCat

- 1 **Clear Logcat** : Haz clic en esta opción para borrar el registro visible.
- 2 **Scroll to the end** : Haz clic en esta opción para saltar a la parte inferior del registro y ver los últimos mensajes. Si luego haces clic en una línea del registro, se detendrá el desplazamiento de la vista en ese punto.
- 3 **Up the stack trace**  y **Down the stack trace** : Haz clic en estas opciones para navegar hacia arriba y hacia abajo por los seguimientos de pila del registro, y selecciona el nombre de archivo anterior o siguiente que aparece en las excepciones impresas. Este comportamiento es el mismo que se genera cuando haces clic en un nombre de archivo del registro.
- 4 **Use soft wraps** : Haz clic en esta opción para habilitar el ajuste de línea y evitar el desplazamiento horizontal. Sin embargo, es posible que las strings no separables sigan requiriendo un desplazamiento horizontal.
- 5 **Print** : Haz clic en esta opción para imprimir los mensajes de Logcat. Una vez que hayas seleccionado tus preferencias de impresión en el cuadro de diálogo que aparece, podrás elegir guardar los mensajes como PDF.
- 6 **Restart** : Haz clic en esta opción para borrar el registro y reiniciar Logcat. A diferencia del botón **Clear Logcat**, este botón recupera y muestra los mensajes anteriores del registro, de modo que es muy útil si Logcat no responde y no quieres perder los mensajes.
- 7 **Logcat header** : Haz clic en esta opción para abrir el diálogo **Configure Logcat Header**, donde puedes personalizar la apariencia de los mensajes de Logcat, por ejemplo, si quieres que se muestren la fecha y la hora.
- 8 **Screen capture** : Haz clic en esta opción para tomar una captura de pantalla.
- 9 **Screen record** : Haz clic en esta opción para grabar un video del dispositivo durante un máximo de 3 minutos.

3.3. Componentes de una aplicación Android



Componentes

En Android, el término "componente" se refiere a las partes fundamentales que componen una aplicación.

- Los componentes principales desempeñan un papel específico en la arquitectura de una aplicación Android y se dividen en:
 - Activity,
 - Views, ViewGroups
 - Layouts
 - Fragments
 - Service
 - ContentProvider.
 - BroadcastReceiver
 - Widget
 - Intent
- Los componentes **interactúan** entre sí a través del sistema de intenciones (Intent) y se pueden declarar en el archivo Manifest.

Actividades / Activities

Una **activity** en Android hace referencia al **punto de entrada de nuestra aplicación**, de manera que podemos decir que se relaciona con las **acciones** que puede realizar el **usuario**. Por lo tanto, cada una de las activities sería una pantalla que nosotros le presentamos al usuario, una pantalla de nuestra App.

Se compone de:

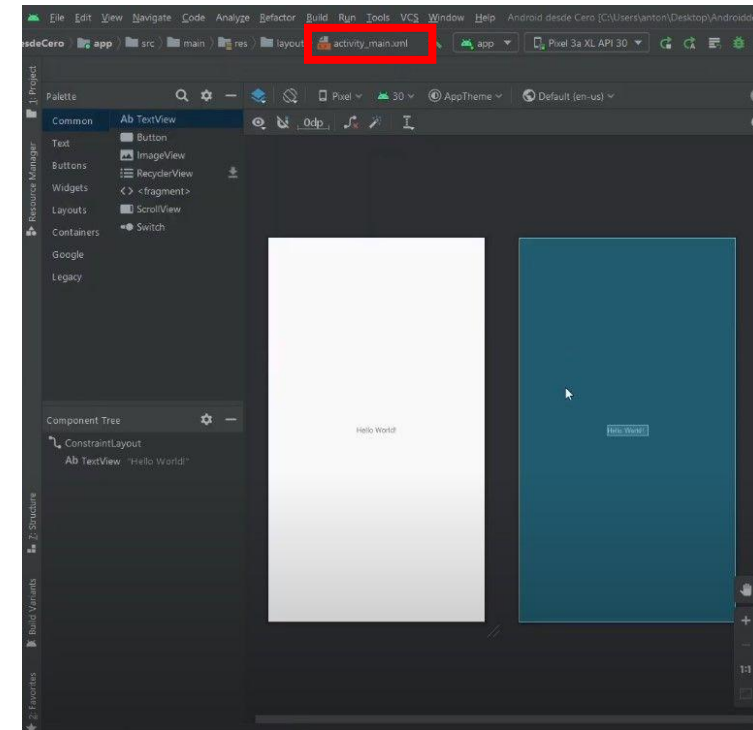
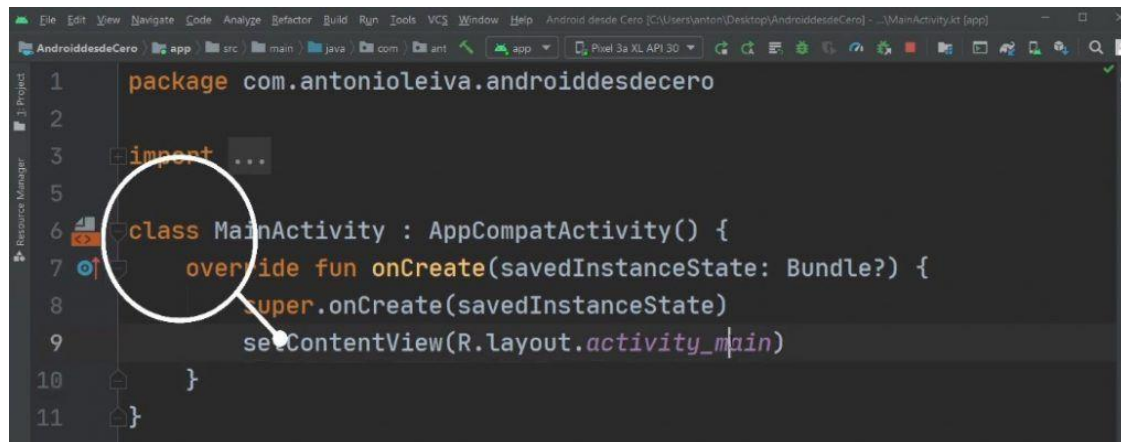
- ☐ **Una clase**, que normalmente extiende de AppCompatActivity. Es donde definimos el código de lo que queremos que haga la App.
- ☐ **Un layout**, que identifica la apariencia de la vista, el diseño. Tiene formato **XML**, pero se puede utilizar el diseñador para hacerlo más sencillo.
- ☐ **Una definición de su uso**, que se realiza en el AndroidManifest.xml.

Cada actividad generalmente corresponde a una sola tarea que el usuario puede realizar, como ver un mapa, enviar un mensaje de texto, reproducir música, etc.

Actividades / Activities

Partes de una Activity

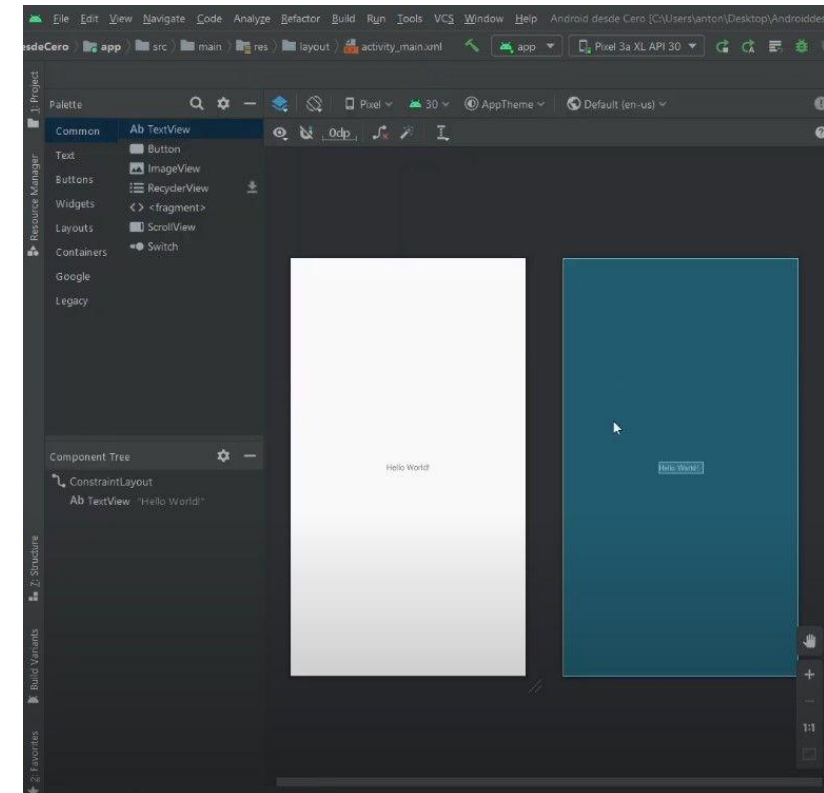
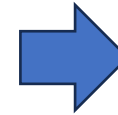
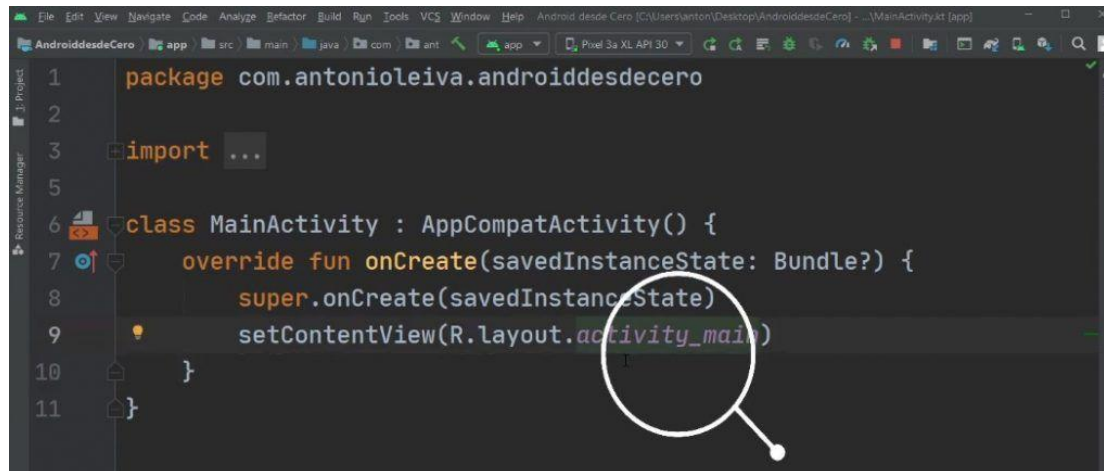
Hay dos cosas de las que debes estar pendiente sobre las Activity, la primera es que por lo general estará compuesta por **una clase y por un archivo XML**, este es un archivo con una estructura específica que nos identifica cuál es la forma que tiene esa pantalla.



Actividades / Activities

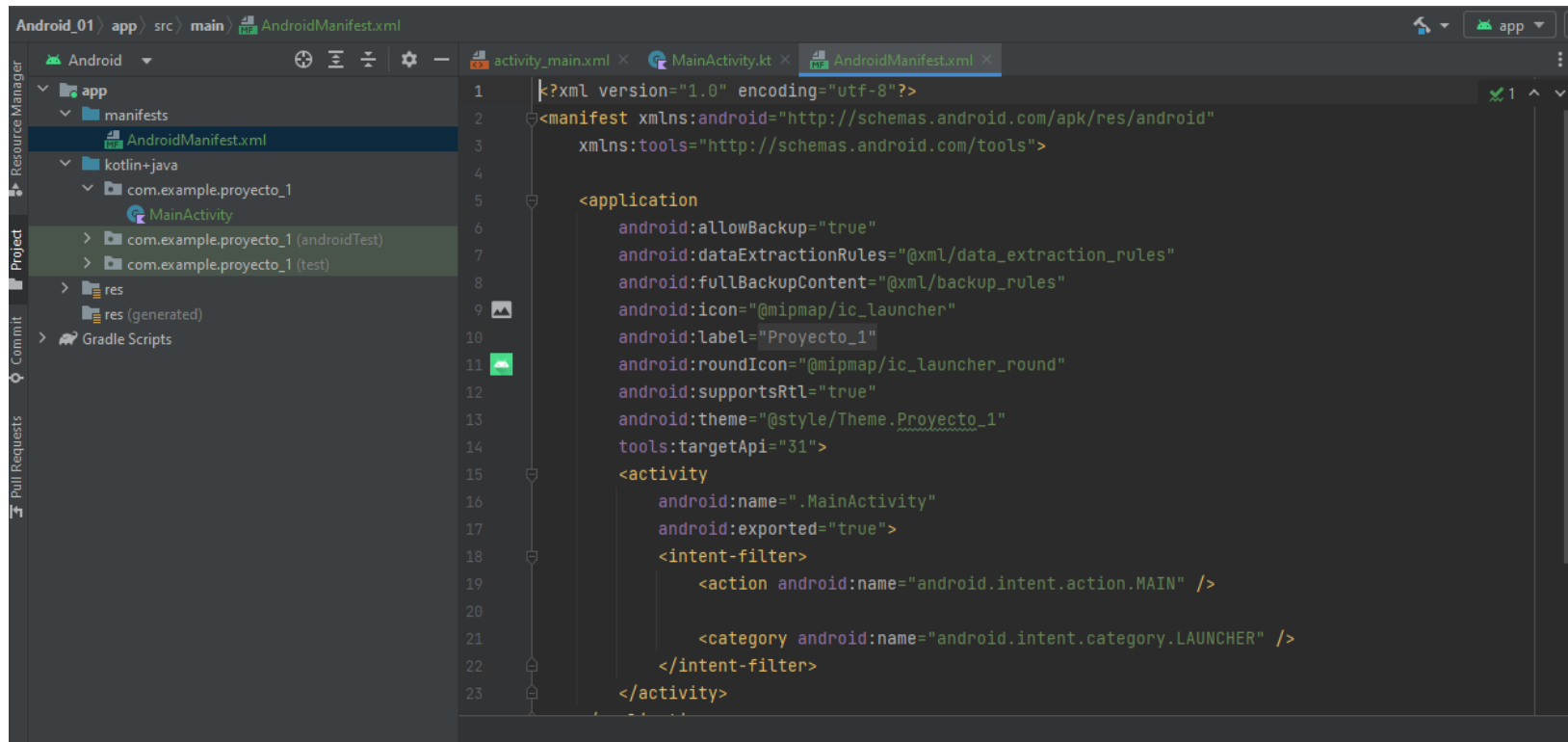
Partes de una Activity

También si entramos en `R.layout.activity_main` haciendo control + click en Windows nos mostrara de forma visual nuestra pantalla.



Actividades / Activities

Si nos vamos a la carpeta de **manifest** situada en la parte lateral izquierda y entramos en el **Android.Manifest.xml**, encontramos un archivo **XML** que es una **configuración de alto nivel** de lo que hace nuestra aplicación y aquí definimos diferentes cosas, como, por ejemplo, en la etiqueta **label** identificamos el nombre de la aplicación:



Actividades / Activities

En este archivo es donde se configura que es lo que va a hacer nuestra aplicación, y luego le tenemos que indicar cuáles son esas pantallas que va a utilizar esos puntos de entrada que mencionábamos que son las Activities; Así que mediante este pequeño XML se le identifica que esta Activity forma parte de esta aplicación; Si no añadimos la Activity aquí no vamos a tener forma de utilizarla dentro de nuestro proyecto:

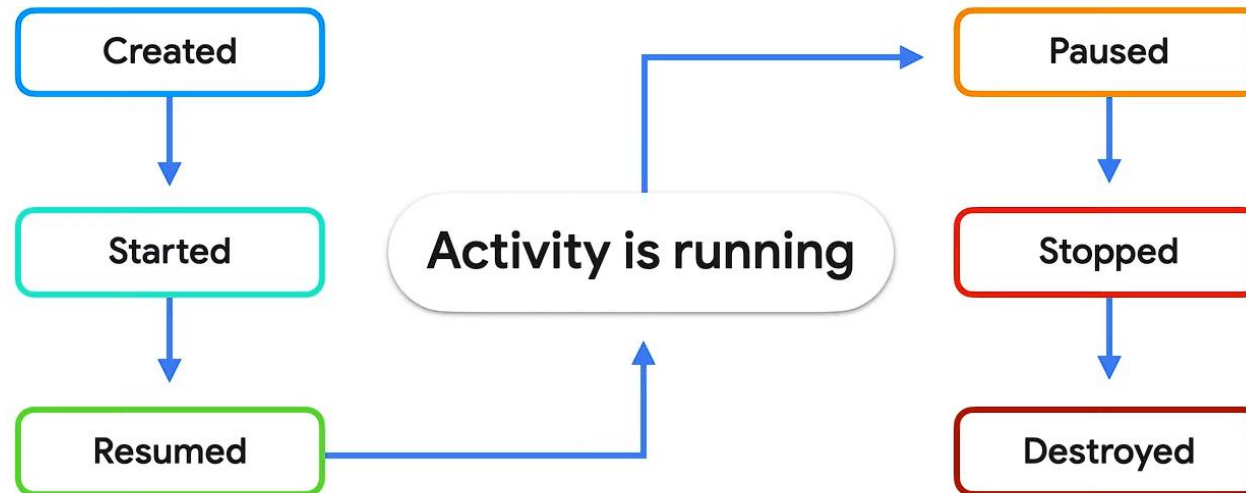
```
<activity
    android:name=".MainActivity"
    android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />

        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
```

Vamos a tener varias Activitys en nuestra aplicación, con estos valores identificamos que esta es la Activity Principal, para que cuando le demos al botón de arrancar sea la que nos va a lanzar primero.

Actividades / Activities

Ciclo de vida de una Activity (1/4)



- `onCreate()`
- `onStart()`
- `onResume()`
- `onPause()`
- `onStop()`
- `onDestroy()`

Actividades / Activities

Ciclo de vida de una Activity (2/4)

Es una serie de pasos por los que va pasando la pantalla en función del estado en el que se encuentra:

- **onCreate()**: es el primero paso, lo primero que se llama nada más crearse la Activity
- **onStart()**: cuando la activity se inicia. Se llama siempre que se llama a `onCreate()`, pero no al revés. `onStart()` se puede llamar cuando la activity pasa a un segundo plano y vuelve a estar visible
- **onResume()**: cuando la activity se reanuda. Se llama siempre que se llama a `onStart()`, pero no al revés. `onResume()` se puede llamar cuando algo se pone por encima de nuestra activity (por ejemplo, un diálogo), pero la activity se sigue viendo. Al recuperar el control, se llama a `onResume()`

Actividades / Activities

Ciclo de vida de una Activity (3/4)

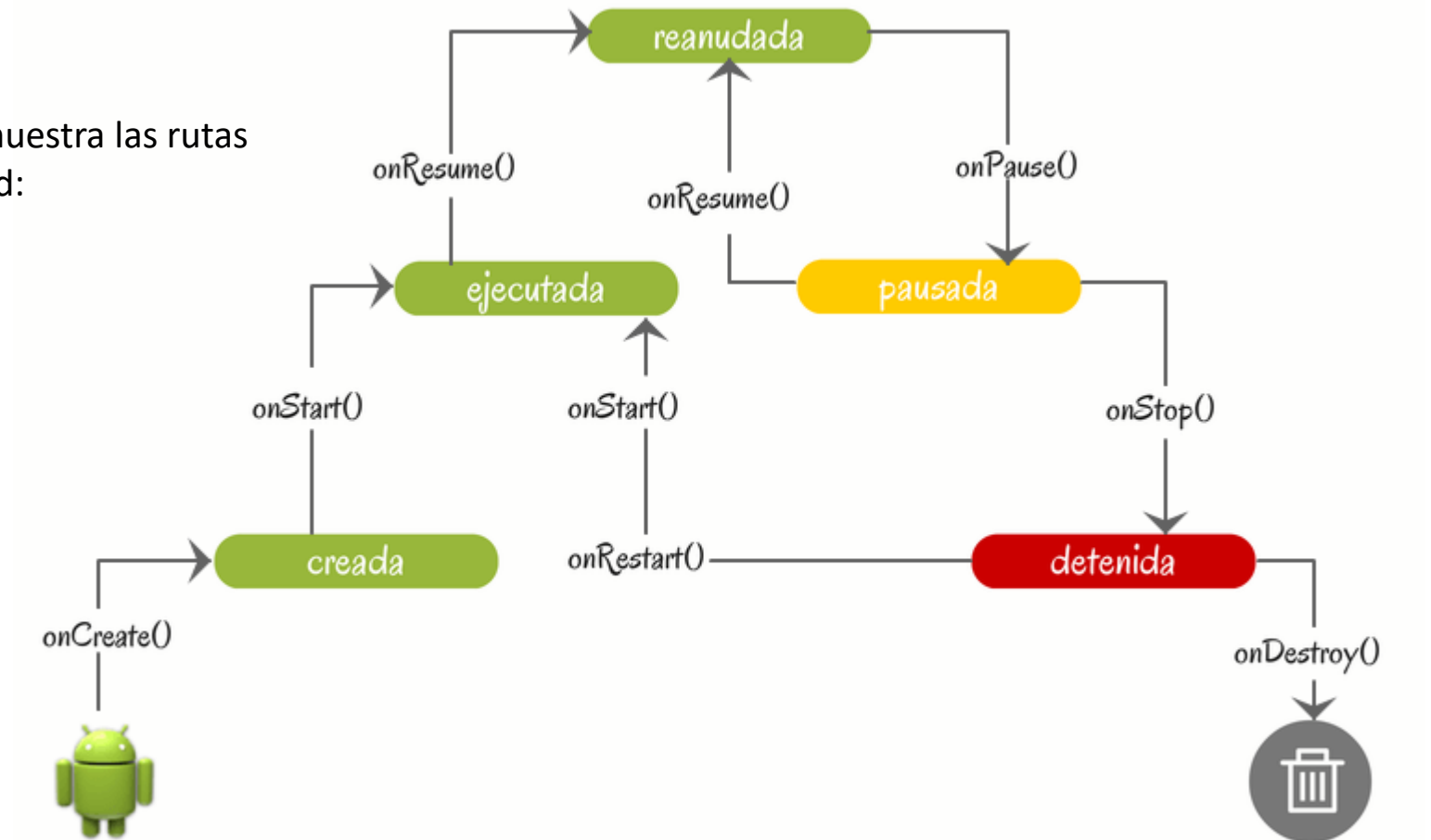
- **onPause()**: el opuesto de `onResume()`. Se llama cuando algo se pone por encima de la activity, pero esta se sigue viendo.
- **onStop()**: el opuesto de `onStart()`. Se llama cuando la App se va a segundo plano, o se lanza otra activity por encima.
- **onDestroy()**: el opuesto de `onCreate()`. Se llama cuando la Activity se destruye. Es lo último que se llamará antes de que la activity desaparezca.

En función de las tareas que estemos ejecutando, nos interesará lanzarlos en un paso del ciclo de vida o en otro. Se pararán en su opuesto.

Actividades / Activities

Ciclo de vida de una Activity (4/4)

El siguiente diagrama más completo, muestra las rutas de estado importantes de una actividad:

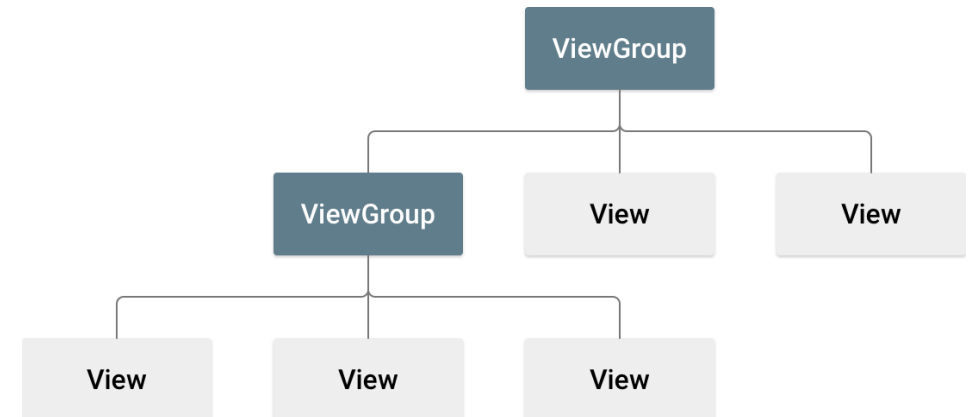


View, ViewGroups

Las vistas son los elementos que componen la interfaz de usuario de una aplicación: por ejemplo, un botón o una entrada de texto. Se debe tener en cuenta es que existen dos tipos de vistas principales:

- Las **Views** que son vistas únicas.
- Los **ViewGroups** que son conjuntos de vistas.

Un **ViewGroup** se dedica a organizar un conjunto de vistas dentro de la pantalla de una forma estructurada, conforme a unas reglas que le asignemos. Lo que este hará será acumular ese conjunto de vistas, y a su vez, está formado por vistas que pueden ser tanto Views como ViewGroups, de esta forma podríamos tener una jerarquía de vistas, es decir, unas dentro de otras.

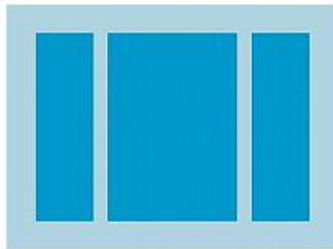


View, ViewGroups

Diseños comunes

Cada subclase de la clase ViewGroup proporciona una manera única de mostrar las vistas que anidas en ella. A continuación, se muestran algunos de los tipos de diseño más comunes:

Diseño lineal



Un diseño que organiza sus elementos secundarios en una sola fila horizontal o vertical. Si la longitud de la ventana supera la de la pantalla, crea una barra de desplazamiento.

Objeto RelativeLayout



Te permite especificar la ubicación de los objetos secundarios en función de ellos mismos (el objeto secundario A a la izquierda del objeto secundario B) o en función del elemento primario (alineado con la parte superior del elemento primario).

Vista web



Muestra páginas web.

Puedes anidar uno o más diseños dentro de otro diseño para crear el diseño de la IU, debes esforzarte por **mantener tu jerarquía de diseño** lo más sencilla posible. Tu diseño se procesa más rápido si tiene menos diseños anidados

View, ViewGroups

Diseños comunes

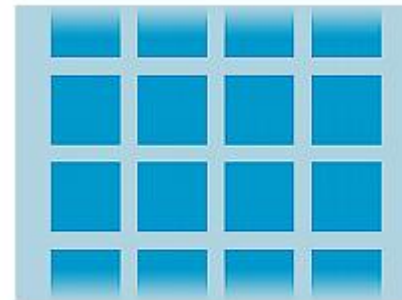
Para mostrar contenidos dinámicos entre fuentes de datos y nuestro diseño (por ejemplo, consultas a bases de datos) tienes que convertir cada entrada en una vista para agregar al diseño, para eso se utiliza un “adaptador de diseño”, con la subclase [AdapterView](#)

Vista de lista



Muestra una sola lista de columnas desplazable.

Vista de cuadrícula



Muestra una cuadrícula desplazable de columnas y filas.

Con la [AdapterView](#), adaptarás una [ListView](#) o [GridView](#), enlazando la instancia [AdapterView](#) con un **Adapter** (**ArrayAdapter**), que cogerá los datos de una fuente externa y crea una View que representa cada entrada de datos.

View, ViewGroups

Lo habitual es que utilicemos vistas que se extienden de una **vista**, que ya tienen creada la interacción y la parte visual por nosotros, por ejemplo:

- Si queremos pintar un texto, **TextView**.
- Si queremos pintar un botón, **Button**
- Si queremos mostrar una imagen, **ImageView**

Es decir, que tenemos una serie de componentes ya preparados que nos van a simplificar mucho la tarea de crear pantallas dentro de Android.

Otro componente, los **Layouts**, nos van a permitir **organizar esas vistas** dentro de sí mismos conforme a unas reglas (por ejemplo, de forma **lineal**, **en cuadrícula** o **indicando la posición absoluta** de cada vista).

Si estamos creando un formulario con una serie de campos, donde el usuario puede introducir información y luego un botón final, para luego enviar esa información a algún sitio; podríamos utilizar un Layout que coloque cada uno de los campos y los botones uno debajo del otro sin necesidad de que tengamos que preocuparnos de cómo se colocan. Normalmente se definen con **código XML**

Layout

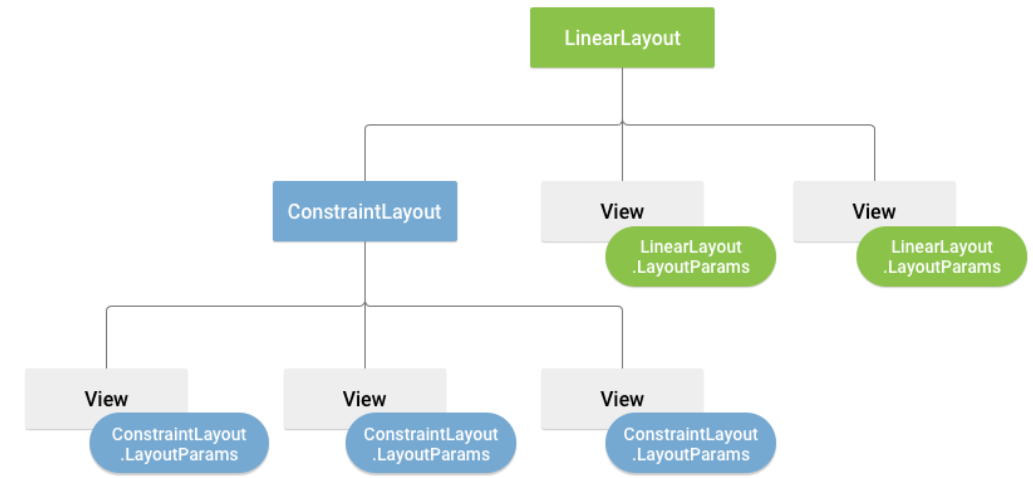
Los atributos de diseño XML denominados **layout_something** definen **parámetros de diseño** para el **objeto View** que son adecuados para el objeto ViewGroup en el que reside.

Todos los grupos de vistas incluyen un **ancho y una altura** (`layout_width` y `layout_height`), y cada vista debe definirlos. Muchos `LayoutParams` también incluyen **márgenes y bordes** opcionales.

Puedes especificar el ancho y la altura con medidas exactas, aunque no es recomendable hacerlo con mucha frecuencia. Generalmente, se usa una de estas constantes para establecer el ancho o la altura:

- **wrap_content** le indica a tu vista que modifique su tamaño conforme a las dimensiones que requiere su contenido.
- **match_parent** le indica a tu vista que se agrande tanto como lo permita su grupo de vistas superior.

En general, **no se recomienda** especificar el ancho ni la altura de un diseño con unidades absolutas como píxeles. En cambio, el uso de medidas relativas, como unidades de píxeles independientes de la densidad (dp), `wrap_content` o `match_parent`, es un mejor, ya que ayuda a que la app se visualice correctamente en distintos tamaños de pantallas.



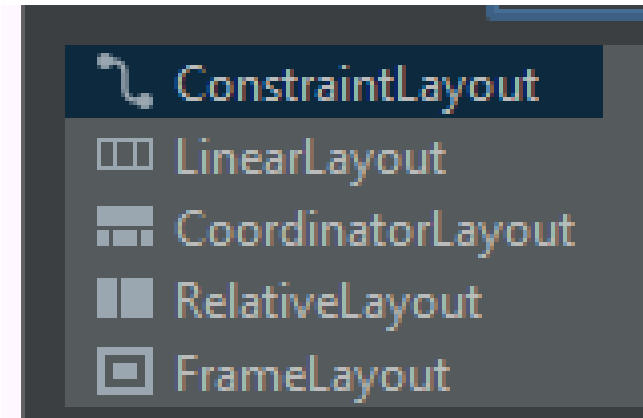
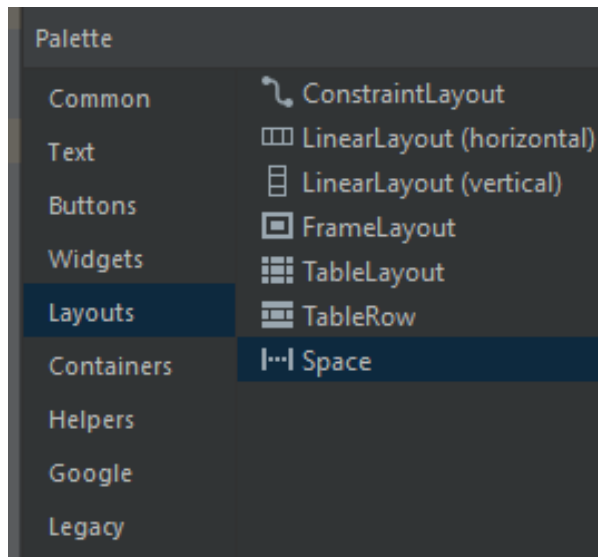
Layout

Por lo tanto, un **layout** define la estructura visual de mi aplicación, es un conjunto de vistas agrupadas de una forma determinada, indicando la manera en la que se pueden presentar los componentes gráficos de nuestra aplicación

- Tenemos diferentes alternativas de layout, algunas son:
- **ConstraintLayout:** Permite crear diseños complejos sin layouts anidados, este es el layout por defecto.
- **LinearLayout:** define agrupaciones en vertical u horizontal
- **TableLayout:** representación gráfica de una tabla, por filas y columnas, se usa el componente TableRow
- **GridLayout:** Similar al TableLayout, sin embargo, su uso no está restringido al uso de TableRow
- **RelativeLayout:** especifica dimensiones relativas para los componentes
- **FrameLayout:** Brinda una alternativa sencilla alineación de componentes.
- **ScrollView:** Permite usar un Scroll dependiendo de lo que queramos mostrar si hay contenido fuera de la pantalla
- **CoordinatorLayout:** Facilitar la implementación de efectos de desplazamiento, animaciones y comportamientos personalizados en una interfaz de usuario.

Layout

Configuración de Layouts en Android Studio desde el modo Diseño en la ventana de archivos XML:

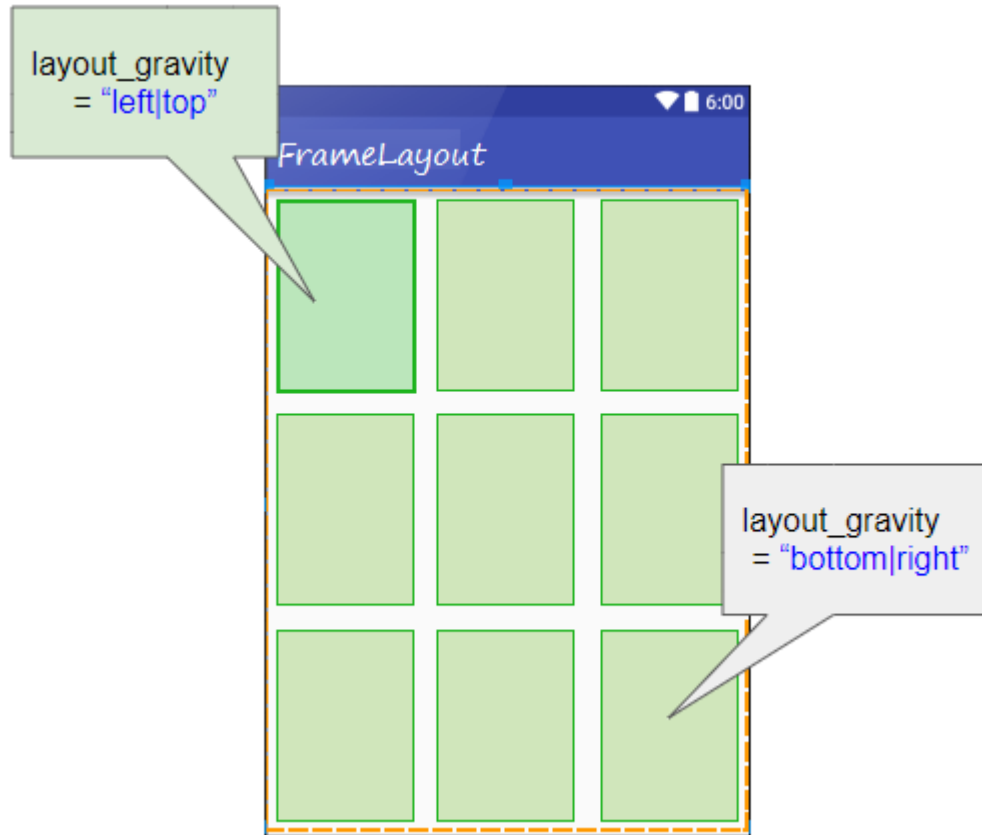


Tipos de Layouts desde ventana en ventana Component Tree

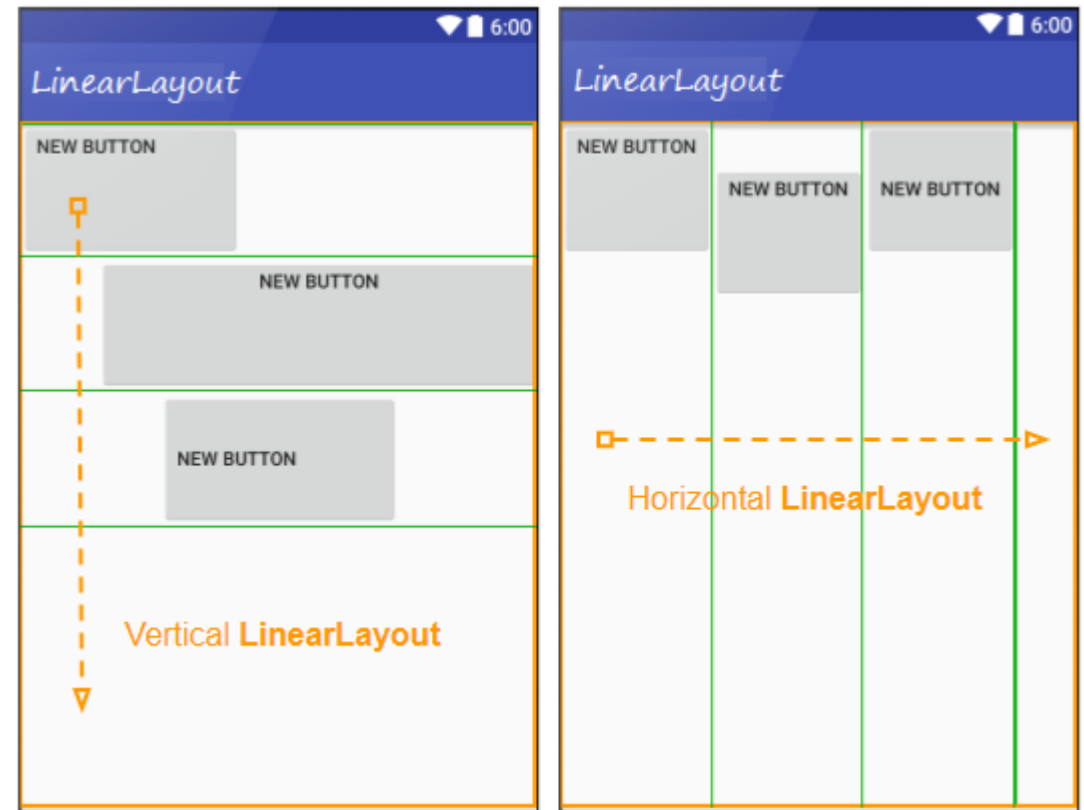
Tipos de Layouts desde ventana “Palette” en Android Studio.

Layout

FrameLayout



LinearLayout



Layout

TableLayout

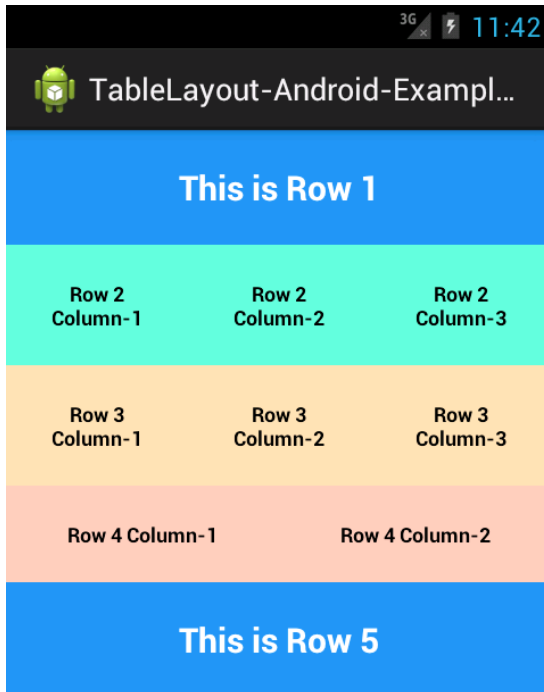
TableRow

TableRow

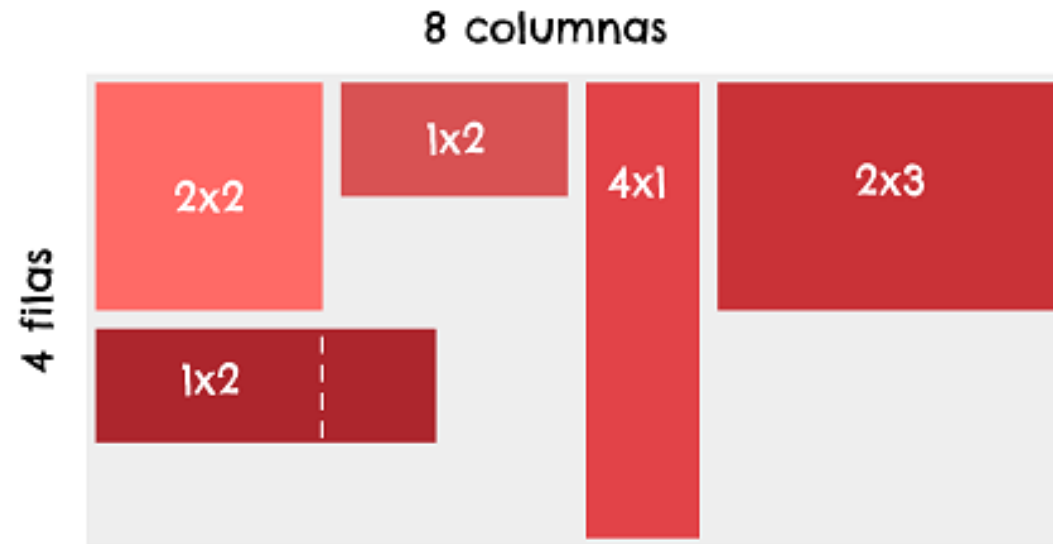
TableRow

TableRow

TableRow



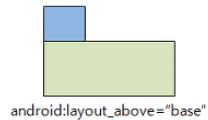
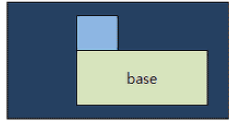
GridLayout



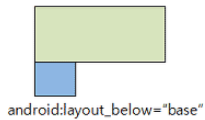
Tiene el fin de crear diseños en tabla complejos.

Layout

RelativeLayout



android:layout_above="base"



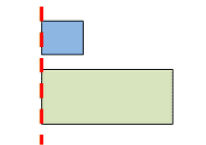
android:layout_below="base"



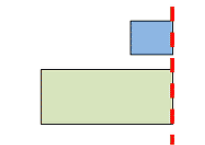
android:layout_toLeftOf="base"



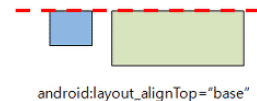
android:layout_toRightOf="base"



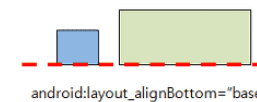
android:layout_alignLeft="base"



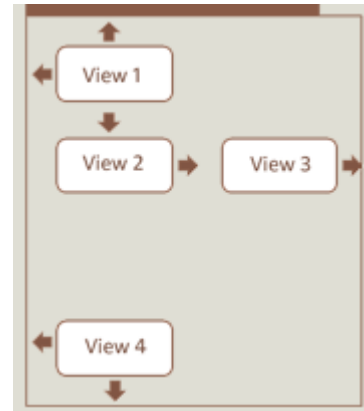
android:layout_alignRight="base"



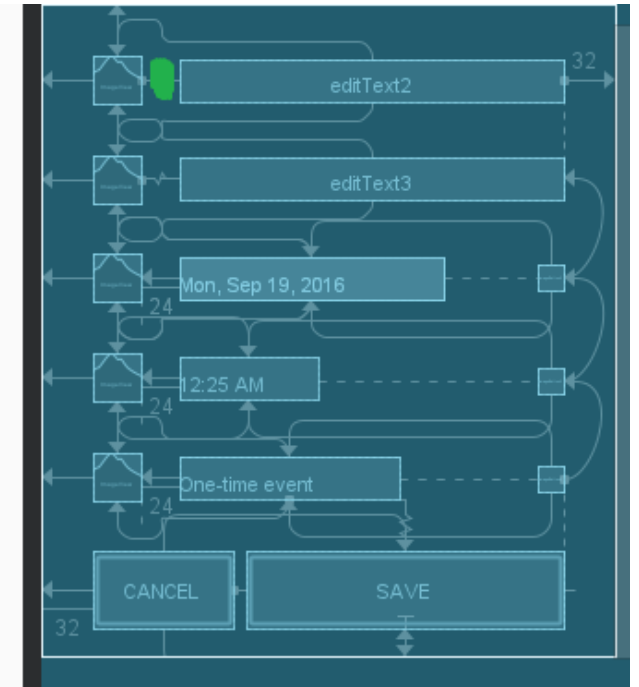
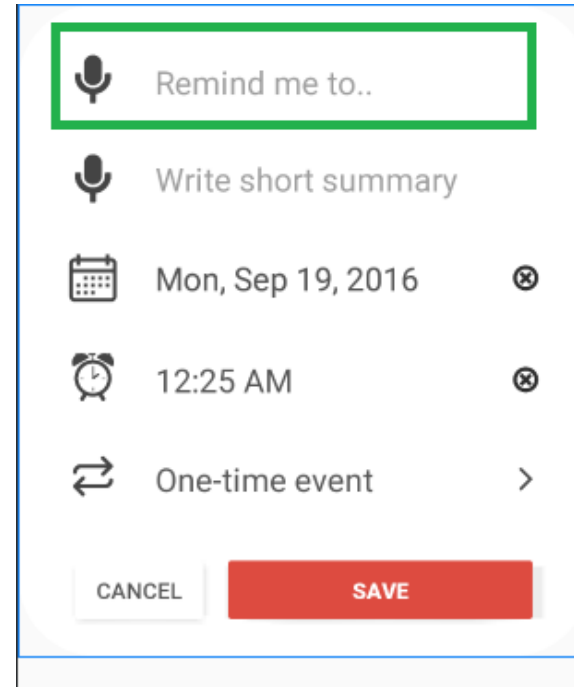
android:layout_alignTop="base"



android:layout_alignBottom="base"

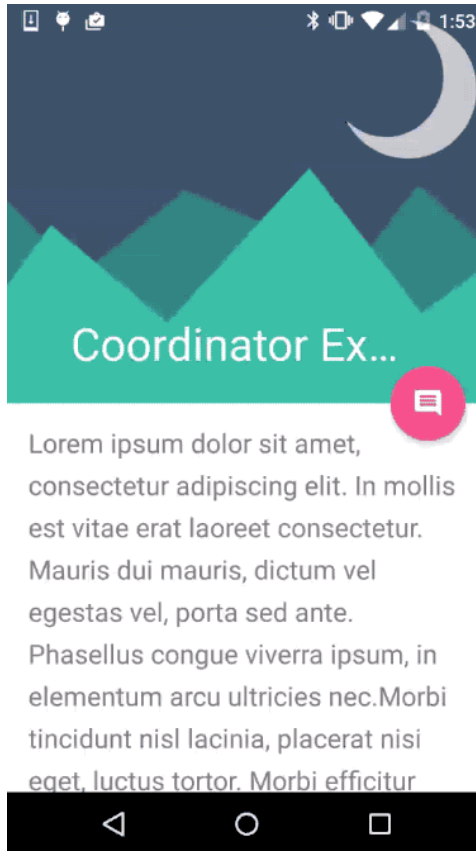


ConstraintLayout

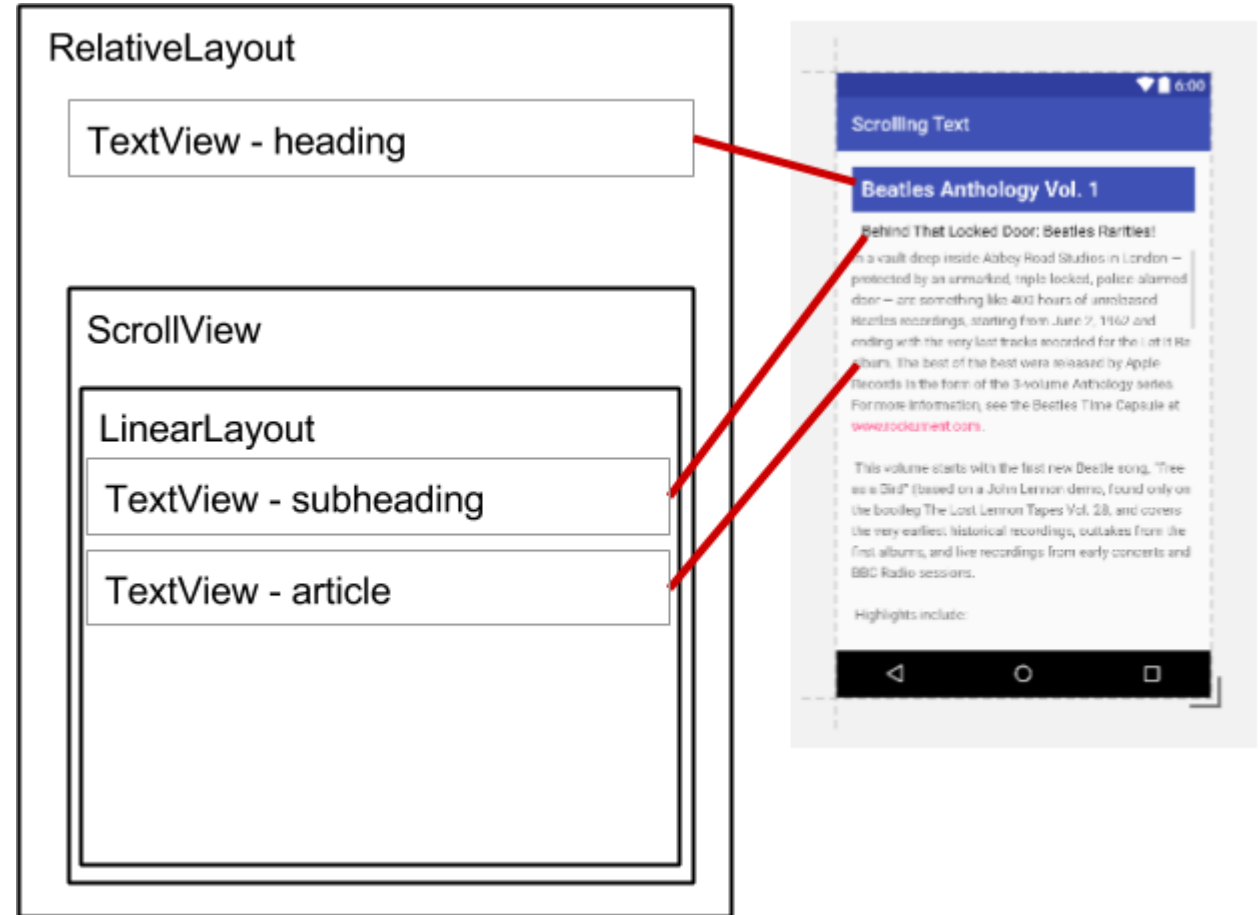


Layout

CoordinatorLayout



ScrollView



Layout

Unidades de medida

Dada la gran cantidad de dispositivos que utilizan Android como sistema operativo es necesario tener muy en cuenta las diferentes resoluciones de pantalla buscando que nuestra aplicación sea visualizada de la mejor manera sin importar el dispositivo en el que se encuentre instalada. En Android se tienen disponibles diferentes unidades de medida para brindar distintas alternativas para especificar el tamaño de un elemento dependiendo de la pantalla del dispositivo:

Nombre	Abreviatura	Equivalencia
Píxeles independientes de la densidad (Density-independent Pixels)	dp	1 dp = 1 pixel en una pantalla de 160 dpi (mdpi)
Píxeles independientes de la escala (Scale-independent Pixels)	sp	1 sp = 1 dp * preferencias del tamaño del usuario
Puntos	pt	1 pt = 1/72 de pulgada
Píxeles	px	1 px = 1 pixel de la pantalla
Milímetros	mm	1 mm = 1 milímetro
Pulgadas (Inches)	in	1 in = 1 pulgada = 25,4 mm

Se recomienda usar en todo momento **dp para los view** pues es independiente de la resolución o tamaño de las pantallas, sin embargo, para los **textos** debe usarse en todo momento **sp** pues brinda una mejor solución al momento de cambiar el tamaño del texto.

Los Fragments

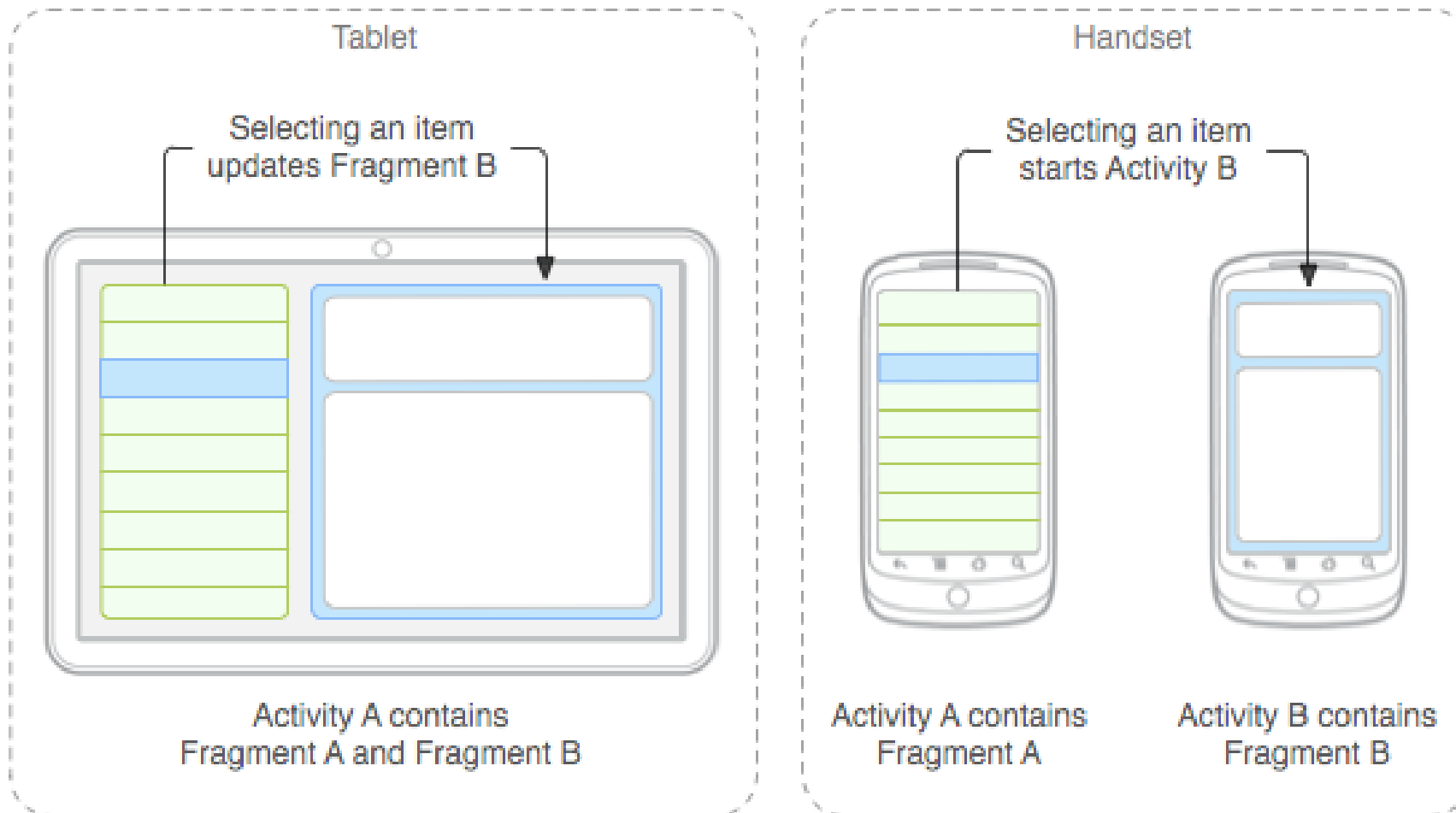
La llegada de las tabletas trajo el problema de que las aplicaciones de Android ahora deben soportar pantallas más grandes. Si diseñamos una aplicación pensada para un dispositivo móvil y luego la ejecutamos en una tableta, el resultado no suele resultar satisfactorio. Para solucionar esto se crearon los **fragments**.

Un fragment está formado por la unión de **varias vistas para crear un bloque funcional** de la interfaz de usuario. Una vez creados los fragments, **podemos combinar uno o varios fragments dentro de una actividad, según el tamaño de pantalla disponible**.

Cada fragment ha de implementarse en una **clase diferente**, la cual tiene una estructura similar a la de una actividad, pero con algunas diferencias. La primera es que esta clase tiene que extender Fragment. El ciclo de vida es muy parecido al de una actividad; sin embargo, dispone de unos cuantos eventos más, que le indican cambios en su estado con respecto a la actividad que lo contiene.

Y se pueden introducir **por código o desde XML**. Introducir un fragment desde XML es más sencillo, sin embargo, una vez introducido ya no podremos reemplazar el fragment por otro. Un fragment añadido desde XML será siempre estático. Si lo añadimos desde código, ganamos la posibilidad de intercambiar el fragment por otro.

Los Fragments

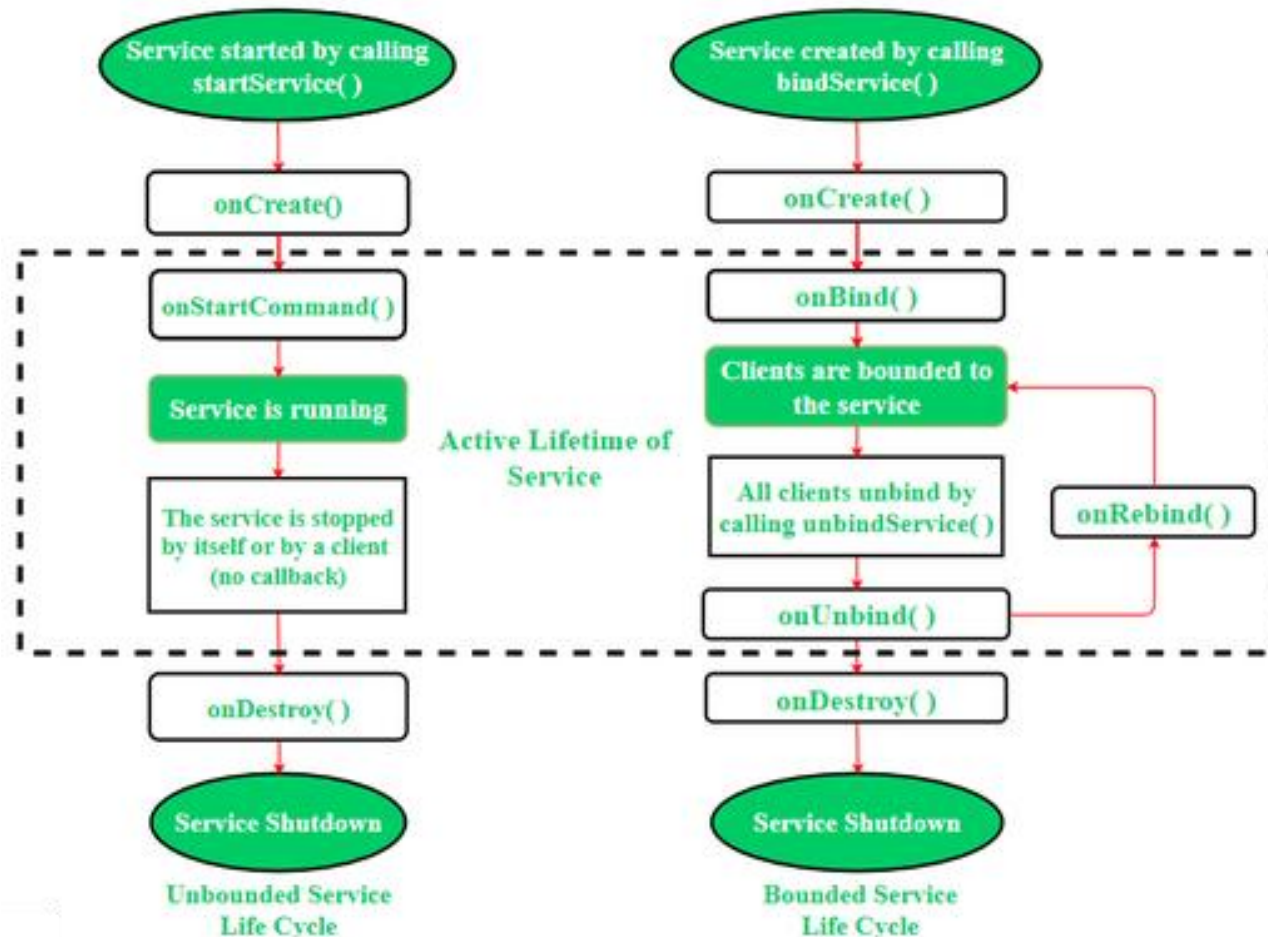


Service (Servicio)

Un servicio es un proceso que se ejecuta “detrás”, en segundo plano, sin la necesidad de una interacción con el usuario, ni interfaz visible. Es algo parecido a un demonio en Unix o a un servicio en Windows. Se utilizan cuando queramos tener en ejecución un código de manera continua, aunque el usuario cambie de actividad. En Android disponemos de dos tipos de servicios: servicios locales, que son ejecutados en el mismo proceso, y servicios remotos, que son ejecutados en procesos separados.

Se utiliza también para realizar operaciones prolongadas, como reproducir música en segundo plano, descargar archivos, o manejar solicitudes de red.

Service (Servicio)



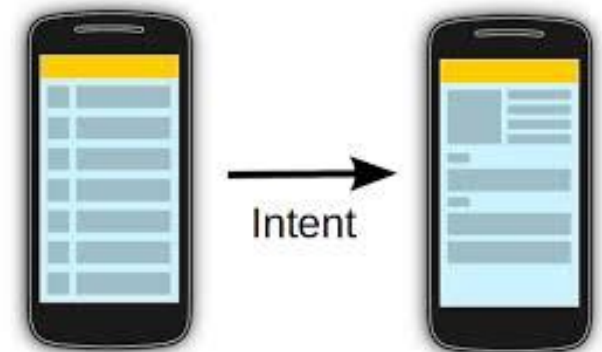
Intent (Intención)

Una intención representa la **voluntad de realizar alguna acción**; como realizar una llamada de teléfono, visualizar una página web. Se utiliza cada vez que queramos **lanzar una actividad, un servicio**, comunicarnos con un servicio, etc.

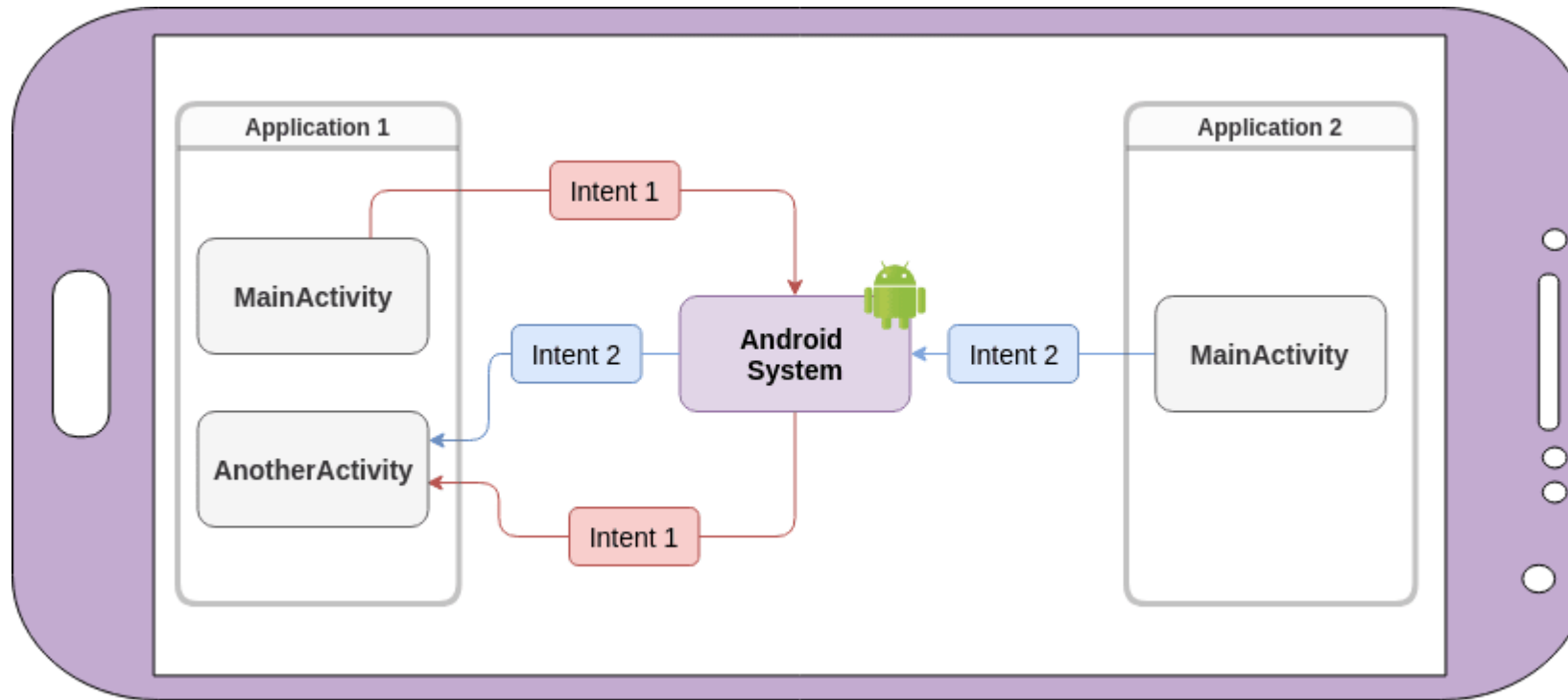
Los componentes lanzados pueden ser **internos o externos** a nuestra aplicación. También utilizaremos las intenciones para el intercambio de información entre estos componentes.

Por lo tanto, un **intent** será el elemento básico de comunicación entre los distintos componentes Android que estamos viendo. Se pueden entender como los mensajes o peticiones que son enviados entre los distintos componentes de una aplicación o entre distintas aplicaciones:

- Un Intent es un objeto que proporciona una descripción abstracta de una operación a realizar.
- Se utiliza para iniciar actividades, servicios, transmitir información entre componentes y más.



Intent (Intención)



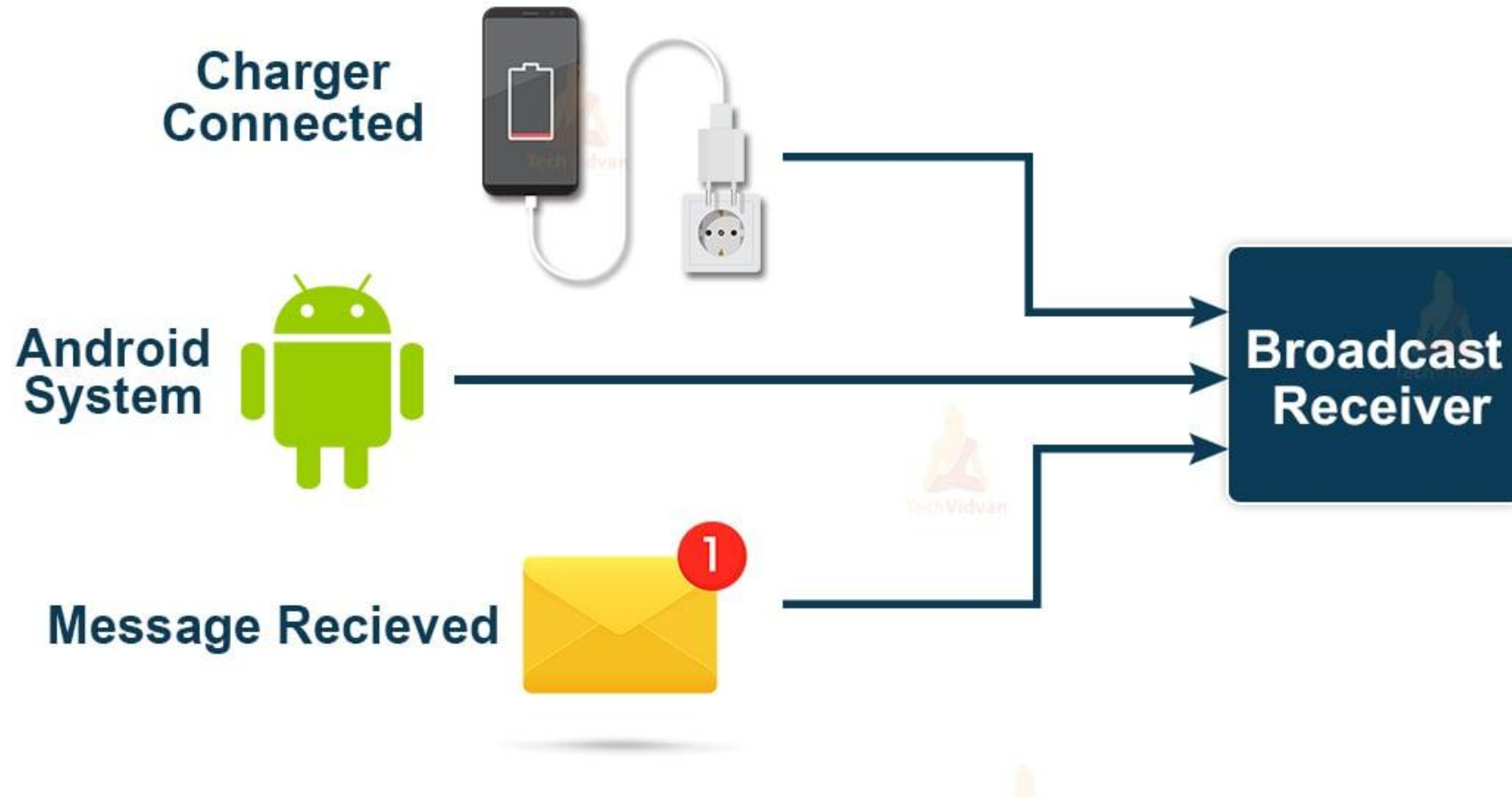
Broadcast Receiver (Receptor de anuncios)

Un broadcast receiver es un componente destinado a detectar y reaccionar ante determinados mensajes o eventos globales generados por el sistema (por ejemplo: “Batería baja”, “SMS recibido”, “Tarjeta SD insertada”, ...) o por otras aplicaciones (cualquier aplicación puede generar mensajes (intents, en terminología Android) de tipo broadcast, es decir, no dirigidos a una aplicación concreta sino a cualquiera que quiera escucharlo).

Por lo tanto:

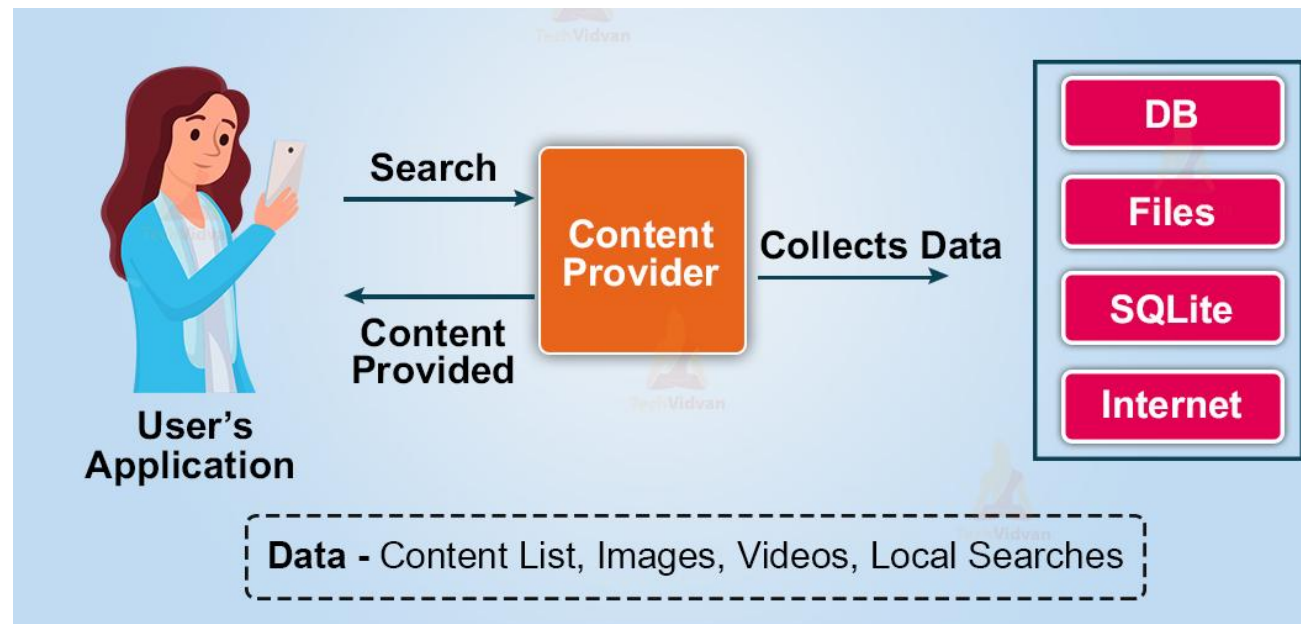
- Un BroadcastReceiver es un componente que responde a mensajes de difusión del sistema o de otras aplicaciones.
- Se utiliza para manejar eventos del sistema y realizar acciones en respuesta a ellos.

Broadcast Receiver (Receptor de anuncios)



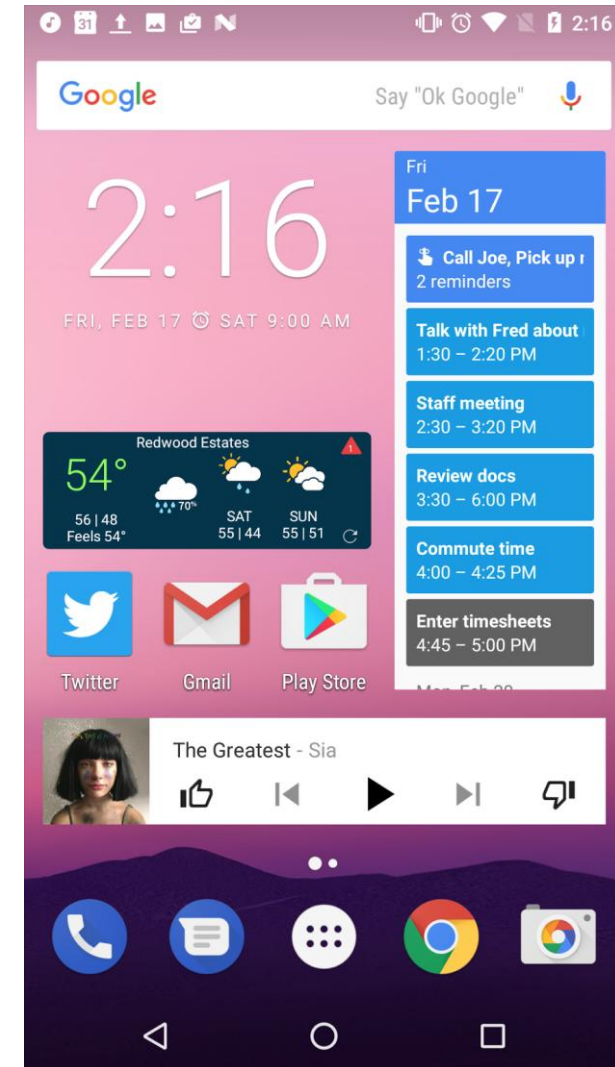
Content Provider (Proveedores de Contenido)

En muchas ocasiones las aplicaciones instaladas en un terminal Android necesitan compartir información. Android define un mecanismo estándar para que las aplicaciones puedan compartir datos sin necesidad de comprometer la seguridad del sistema de ficheros. Con este mecanismo podremos acceder a datos de otras aplicaciones, como la lista de contactos, o proporcionar datos a otras aplicaciones.



Widget

Los widgets son elementos visuales, normalmente interactivos, que pueden mostrarse en la pantalla principal (home screen) del dispositivo Android y recibir actualizaciones periódicas. Permiten mostrar información de la aplicación al usuario directamente sobre la pantalla principal.



En resumen

En resumen, en el desarrollo de aplicaciones Android, las **Activity** representan las pantallas de la interfaz de usuario, los **Componentes** son los elementos fundamentales de la aplicación, las **View** y **ViewGroup** son los elementos de interfaz de usuario y sus contenedores, los **Layouts** definen la estructura visual, los **Fragments** permiten la modularidad en la interfaz, y los **Services** realizan operaciones en segundo plano sin una interfaz de usuario visible. Estos conceptos trabajan juntos para construir aplicaciones Android efectivas y bien estructuradas.



Otros conceptos

1. Manifest (Manifiesto):

- El archivo de manifiesto (**AndroidManifest.xml**) proporciona información esencial sobre la aplicación al sistema Android.
- Contiene información sobre las actividades, servicios, permisos y configuraciones de la aplicación.

2. Resource (Recursos):

- Los recursos incluyen elementos como gráficos, cadenas, diseños y valores que se utilizan en la aplicación.
- Están organizados en carpetas específicas y se accede a ellos mediante identificadores generados automáticamente.

3. Adapter:

- Los **Adapter** son objetos que proporcionan datos a las vistas (por ejemplo, listas o cuadros de diálogo) y controlan cómo se muestran esos datos.
- Se utilizan comúnmente para vincular conjuntos de datos a elementos de la interfaz de usuario.

4. SQLite y ContentProvider:

- **SQLite** es una base de datos incorporada en Android que se utiliza para almacenar y gestionar datos locales.
- **ContentProvider** es un componente que permite compartir datos entre aplicaciones y acceder a ellos de manera segura.

5. Gradle:

- **Gradle** es una herramienta de construcción utilizada para automatizar y gestionar el proceso de construcción de la aplicación.
- Se utiliza para gestionar dependencias, compilar código y empaquetar la aplicación.

6. SharedPreferences:

- **SharedPreferences** se utiliza para almacenar datos primitivos en pares clave-valor de manera persistente.
- Es comúnmente utilizado para almacenar configuraciones y preferencias de la aplicación.

7. RecyclerView:

- **RecyclerView** es un widget de interfaz de usuario más avanzado que reemplaza a **ListView** y se utiliza para mostrar conjuntos grandes de datos de manera eficiente.

8. Retrofit y OkHttp:

9. **Retrofit** es una biblioteca para realizar solicitudes de red en Android de manera sencilla.

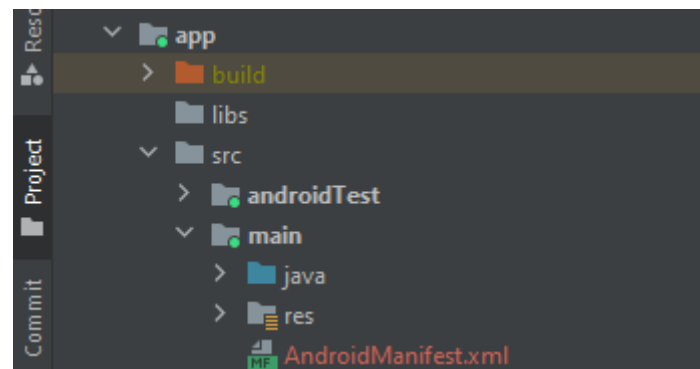
10. **OkHttp** es una biblioteca de cliente HTTP utilizada comúnmente junto con Retrofit para gestionar solicitudes y respuestas HTTP.

Archivo Android Manifest

El archivo **AndroidManifest** es un archivo de configuración que debe tener nuestro proyecto android, este archivo manifiesto describe información importante del proyecto.

En el se define información general como el nombre de la app, el icono, el tema y componentes de la aplicación como actividades, servicios, content providers, permisos de acceso a funcionalidades del sistema como la cámara, el internet, bluetooth entre otros, así como determinar cual es la actividad principal del sistema.

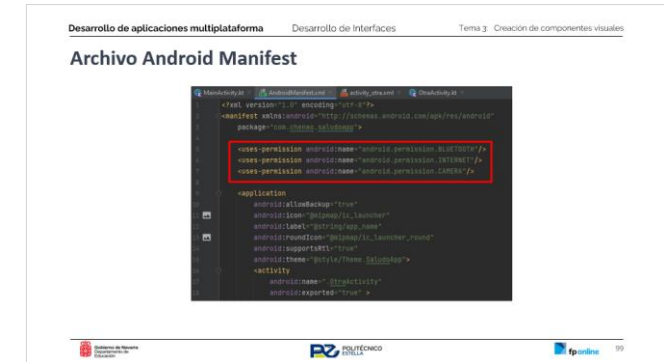
Este archivo lo podemos encontrar en el directorio manifest y tiene la siguiente estructura:



Archivo Android Manifest

Sus elementos básicos son:

- **icon:** Corresponde a la ruta donde se encuentra el icono de la aplicación.
- **label:** Ruta donde se encuentra el nombre de la aplicación.
- **theme:** Ruta con el estilo que usa la aplicación en cuanto al tema.
- **activity:** Corresponde a la parametrización de actividades que usa la aplicación

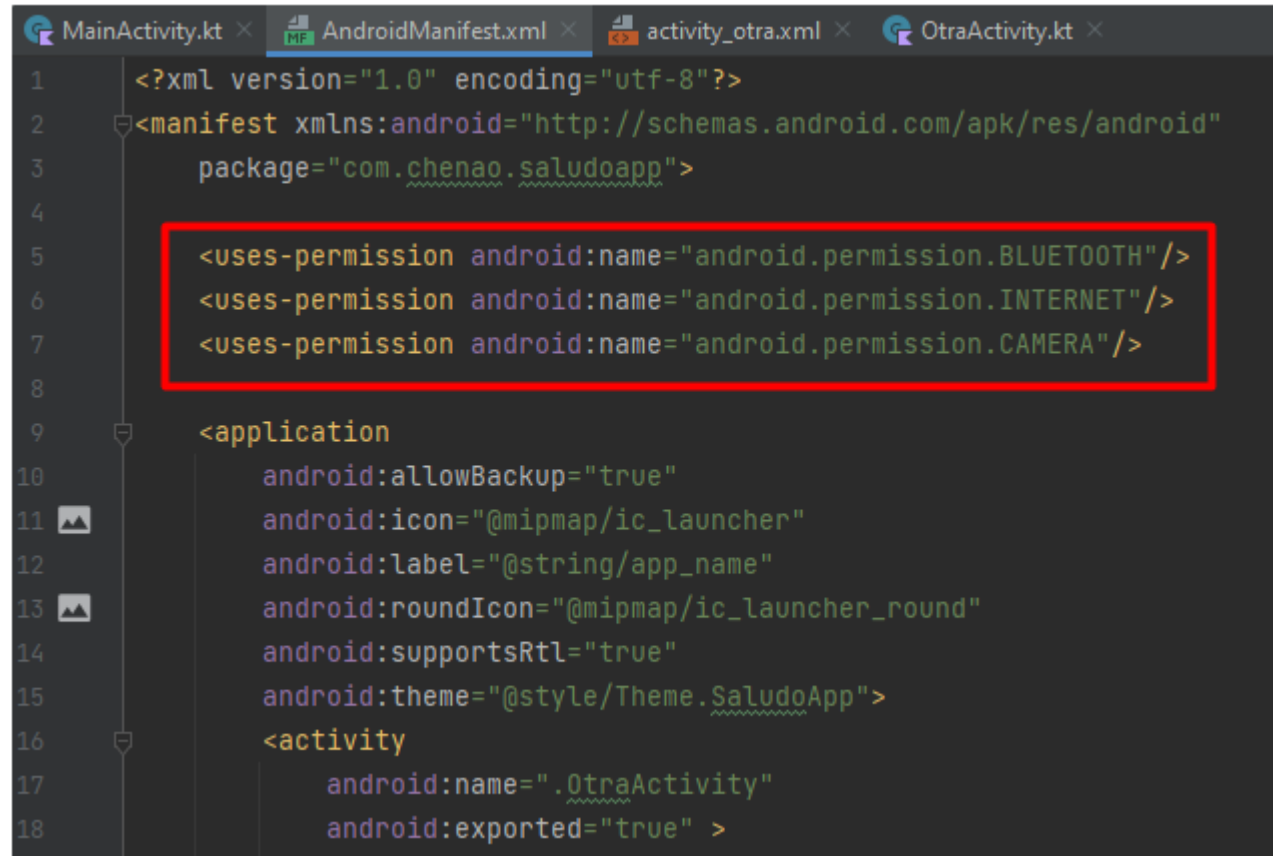


Por ejemplo, en el caso de las actividades, ya sabemos que una app se compone de actividades, estas corresponden a las pantallas de la aplicación y se divide en la parte gráfica y la parte de lógica del sistema, para que la app puede cargar dichas pantallas, **estas deben ser parametrizadas en el archivo manifest.**

Por ejemplo, algunas de las modificaciones más comunes pueden ser la posibilidad de definir cuál es la primera pantalla que se presenta, para hacer esto se tiene un bloque de código **<intent-filter>** y en él la propiedad que indica que esa va a ser la primera pantalla, si queremos podemos modificar este filtro y agregarlo en “OtraActivity”, cambiando también la propiedad “android:exported=”true”

Otra modificación al archivo muy común es la vinculación de permisos en la aplicación, aunque esto también se puede hacer por medio de lógica, desde aquí se pueden agregar permisos de uso de cámara, internet entre muchos otros.

Archivo Android Manifest



```
1  <?xml version="1.0" encoding="utf-8"?>
2  <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3    package="com.chenao.saludoapp">
4
5    <uses-permission android:name="android.permission.BLUETOOTH"/>
6    <uses-permission android:name="android.permission.INTERNET"/>
7    <uses-permission android:name="android.permission.CAMERA"/>
8
9    <application
10      android:allowBackup="true"
11      android:icon="@mipmap/ic_launcher"
12      android:label="@string/app_name"
13      android:roundIcon="@mipmap/ic_launcher_round"
14      android:supportsRtl="true"
15      android:theme="@style/Theme.SaludoApp">
16      <activity
17        android:name=".OtraActivity"
18        android:exported="true">
```

3.4. Kotlin



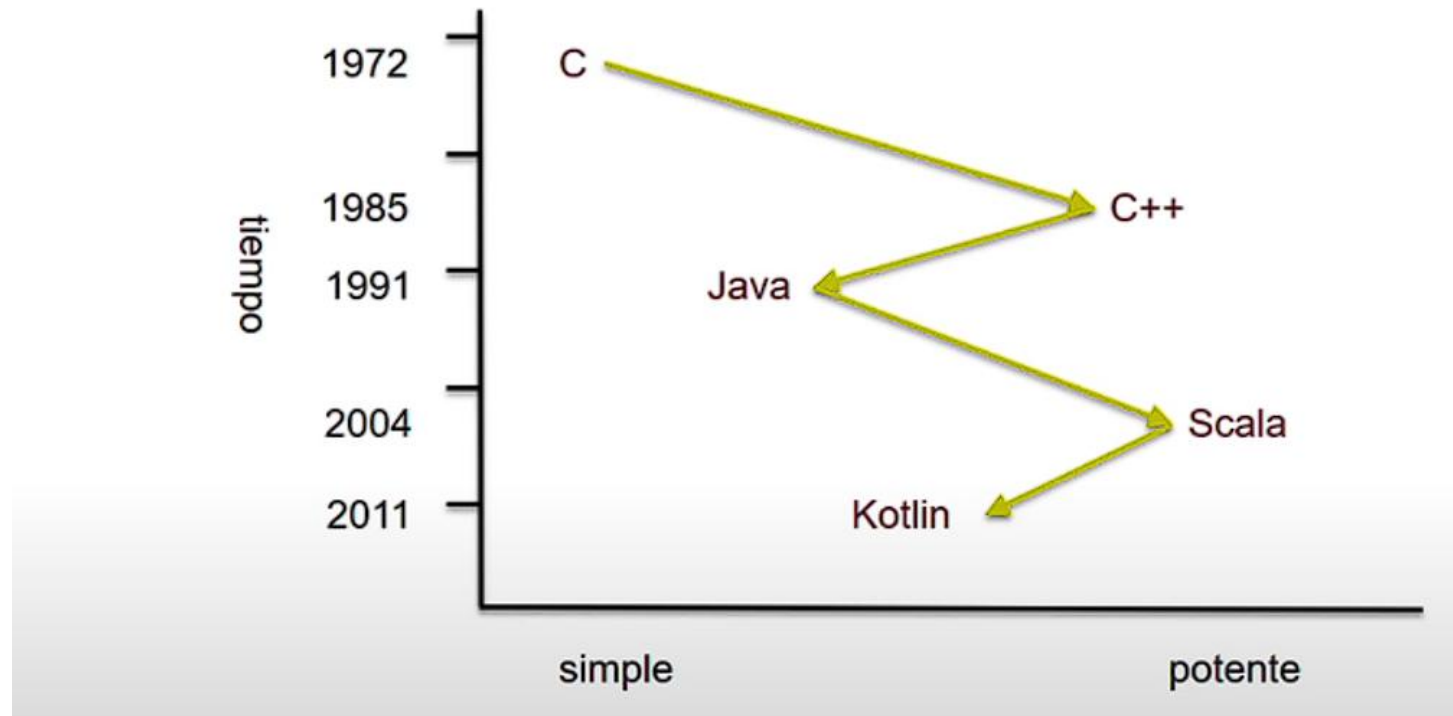
INTRODUCCIÓN A KOTLIN

- ❑ Kotlin fue creado en 2010 por JetBrains, (responsable de IntelliJ IDEA)
- ❑ Necesitaban un lenguaje más conciso que Java, que compilara en Java (JVM).
- ❑ En 2017, recibe un importante impulso al ser nombrado por Google como lenguaje oficial en Android al mismo nivel que Java.
- ❑ En la actualidad también puede ser compilado a JavaScript o incluso para iOS.
- ❑ Su nombre proviene de una isla en San Petersburgo, donde fue creado.

¿NO HAY YA SUFICIENTES LENGUAJES?

- ☐ Cada vez que aparece un nuevo lenguaje de programación, lo primero que pensamos es: ¿No hay ya suficientes lenguajes de programación?
- ☐ ¿Merece la pena aprender uno nuevo?
- ☐ Muchos desarrolladores de Android opinan que merece la pena aprender Kotlin.
- ☐ Revisando sus ventajas encontramos que este lenguaje es:
 - ✓ Más Conciso
 - ✓ Legible y expresivo
 - ✓ Código más seguro
 - ✓ Interoperable con Java

LOS ORÍGENES DE KOTLIN



EJEMPLOS: CONCISO

Lambdas

```
boton.setOnClickListener(  
    new View.OnClickListener(){  
        @Override  
        public void onClick(View v){  
            hazAlgo();  
        }  
    });
```

```
boton.setOnClickListener { hazAlgo() }
```

Definición de clases

```
public class Persona {  
    private String nombre;  
    public String getNombre() {  
        return nombre;  
    }  
    public void setNombre(String n){  
        nombre = n;  
    }  
}
```

```
class Persona {  
    var nombre: String = ""  
}
```

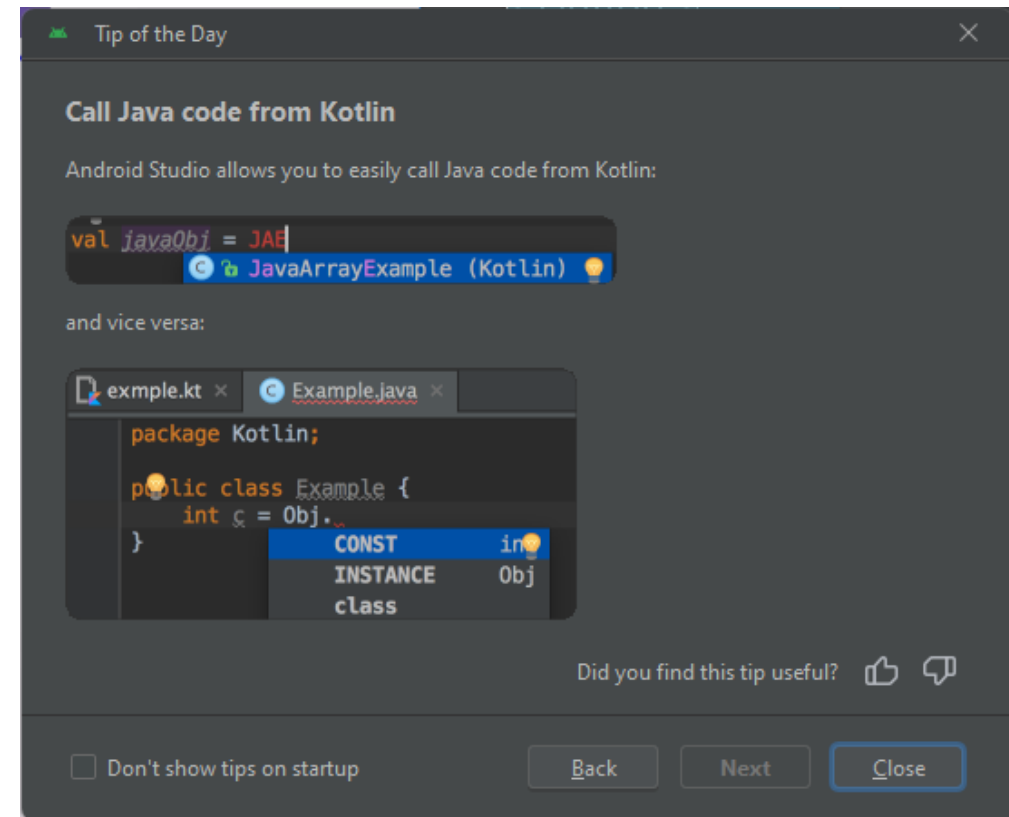
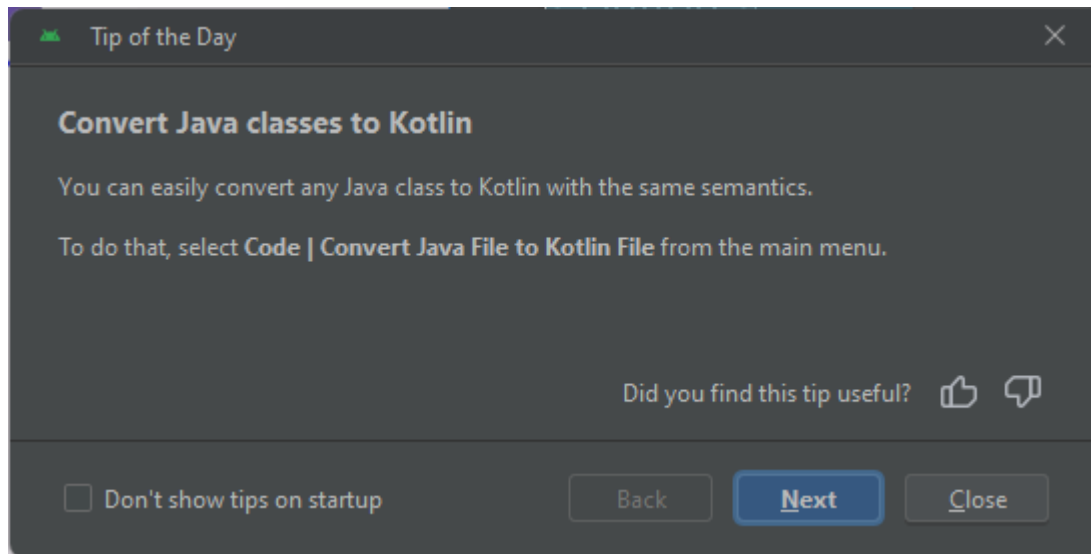
Adiós a findViewById():

```
TextView textView = (TextView)  
    findViewById(R.id.textView);  
textView.setText("¡Hola Mundo!");
```

```
textView.text = "¡Hola Mundo!"
```

EJEMPLOS: CONCISO

Herramientas de Android Studio



EJEMPLOS: LEGIBLE Y EXPRESIVO

Código conciso => más legible

Adiós a los getters y setters

<code>textView.setText("¡Hola Mundo!");</code>	<code>textView.text = "¡Hola Mundo!";</code>
--	--

Funciones con nombres de atributos:

<code>format("hola", true, true);</code>	<code>format("hola", normalizeCase = true, upperCaseFirstLetter = true);</code>
--	---

Estructuras `for` y `when` más expresivas.

Extiende la funcionalidad de una clase sin usar la herencia de forma legible y natural.

EJEMPLOS: INTEROPERABLE CON JAVA

Podemos llamar a cualquier clase o librería en Java.:

```
package com.example.mi_app;  
  
import android.util.Log;  
import android.view.View;  
import android.widget.EditText;  
...
```

```
package com.example.mi_app;  
  
import android.util.Log;  
import android.view.View;  
import android.widget.EditText;  
...
```

En un mismo proyecto, unos desarrolladores pueden estar escribiendo unas partes en Java y otras en Kotlin.

La migración puede ser gradual.

EJEMPLOS: DECLARACIÓN DE VARIABLES

La declaración de variables utiliza una sintaxis diferente a Java. Puedes apreciar cómo es necesario utilizar la palabra reservada `var`:

```
int i = 1;  
String s = "Hola";  
double d;
```

```
var i: Int = 1  
var s: String = "Hola"  
var d: Double
```

```
var i = 1  
var s = "Hola"  
var d: Double
```

Como se muestra en la tercera columna, el tipo de una variable puede ser omitido en caso de que pueda deducirse del contexto. Esto no quiere decir que una variable puede cambiar de tipo más adelante.

Existen dos tipos de variables en Kotlin, mutables (**var**) e inmutables (**val**). Para diferenciarlas iniciaremos **var** o **val**:

var (variable mutable)

Se utiliza para declarar variables que pueden ser modificadas después de su inicialización.
Puede cambiar su valor durante el tiempo de vida de la variable.

val (variable immutable)

Se utiliza para declarar variables cuyo valor no puede ser cambiado después de su inicialización.
Una vez que se asigna un valor a una variable `val`, no se puede modificar.

Vocabulario de Aspectos básicos de Android en Kotlin

CURSO BÁSICO

<https://developer.android.com/courses/android-basics-kotlin/course?hl=es-419>

Guía rápida de Kotlin para conocer los aspectos básicos de Android en Kotlin

<https://developer.android.com/courses/android-basics-kotlin/kotlin-quick-guide/?hl=es-419>

Introducción a Android Studio

<https://developer.android.com/studio/intro?hl=es-419>

Conceptos de programación

- **Clase:** una clase vendría a ser una representación de un elemento de la vida real.
- **Herencia:** es la capacidad que tiene una clase hijo de tener las propiedades o métodos del padre, así como en la vida real, cuando heredamos el color de ojos de nuestro padre, altura, capacidades físicas y que nosotros podemos extender más allá de lo que nuestro padre nos heredó.
- **Override:** cuando queremos cambiar una función de la clase padre hace uso de override para agregar la funcionalidad deseada.
- **Función:** trozo de código al que podemos llamar para que se ejecuta una acción o estados, que son valores de esa clase, que puede almacenar para luego hacer cosas con ellos. Podemos ejecutar llamando al nombre de esa función, de tal forma que podemos guardar cosas más complicadas en una única función y esa función la podemos llamar desde distintos sitios y reutilizar esa parte del código.
- **Variable:** una variable es un elemento que guarda un valor de forma temporal.
- **Objetos:** De una única clase se pueden generar un número ilimitado de objetos. Normalmente a estos **objetos** se les llama instancias específicas de una clase, de tal forma que, si los objetos tienen distintos estados, por ejemplo: una clase Persona la cual tendrá un nombre y una edad. Teniendo el nombre y la edad de esa clase podemos crear dos objetos, un objeto 1 del tipo persona y otro objeto 2 del tipo persona también, por ende, se puede modificar el estado de cada objeto sin afectar al otro.

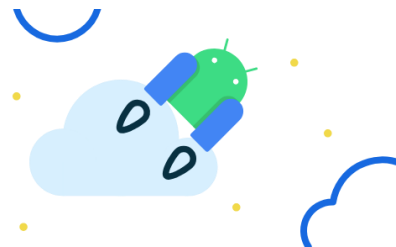
Android Jetpack

Android Jetpack

Android Jetpack es un **conjunto de bibliotecas, herramientas y guías** arquitectónicas que facilitan la creación de excelentes aplicaciones de Android.

El objetivo de Jetpack es acelerar y simplificar el proceso de desarrollo de aplicaciones. Maneja actividades como tareas en segundo plano, navegación y administración del ciclo de vida, por lo que puedes eliminar códigos repetitivos y centrarte en lo que hace que tu aplicación. AdemEl objetivo de Jetpack es acelerar y simplificar el proceso de desarrollo de aplicacionesás estos componentes permiten tener menos bloqueos y menos uso de memoria.

Es compatible tanto con Java como con Kotlin, pero se alinea especialmente bien con Kotlin



[Android Jetpack](#)

[Jetpack Compose, el kit de herramientas para el desarrollo de IU de apps - Android Developers](#) | [Jetpack Compose](#) | [Android Developers](#)

Android Jetpack

- **Arquitectura:** Proporciona bibliotecas y pautas para diseñar aplicaciones Android de manera robusta y mantenible. Incluye componentes como ViewModel, LiveData, Room (para trabajar con **bases de datos**), Navigation (para facilitar la **navegación entre pantallas**) y **Data Binding** (para vincular datos directamente a la interfaz de usuario).
- **Comportamiento del usuario:** Incluye bibliotecas que simplifican la implementación de patrones de diseño de interfaz de usuario comunes. Por ejemplo, ViewPager2 para implementar deslizamientos entre pantallas, RecyclerView para listas eficientes y DrawerLayout para crear menús de navegación.
- **Fundamentos del sistema:** Proporciona componentes y herramientas esenciales para el desarrollo de aplicaciones, como AppCompat (para garantizar la compatibilidad con versiones antiguas de Android), Palette (para extraer colores de imágenes) y la biblioteca de animaciones.
- **Herramientas:** Incluye herramientas que mejoran la eficiencia del desarrollo, como Android KTX (extensiones de Kotlin para Android), Navigation Editor (en Android Studio) y Benchmarking.

Android Jetpack

Cómo usar la biblioteca de Jetpack en tu app

Abre el archivo settings.gradle y agrega el repositorio google() en el bloque dependencyResolutionManagement { repositories {...}} como se muestra a continuación:

```
dependencyResolutionManagement {  
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)  
    repositories {  
        google()  
        jcenter()  
    }  
}
```

Luego, puedes agregar componentes de Jetpack, como componentes de arquitectura (por ejemplo LiveData y ViewModel), en el archivo build.gradle del módulo, como se muestra a continuación:

```
dependencies {  
    val lifecycle_version = "2.2.0"  
  
    implementation("androidx.lifecycle:lifecycle-livedata-ktx:$lifecycle_version")  
    implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:$lifecycle_version")  
    ...  
}
```

Android Jetpack

Cómo usar la biblioteca de Jetpack en tu app

Las bibliotecas de Jetpack se envían por separado del SO Android, por lo que sus actualizaciones pueden realizarse de forma independiente y con mayor frecuencia.

Se pueden usar por separado o combinadas para abordar las diferentes necesidades de tus apps.

- ✓ **WorkManager** para tus necesidades de programación en segundo plano
- ✓ **Room** para tener almacenamiento persistente de datos
- ✓ **Navigation** para administrar el flujo de navegación de tu aplicación
- ✓ **CameraX** para las necesidades de cámara de tu app
- ✓ Consultar la **Descripción general** de todas las bibliotecas de Jetpack

Las bibliotecas de Jetpack se publican en el espacio de nombres de androidx

Bibliotecas de Jetpack por tipo

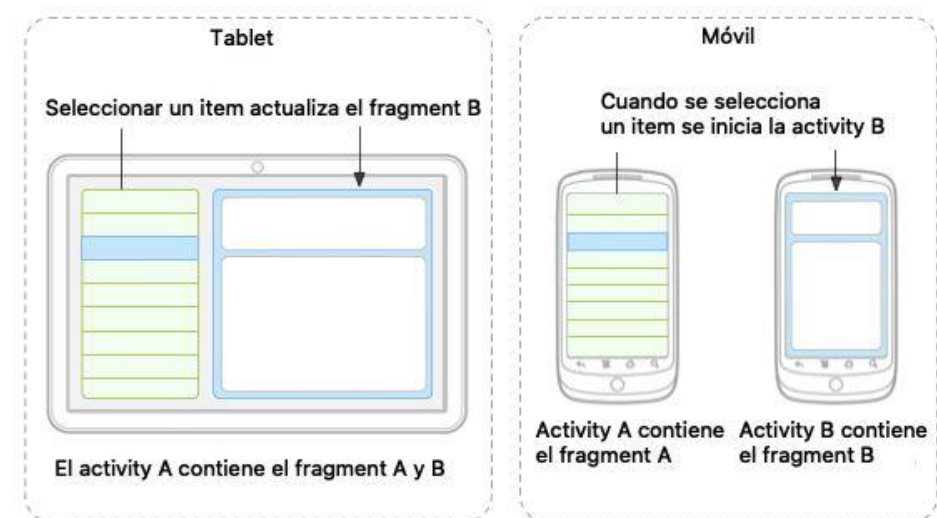
Fragmentos

Fragmentos

Los fragments, representan un componente básico a la hora de desarrollar una app.

Un fragment es una variante de la activity, pero con menos potencia, pero más manejable, es decir, su comportamiento es muy similar (tienen un layout y una clase para programar) pero tienen que estar dentro de una activity. La diferencia reside en que este componente es más reutilizable, y se puede añadir junto a más fragments en una misma activity.

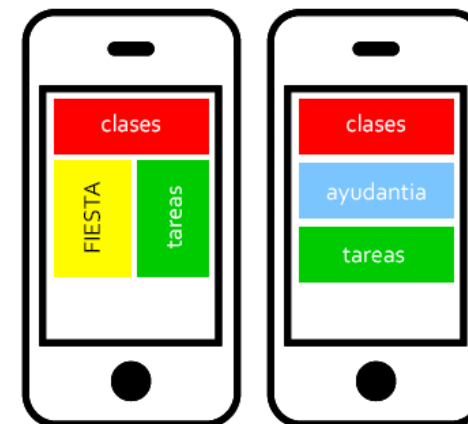
Su popularidad empezó con la moda de las tablets, eran pantallas muy grandes que permitían aprovechar más los fragments añadiendo varios en una sola activity, mostrando más información que en un móvil.



Fragmentos

Un Fragmento representa un comportamiento o una parte de la interfaz de usuario en una `FragmentActivity`:

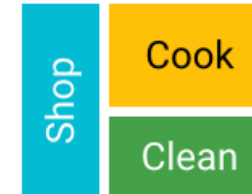
- ❑ Es una sección «modular» de interfaz de usuario embebida dentro de una **actividad anfitriona**, el cual permite versatilidad y optimización de diseño. Se trata de **miniactividades** contenidas dentro de una actividad anfitriona, manejando su propio diseño (un recurso layout propio) y ciclo de vida.
- ❑ Puedes **combinar varios fragmentos** en una sola actividad para crear una IU multipanel y volver a usar un fragmento en diferentes actividades.



Fragmentos

- ✓ **Modularidad** : divide el código de actividad complejo en fragmentos para una mejor organización y mantenimiento.
- ✓ **Reutilizabilidad** : colocar comportamientos o partes de la interfaz de usuario en fragmentos que puedan compartir múltiples actividades.
- ✓ **Adaptabilidad**: representar secciones de una interfaz de usuario como diferentes fragmentos y utilizar diferentes diseños según la orientación y el tamaño de la pantalla.

Modularity



Reusability



Adaptability



Fragmentos

Características:

- ☐ Puedes pensar en un fragmento como **una sección modular de una actividad** que tiene un ciclo de vida propio, que recibe sus propios eventos de entrada y que puedes agregar o quitar mientras la actividad se esté ejecutando (algo así como **una "subactividad" que puedes volver a usar en diferentes actividades**).
- ☐ Un fragmento siempre debe estar alojado en una actividad y **el ciclo de vida del fragmento** se ve afectado directamente por el ciclo de vida de la actividad anfitriona. Por ejemplo, cuando la actividad está pausada, también lo están todos sus fragmentos, y cuando la actividad se destruye, lo mismo ocurre con todos los fragmentos.
- ☐ Cuando se agrega un fragmento como parte de la presentación de la actividad, se encuentra en **ViewGroup** dentro de la jerarquía de vistas de la actividad y el fragmento define su propio diseño de vistas.
- ☐ Puedes insertar un fragmento en el diseño de la actividad declarando el fragmento en **el archivo de diseño (un archivo XML)**, como elemento <fragment>, o desde el código de tu aplicación agregándolo a un archivo existente ViewGroup.